Documentação do SGC / Agilis

Sistema de acompanhamento semanal dos Centros do Departamento de Planejamento, Orçamento e Gestão.

Gerado em 07/11/2026

Conteúdo

1. Índice da documentação docs/README.md
2. Visão geral docs/visao-geral.md
3. Guia do usuário docs/guia-do-usuario.md
4. Regras de negócio docs/regras-de-negocio.md
5. Arquitetura docs/arquitetura.md
6. Referência da API docs/api.md
7. Banco de dados docs/banco-de-dados.md
8. Operação docs/operacao.md
9. Glossário docs/glossario.md
10. Anexo: README README.md
11. Anexo: CLAUDE.md (regras detalhadas) CLAUDE.md
12. Anexo: Autenticação AUTENTICACAO.md
13. Anexo: Homologação HOMOLOGACAO.md
14. Anexo: Migrações MIGRACOES.md
15. Anexo: Testes PostgreSQL TESTES_POSTGRESQL.md
16. Anexo: Vercel VERCEL.md
17. Anexo: Multi-empresa MULTIEMPRESA.md
18. Anexo: Reuniões e ata REUNIOES.md
19. Anexo: Ensaio da TV ENSAIO_TV.md
20. Anexo: Estabilização ESTABILIZACAO.md
21. Anexo: Publicação de 05/10/2026 PUBLICACAO-2026-10-05.md

ÍNDICE DA DOCUMENTAÇÃO

Documentação do SGC / Agilis

Sistema para gerir e acompanhar a produtividade semanal dos Centros do Departamento de Planejamento, Orçamento e Gestão. O uso principal é a reunião presencial de terça-feira, 10h, do Diretor com os chefes de seção, com TV por HDMI na sala.

Esta pasta reúne a documentação geral. Os documentos que já existiam na raiz continuam valendo e são citados onde aprofundam um assunto.

Por onde começar

Quero...	Leia
Entender o que o sistema faz e para quem	visao-geral.md
Usar o sistema (por perfil)	guia-do-usuario.md
Entender as regras de negócio	regras-de-negocio.md
Entender como o código é organizado	arquitetura.md
Consultar as rotas da API	api.md
Consultar tabelas e migrações	banco-de-dados.md
Instalar, configurar, publicar e testar	operacao.md
Ver o significado dos termos	glossario.md

Documentos da raiz do repositório

Arquivo	Assunto
README.md	Como rodar, comandos e limites conhecidos
CLAUDE.md	Instruções para quem desenvolve (com o Claude Code); a fonte mais detalhada das regras
AUTENTICACAO.md	Login por e-mail e senha com Supabase Auth
HOMOLOGACAO.md	Roteiro de homologação com o Supabase real
MIGRACOES.md	Como as migrações do banco funcionam
TESTES_POSTGRESQL.md	Testes de integração com PostgreSQL descartável
VERCEL.md	Implantação na Vercel
MULTIEMPRESA.md	Plano e decisões do suporte a várias empresas
REUNIOES.md	Presença, informações e revisão da ata
ENSAIO_TV.md	Roteiro de ensaio do Modo Reunião na TV
ESTABILIZACAO.md	Andamento das correções de estabilização
PUBLICACAO-2026-10-05.md	Registro de uma publicação

Convenções do projeto

- Idioma do produto, da interface, dos commits e dos comentários: português do Brasil. Nomes de código (tabelas, colunas, funções) também são em português.
- Datas em ISO (`AAAA-MM-DD`) no banco e na API; na tela, formatadas com `br()`.
- Erros de API: `{ "erro": "mensagem em português" }` com o status HTTP adequado.
- Dados de exemplo são fictícios. Não inclua dados reais do Departamento no repositório.

Visão geral

Propósito

O Departamento de Planejamento, Orçamento e Gestão é dividido em Centros e uma Coordenação, cada um com um chefe. Toda terça-feira, às 10h, o Diretor se reúne com esses chefes. O sistema existe para que essa reunião seja conduzida com dados atualizados: o que foi feito, o que vem a seguir, o que está atrasado ou travado e o que foi decidido.

O projeto nasceu como protótipo com dados fictícios, para validar o fluxo com o Diretor. Hoje também suporta várias empresas (órgãos) no mesmo sistema, com a empresa original como Empresa 1 (ver MULTIEMPRESA.md).

Ciclo semanal

A semana vai de **terça a segunda**. O fechamento é na **segunda, 18h**.

1. **Durante a semana**, o Diretor (ou o Apoio) direciona ações às seções, sempre com prazo. Os chefes executam: mudam o status, registram tempo em minutos, criam tarefas, registram impedimentos e comentam.
2. **Até a segunda, 18h**, cada chefe envia a **atualização semanal**. O relato é montado pelo servidor a partir das ações da seção e das subseções; o chefe acrescenta observações e envia.
3. **Na terça**, o Diretor ou o Apoio abre o **Modo Reunião** na TV: combinados, visão geral das seções em ordem de necessidade (vermelhas primeiro) e uma tela por seção, com ações, pedidos de prazo e decisões ao vivo.
4. **Ao encerrar**, o sistema gera a **ata em rascunho**. Ela é revisada, editada e enviada (nesta versão, "enviar" libera a leitura dentro do sistema).

Semáforo

Cada unidade de acompanhamento recebe uma cor, com forma e texto (nunca só cor):

Cor	Quando
Vermelho	Há ação atrasada, ou impedimento crítico aberto em ação em curso
Amarelo	A atualização não foi enviada, ou há ação vencendo em até 2 dias
Verde	Todo o resto

A cor de uma seção considera as subseções (vale a pior). Os limiares ainda precisam ser validados com o Diretor.

Perfis

Perfil	Resumo
Diretor	Direciona ações, decide prazos, conduz a reunião, gere estrutura e contas de Chefe e Apoio
Apoio do Diretor	Quase igual ao Diretor, sem Estrutura; decide prazos apenas dentro de reunião em andamento
Chefe de seção	Vê e age na própria seção e nas subordinadas; envia a atualização semanal. Pode haver um suplente com os mesmos poderes
Membro da equipe	Lê tudo da própria seção e age só no que é dele (atribuído ou criado por ele)
Consulta	Só lê as ações das unidades
Administrador	Acesso total dentro da empresa, inclusive contas, perfis e estrutura
Plataforma	Administra as empresas; não lê dados de negócio

Detalhes de permissões em regras-de-negocio.md e api.md.

Telas principais

- **Painel** (gestão): semáforo por unidade, com detalhe de cada seção.
- **Direcionar ação** e **Ações**: criação e acompanhamento, com filtros.
- **Pedidos de prazo**: decisão do Diretor (ou do Apoio, em reunião).
- **Início**, **Minha atualização**, **Minhas ações**, **Histórico** (chefe).
- **Agenda**: calendário mensal com ações (pelo prazo) e tarefas (pela data); todos os perfis.
- **Combinados**, **Reuniões e atas**, **Modo Reunião**.
- **Produtividade**: indicadores das unidades.
- **Estrutura / Contas / Configuração**: seções, pessoas, vocabulário, cabeçalho da ata.
- **Quadro de avisos** no Início do chefe.
- **Empresas** (perfil Plataforma).

No celular (até 760 px) o sistema vira um app instalável, com abas inferiores e a folha "Mais".

Limites conhecidos

- Sem e-mail nem lembretes; "enviar ata" apenas libera a leitura no sistema.
- Sem fechamento automático da segunda 18h: o chefe ainda consegue enviar correção depois.
- Sem anotações privadas do Diretor.
- O Modo Reunião não foi validado com TV/HDMI reais nem em navegadores além do Chromium.
- A validação com o projeto Supabase real depende do roteiro de HOMOLOGACAO.md.
- Não crie a 2ª empresa antes de a migração 23 estar aplicada em produção.

Guia do usuário

Os nomes das telas e dos tipos de unidade podem variar por empresa (vocabulário). Este guia usa os nomes do órgão original.

Todos os perfis

- **Entrar:** e-mail e senha. Conta criada por outra pessoa pede a troca da senha no primeiro acesso.
- **Agenda:** calendário do mês à esquerda e, à direita, o painel do dia escolhido, com ações (pelo prazo) e tarefas (pela data). Círculo = ação, quadrado = tarefa; vermelho só quando aquela ação ou tarefa está atrasada. Filtros: *Mostrar* (Tudo/Ações/Tarefas), *De quem* ("Só o que é meu", para chefe e administrador com seção) e *Seção*. No celular ficam atrás do botão "Filtros".
- **Celular:** abas inferiores e a folha "Mais" com o menu completo. Pode ser instalado como aplicativo.

Chefe de seção

1. **Início:** quadros "Chegaram para você" (ações novas), "Aguardando sua validação" e "Precisa de atenção" (atrasadas, sem atualização há 7 dias, prazo até a reunião, com botões rápidos de status e tempo), mais o aviso vigente.
2. **Minhas ações:** abra a ação para mudar o status, registrar o **tempo em minutos**, comentar, criar tarefas e registrar impedimentos. *Concluir exige tempo maior que zero.*
3. **Pedir novo prazo:** em ação criada por outra pessoa, use o pedido de prazo e aguarde a decisão.
4. **Minha atualização:** confira o relato montado automaticamente, acrescente observações e envie até a **segunda, 18h**. Se algo mudar depois, a tela avisa que o relato está desatualizado; enviar de novo cria nova versão.
5. **Minha equipe:** cadastre membros, desative ou reative e redefina senhas.
6. **Redirecionar** uma ação para a própria seção ou subseções; **atribuir** um responsável; **validar** ou **devolver** o que um membro concluiu.
7. **Histórico, Combinados e Atas:** consulta. Participantes registrados podem sugerir revisão em ata publicada.

Se você tem um **suplente**, ele tem os mesmos poderes e responde na sua ausência.

Membro da equipe

Em **Minhas ações** há as abas "Minhas" e "Da seção, só leitura". Você muda status, registra tempo, cria tarefas e comenta nas ações atribuídas a você ou criadas por você. Em ação de outra pessoa, só marca como feita a tarefa que é sua e comenta onde tem tarefa. Relatos, reuniões, atas, estrutura e contas não estão disponíveis.

Diretor e Apoio

- **Painel:** semáforo por unidade, em ordem de necessidade; abra uma unidade para o detalhe.
- **Direcionar ação:** escolha "todos os Centros" ou seções específicas, informe o prazo. O sistema avisa se as seções já têm ações com o mesmo prazo. Depois, "Acrescentar seções" em "Últimas ações direcionadas" inclui outras seções com a mesma ação.
- **Ações:** lista com filtros (inclusive "demanda de entidade externa").
- **Pedidos de prazo:** aprovar ou recusar. O Apoio só decide dentro de reunião em andamento.
- **Aceitar ou devolver** ações concluídas que o Diretor demandou.
- **Combinados:** criar, editar e arquivar (nunca excluir).
- **Reuniões e atas**, **Produtividade**, **Quadro de avisos** (Diretor) e **Configuração**.
- O Diretor também gere **Estrutura** (seções) e contas de Chefe e Apoio. O Apoio não vê ações internas das subseções.

Conduzindo a reunião (Modo Reunião)

1. Conecte o notebook à TV por HDMI.
2. Menu **Iniciar reunião**. A tela de conferência mostra seções sem atualização, pedidos de prazo e combinados. A reunião só é criada ao clicar em **Iniciar reunião**; recarregar a página não abre outra, e uma reunião em andamento é retomada por **Retomar reunião**.
3. Navegue com as setas do teclado (direita avança, esquerda volta): combinados → visão geral → uma tela por seção.
4. Em cada seção, edite ações, decida pedidos de prazo, registre decisões, crie novas ações, registre informações e a **presença**.
5. **Encerrar** gera a ata em rascunho. Revise (barra de formatação), **Baixar PDF** se quiser e **Enviar**.
6. Plano B: "Versão para impressão" da pauta.

Consulta

Tela **Ações dos Centros**: lista de todas as ações dos Centros (sem as internas), só para leitura, mais a Agenda. Nenhum botão de alteração aparece.

Administrador

Faz tudo o que os demais fazem, em qualquer seção, e gere contas, perfis e estrutura em "Contas e estrutura". Exclui versões do histórico de atualizações quando necessário. O Administrador vinculado à Diretoria Adjunta tem a tela "Início da seção" como o chefe.

Plataforma

Tela **Empresas**: criar empresa com seu primeiro Diretor, editar nome e sigla, ocultar da lista de entrada, desativar ou reativar (desativar derruba as sessões). Entra por `/?empresa=plataforma`.

Regras de negócio

Fonte: plano do projeto. **Não altere sem combinar com o Diretor.** Este texto é um resumo legível; o detalhe técnico (nomes de rotas, campos e funções) está em CLAUDE.md. As constantes compartilhadas estão em `public/js/regras.js`; as regras de data e semáforo, em `server/logic.js`.

Semana, fechamento e semáforo

- Semana = terça a segunda. Fechamento: segunda, 18h.
- `refTerca`: na terça depois das 12h, a referência passa para a terça seguinte. O dia, a hora da reunião e o corte são configuráveis por empresa.
- Semáforo da seção: vermelho (ação atrasada ou impedimento crítico aberto em ação em curso), amarelo (atualização não enviada ou ação vencendo em até 2 dias), verde (o resto). Subseções entram na cor do cartão acima (vale a pior).
- "Ação sem atualização" (7 dias sem movimentação) é só informativa: não muda o semáforo.

Ações

- Status: `a_fazer`, `em_andamento`, `bloqueada`, `concluida`, com transições em `TRANSICOES`.
- **Concluir exige tempo registrado maior que 0 minutos** (erro 422). O chefe registra tempo em minutos.
- Voltar para `a_fazer` exclui todo o tempo registrado e registra um comentário com a quantidade excluída.
- Prioridades: alta, média, baixa. Esforço estimado: pequeno (1), médio (2), grande (3).
- Toda ação nasce com prazo. Diretriz do Diretor vira uma ação por seção destinatária.
- Antes de direcionar, o sistema avisa sobre ações em aberto com o mesmo prazo nas seções alvo. É só aviso: não bloqueia.

Autoria e edição

O autor (chefe ou suplente que criou; a gestão, nas criadas pela gestão; o Administrador, em todas) edita título e detalhamento enquanto a ação não foi apresentada em reunião, muda o prazo enquanto está em curso e exclui só se não foi apresentada nem concluída. Ação concluída ou apresentada não é excluída por ninguém. Status, tempo, tarefas e impedimentos ficam com quem executa. Toda edição com mudança registra um comentário.

Prazo

O prazo de ação criada por outra pessoa só muda por **pedido** (`pedidos_prazo`). O Diretor decide; o Apoio só decide dentro de reunião em andamento, por delegação. O autor altera direto pela edição. Toda mudança grava o prazo anterior e o novo (`acao_prazos`).

Tarefas

Servem de apoio: não mudam o status nem a regra de concluir. São listadas por data, as sem data no fim; tarefa com data passada e não feita fica marcada como atrasada (só destaque visual). Podem ser editadas e excluídas livremente por quem executa a ação; o membro dono da tarefa também a marca como feita. Diretor e Apoio só veem.

Impedimentos

Pertencem a uma ação, com ciclo aberto → resolvido. São histórico: nunca editados nem excluídos (para corrigir, resolva e registre outro). `bloquear` só vale com a ação em andamento; `retomar` volta a ação bloqueada para em andamento. Concluir a ação resolve os impedimentos abertos.

Redirecionar

Muda a seção responsável levando todo o histórico e registra um comentário com origem, destino e motivo. A gestão escolhe qualquer seção ativa; o chefe, só a própria seção ou as subseções. Não vale para ação concluída, encerrada ou arquivada.

Ação recebida

A ação "chega" à seção quando vem de fora (diretriz, seção acrescentada, criada pela gestão, redirecionamento). Fica `nova` para quem a recebe até a pessoa abri-la. O Início do chefe tem o quadro "Chegaram para você". A gestão vê a ciência por seção. Só aviso dentro do sistema, sem e-mail.

Validação

Depois de concluída, só duas origens têm validação:

1. **Atribuída pelo chefe a um membro:** o chefe valida ou devolve com motivo.
2. **Demandada pelo Diretor** (diretriz ou "Demanda do Diretor"): o Diretor aceita ou devolve.

Quando as duas valem, primeiro o chefe, depois o Diretor. As demais ações ficam prontas ao concluir. Mudar status, reabrir ou trocar o responsável recomeça a validação do chefe.

Demanda de entidade externa

Marca a ação demandada por entidade fora da subordinação ao Diretor (órgão de controle, outra secretaria). O nome (até 120 caracteres) é obrigatório quando marcada; torna a ação visível ao Diretor e não muda o semáforo.

Estrutura

- Tipos: Centro, Coordenação, Subseção e Diretoria Adjunta (no máximo uma, sem chefe). Níveis configuráveis por empresa (2 a 6, padrão 3).
- **Unidade de acompanhamento** (cartão da reunião, linha do Painel e do semáforo): Diretoria Adjunta, Centro e Coordenação. A Subseção nunca é cartão e soma no cartão acima.

- Ações internas de subseção (`interna = 1`) não aparecem ao Diretor nem ao Apoio, só em contagens.
- Seção fora da reunião (`na_reuniao = 0`) sai dos cartões, da pauta e da ata; o Painel continua mostrando, com a marca.
- A Diretoria Adjunta não recebe diretriz "todos os Centros". O Diretor Adjunto (Administrador vinculado à Diretoria Adjunta) não direciona ações para a própria seção.

Perfis e permissões

Perfil	Pode
Diretor	Direciona, decide prazos, conduz a reunião, aceita/devolve, gere estrutura e contas de Chefe e Apoio. Não vê ações internas
Apoio	O mesmo, exceto Estrutura; decide prazos só em reunião em andamento
Chefe	Age na própria seção e subordinadas; envia a atualização; cadastra a equipe; pode ter um suplente com os mesmos poderes
Membro	Lê a seção; age no que é atribuído a ele ou criado por ele; não atribui, não arquiva, não redireciona, não pede prazo
Consulta	Só lê ações (e tarefas) de todos os Centros, sem as internas e sem relatos
Administrador	Tudo dentro da empresa, inclusive ações internas e contas
Plataforma	Cria e gerencia empresas; não lê dados de negócio

Travas de integridade: sempre há um Diretor ativo; Administrador e Plataforma só nascem por comando de linha (`admin:criar`, `plataforma:criar`); contas com senha definida por terceiros exigem troca no primeiro acesso.

O chefe com `ver_outros_centros` (ligado pelo Diretor ou Administrador) lê as ações de todos os Centros, só leitura.

Atualização semanal

- Versionada (`secao_id` + `semana` + `versao`): enviar de novo cria nova versão; o histórico é mantido. O Administrador pode excluir uma versão (para corrigir um relato de teste ou enviado por engano).
- O relato é montado pelo servidor a partir das ações e impedimentos da seção e subseções. Concluídas: 7 dias antes da reunião até a véspera. Programadas: do dia da reunião a 7 dias depois. Atrasadas: prazo anterior a hoje e não concluídas. Impedimentos: os abertos em ações em curso.
- O envio recalcula no servidor e congela uma cópia que **nunca contém ações internas, só contagens**.
- Não há trava automática após segunda 18h: o chefe ainda consegue enviar correção.

Reunião e ata

- Situação: `em_andamento` → (encerrar) `rascunho` → (enviar) `enviada`. Reabrir só a partir de rascunho.
- A ata é gerada ao encerrar. Diretor, Apoio e Administrador a editam (também depois de enviada) e podem excluí-la; excluir apaga as decisões e desvincula ações e pedidos, que permanecem.
- Ao encerrar, as ações dos cartões ficam marcadas como apresentadas (a data da primeira reunião não muda).
- **Combinados:** arquivar, nunca excluir. Cada reunião guarda uma cópia (`combinados_snapshot`). Aviso quando há mais de 7 ativos. Frequência: sempre, primeira do mês ou quando mudarem. Não entram no texto da ata.
- A ata usa marcação leve: `**negrito**`, `*itálico*`, listas com `- ` ou `1. `. O texto é escapado antes de aplicar as marcas.
- Presença, informações e sugestões de revisão: ver REUNIOES.md.

Quadro de avisos

Uma mensagem do Diretor ou Administrador, com início e fim (fim obrigatório e nunca no passado). Aparece no Início do chefe e da Diretoria Adjunta; no máximo uma por vez (a vigente de maior id). Exclusão definitiva.

Produtividade

Avalia **unidades**, não pessoas isoladas, sempre comparando com o período anterior de mesma duração. Indicadores: entregas (com pontos pelo esforço), pontualidade, retrabalho, tempo de ciclo, dedicação e carga de hoje. Só soma ações visíveis a quem consulta. Por pessoa: apenas Diretor, Administrador e o chefe da unidade. Não compare unidades de natureza diferente nem use um indicador sozinho.

Multi-empresa e vocabulário

Cada empresa tem nome, sigla, logo, cabeçalho e brasões da ata, dia/hora da reunião e **vocabulário próprio** (nomes dos tipos de unidade e do perfil do topo). Todo texto de interface ou mensagem que cite Centro, Coordenação, Subseção, Diretoria Adjunta ou Diretor deve vir do vocabulário, não fixo no código. A entrada pede só e-mail e senha: o servidor descobre o órgão pela conta.

Arquitetura

Visão de conjunto

```
Navegador (SPA em JavaScript puro, módulos ES, sem build)
        │  fetch /api/*  (cookie de sessão HttpOnly + CSRF)
        ▼
Express 4 (server/app.js)
  segurança HTTP → compressão → JSON → estáticos → auth → escopo da empresa
  → travas por perfil (Consulta, Plataforma, Membro) → rotas por assunto → tratador de erros
        │
        ▼
Adaptador de banco (server/db.js), interface assíncrona
  ├─ SQLite  (better-sqlite3, WAL, foreign_keys ligado)  → desenvolvimento e demonstração
  └─ PostgreSQL (pg)                                      → produção (Supabase)
```

Autenticação em produção: Supabase Auth (e-mail e senha), mediada pelo servidor Express. Hospedagem prevista: Vercel (função Node + estáticos). Sem ORM: SQL direto.

Estrutura de pastas

Pasta	Conteúdo
`server/`	API Express, adaptadores de banco, migrações, seed e comandos administrativos
`public/`	Interface: `index.html`, `estilo.css`, `js/` (uma tela por arquivo), `manifest.webmanifest`, `sw.js`, `imagens/`, `fontes/`
`test/`	Testes com o executor nativo do Node (`node --test`); `test/postgres/` usa um cluster PostgreSQL descartável
`scripts/`	SQLs de apoio (provisionamento Supabase, limpezas pontuais) e `contagem-registros.js`
`docs/`	Esta documentação
`certs/`	Certificado CA do Supabase para conexão PostgreSQL verificada

Servidor

Montagem (`server/app.js`)

`createApp(db, { auth })` monta, nesta ordem: cabeçalhos de segurança, compressão, JSON (limite de 200 kb), estáticos, autenticação, escopo da empresa e três travas por perfil. Depois registra as rotas e, por último, o tratador de erros.

As três travas são **listas fechadas**: o perfil Consulta só lê quatro rotas de ações; o Plataforma só acessa `/bootstrap` e `/plataforma/*`; o Membro só acessa `ROTAS_MEMBRO`. Rota nova não precisa fazer nada para ficar fechada a esses perfis.

Arquivos por responsabilidade

Arquivo	Responsabilidade
`index.js`	Ponto de entrada: configura auth, abre o banco, migra (SQLite) ou verifica (PostgreSQL), sobe o servidor
`contexto.js`	Auxiliares comuns às rotas: `q`, `q1`, `run`, `hoje`, configuração da reunião, vocabulário
`rotas-estrutura.js`	Seções, usuários, equipe
`rotas-acoes.js`	Diretrizes, ações, tarefas, impedimentos, redirecionamento, validação, pedidos de prazo
`rotas-semana.js`	Bootstrap, atualização semanal, histórico, painel, pauta, detalhe da seção
`reunioes.js`	Combinados, configuração, reuniões, decisões, ata
`reuniao-registros.js`	Presença, informações e sugestões de revisão da ata
`ata-cabecalho.js`, `ata-registros.js`	Cabeçalho/brasões e registros da ata
`rotas-avisos.js`	Quadro de avisos
`rotas-produtividade.js`, `produtividade.js`	Indicadores de produtividade das unidades
`rotas-plataforma.js`	Gestão de empresas (perfil Plataforma)
`helpers.js`	Painel, pauta e montagem de cartões
`relato.js`	Montagem do relato semanal
`logic.js`	Regras puras: datas, semana, fechamento, semáforo
`empresa.js`	Escopo da empresa por requisição
`auth.js`, `cadastro-chefes.js`, `vincular-identidade.js`, `verificar-auth.js`	Login, sessão, CSRF, criação de contas
`seguranca-http.js`, `seguranca-supabase.*`	Cabeçalhos HTTP e endurecimento do Supabase (RLS)
`db.js`, `schema.sql`	Esquema, abertura do banco e árvore de seções (CTE recursiva)
`migrations.js`, `migration-*.js`, `migrate.js`	Migrações incrementais e comando do migrador
`seed.js`, `export-sqlite-to-pg.js`	Dados fictícios e exportação SQLite → PostgreSQL
`criar-*.js`, `redefinir-senha.js`, `secao-administrador.js`	Comandos administrativos (veja operacao.md)

Banco de dados

- A interface dos dois adaptadores é assíncrona: aguarde `get`, `all`, `run`, `seed` e os helpers de árvore.
- Transações: `await db.transaction(async () => { ... })`. Transações aninhadas são rejeitadas. No SQLite, consultas externas aguardam a transação; no PostgreSQL, o cliente é mantido por

contexto assíncrono.
- A hierarquia de seções é consultada no banco, sem cache.

Multi-empresa

A empresa vem **da conta logada**, nunca do cliente (`empresaAtual()` em `empresa.js`). Todo SQL de tabela raiz filtra por `empresa_id`; em `visiveisAcoes` o `a.empresa_id = ?` é sempre o primeiro marcador. `SGC_LINT_EMPRESA=1 npm test` falha em SQL de tabela raiz sem `empresa_id` dentro de uma requisição. Detalhes em MULTIEMPRESA.md.

Tempo

Nunca chame `new Date()` nas regras: use `agora()` de `server/logic.js`, que obedece a `SGC_NOW`. O fuso de negócio é `America/Bahia`. Datas civis usam aritmética UTC. Exceção: a segurança (sessões, tentativas de login) usa `Date.now()` real.

Interface

Estrutura

- `index.html` carrega `js/main.js`, que faz o roteamento por hash (`#/agenda`, `#/acoes-centros`, ...).
- Cada tela é uma função `(raiz, { id, q, refresh })` que preenche um elemento. Esse elemento está solto do documento durante a montagem, então use `$('#id', raiz)` e nunca `document.querySelector`.
- Diálogos usam `<dialog>` via `abrirForm` (`js/ui.js`). Eventos usam delegação: `on(raiz, tipo, seletor, fn)`.

Módulos principais

Módulo	Função
`main.js`	Roteamento, menu, abas do celular (`ABAS_MOVEL`), instalação do app
`api.js`, `estado.js`	Chamadas à API e estado da sessão
`regras.js`	**Fonte única** de status, transições, prioridades, ordem do semáforo — compartilhada com o servidor
`datas.js`	Funções de data no fuso de negócio
`vocabulario.js`, `vocab.js`	Vocabulário por empresa (puro) e atalhos da tela
`telas-chefe.js`, `telas-diretor.js`	Telas por perfil
`acao-comum.js`	Detalhe da ação (`abrirAcao`), confirmação de prazo
`agenda.js`, `consulta.js`, `equipe.js`, `estrutura.js`, `avisos.js`, `combinados.js`	Telas por assunto
`reuniao.js`, `reuniao-historico.js`, `presenca-ordem.js`	Modo Reunião

Módulo	Função
`atas.js`, `ata-*.js`, `pdf-ata.js`	Ata: edição, marcação leve, cabeçalho, PDF (jsPDF no navegador)
`produtividade.js`, `plataforma.js`, `senha.js`, `relato-vista.js`	Demais telas
`vendor/`	jsPDF e Quill, vendorizados (sem CDN)

Estilo

`estilo.css` traz o sistema visual (paleta petróleo, títulos em serifa, semáforo com forma e texto), o modo TV (`body.tv`, classes `.tv-*`), a seção "Celular" e regras de impressão.

Aplicativo instalável

`manifest.webmanifest` e `sw.js`. O service worker é deliberadamente **sem cache**: o sistema tem sessão e regras sensíveis a versão; ele existe apenas para satisfazer o critério de instalação.

Segurança

- Sessão HttpOnly persistida por hash; CSRF; limite de tentativas de login. Detalhes em AUTENTICACAO.md.
- `configurarAuth()` falha sem configuração institucional. `SGC_AUTH_MODE=demo` aceita `x-user-id` apenas fora de produção e só em loopback.
- Nunca vincule identidades por e-mail e nunca confie em perfil enviado pelo cliente.
- Permissões de perfil e propriedade são conferidas nas rotas (`permit` e verificações internas).
- Migração 4 habilita RLS e revoga acesso público aos objetos no PostgreSQL.

Testes

`npm test` roda a suíte em SQLite em memória, mais o adaptador PostgreSQL com pool simulado. `npm run test:postgres` cria um PostgreSQL descartável. Veja operacao.md.

Referência da API

Todas as rotas ficam sob `/api`, falam JSON e exigem sessão (exceto as de entrada). Erros: `{ "erro": "mensagem" }` com status 400, 403, 404, 409 ou 422 (500 vira "Erro interno").

Mutações exigem o cabeçalho `X-CSRF-Token` (emitido no login) e `Origin` igual à origem pública configurada (veja AUTENTICACAO.md).

Legenda de perfis: D = diretor · A = apoio · Adm = administrador · C = chefe · M = membro · Con = consulta · Pl = plataforma. "Qualquer" = qualquer perfil autenticado, ainda sujeito às travas de lista fechada (Consulta, Membro, Plataforma) descritas em arquitetura.md.

Onde a coluna diz "regra na rota", a rota não usa `permit`: a decisão de quem pode é tomada dentro dela (`executa`, `ehAutor`, `visiveisAcoes`).

Autenticação (`server/auth.js`)

Método e rota	Quem	Função
GET `/auth/config`	público	Modo de autenticação e configuração pública
GET `/auth/empresas`	público	Empresas listadas na entrada (demonstração)
GET `/usuarios-demo`	só modo demo	Perfis de demonstração
POST `/auth/entrar`	público	Login (e-mail e senha); `empresa` opcional restringe
POST `/auth/trocar-senha`	autenticado	Troca de senha (obrigatória no primeiro acesso de contas criadas por terceiros)
POST `/auth/sair`	autenticado	Encerra a sessão

Sessão, semana e painel (`rotas-semana.js`)

Rota	Quem	Função
GET `/bootstrap`	qualquer	Usuário, empresa, vocabulário, contadores (`acoes_novas`, `acoes_a_validar`), semana de referência
GET `/atualizacao`	C, Adm	Relato ao vivo da seção (`?resumo=1` pula o relato); traz `desatualizada`
PUT `/atualizacao`	C, Adm	Envia a atualização (`semana`, `observacoes`); servidor recalcula e congela a cópia
GET `/historico`	C, Adm	Histórico de atualizações da seção
DELETE `/atualizacoes/:id`	Adm	Exclui uma versão do histórico
GET `/painel`	D, A, Adm	Semáforo por unidade

Rota	Quem	Função
GET `/pauta`	D, A, Adm	Pauta da reunião
GET `/secoes/:id/detalhe`	D, A, Adm	Detalhe de uma seção

Estrutura e pessoas (`rotas-estrutura.js`)

Rota	Quem	Função
GET `/secoes`	qualquer	Lista de seções visíveis (traz `membros` na Diretoria Adjunta)
POST `/secoes`	D, Adm	Cria seção
PATCH `/secoes/:id`	D, Adm	Edita seção (nome, tipo, superior, chefe, `na_reuniao`…); valida a hierarquia
DELETE `/secoes/:id`	D, Adm	Exclui seção (pela função `excluirCadastro`, em `crud-cadastros.js`)
GET `/usuarios`	D, A, Adm	Lista de contas (`suplente` marcado)
POST `/usuarios`	D, Adm	Cria conta
POST `/usuarios/chefes`	D, Adm	Cadastra chefe (cria a conta no Supabase Auth)
POST `/usuarios/acesso`	D, Adm	Cadastra acesso; `suplente: true` cria suplente
PATCH `/usuarios/:id`	D, Adm	Edita perfil, ativação, `ver_outros_centros`
PATCH `/usuarios/:id/login`	D, Adm, C	Redefine senha/login sem trocar e-mail
DELETE `/usuarios/:id`	D, Adm	Exclui conta (pela função `excluirCadastro`; conta com vínculos de histórico, como ciência de ação, não é excluída: use Desativar)
GET `/equipe`	C, Adm	Equipe da seção
POST `/equipe`	C, Adm	Cadastra membro da equipe
PATCH `/equipe/:id`	C, Adm	Desativa ou reativa membro

Diretrizes e ações (`rotas-acoes.js`)

Rota	Quem	Função
POST `/diretrizes`	D, A, Adm	Direciona ação a todos os Centros ou seções escolhidas (com prazo)
GET `/diretrizes`	D, A, Adm	Últimas ações direcionadas, com ciência por seção
POST `/diretrizes/:id/secoes`	D, A, Adm	Acrescenta seções a um direcionamento

Rota	Quem	Função
GET `/acoes`	qualquer	Lista. Filtros: `situacao`, `prazo`, `origem=externa`, `escopo=todos`
POST `/acoes`	C, M, D, A, Adm	Cria ação
GET `/acoes/:id`	qualquer	Detalhe, com `pode_*`, `somente_leitura`, `prazos`, `vistas`
PATCH `/acoes/:id`	regra na rota	Edita título, detalhamento, prazo, status, prioridade, responsável, esforço
DELETE `/acoes/:id`	regra na rota	Exclui (só se não apresentada nem concluída)
POST `/acoes/:id/vista`	qualquer	Registra a ciência de recebimento
POST `/acoes/:id/tempo`	regra na rota	Registra tempo em minutos
POST `/acoes/:id/comentarios`	C, M, D, A, Adm	Comenta
POST `/acoes/:id/pedido-prazo`	C, Adm	Pede novo prazo
POST `/acoes/:id/impedimentos`	C, M, D, A, Adm	Registra impedimento
POST `/acoes/:id/impedimentos/:iid/resolver`	C, M, D, A, Adm	Resolve impedimento
GET `/acoes/:id/executores`	C, Adm	Quem pode ser responsável pela ação
GET `/acoes/:id/responsaveis`	regra na rota	Quem pode ser responsável por tarefa
POST `/acoes/:id/checklist`	regra na rota	Cria tarefa
PATCH `/acoes/:id/checklist/:iid`	regra na rota	Edita ou marca tarefa
DELETE `/acoes/:id/checklist/:iid`	regra na rota	Exclui tarefa
POST `/acoes/:id/redirecionar`	C, D, A, Adm	Muda a seção responsável, levando o histórico
POST `/acoes/:id/arquivar` · `/desarquivar`	C, Adm	Arquiva ou restaura
POST `/acoes/:id/validar`	C, Adm	Chefe valida ação concluída que delegou
POST `/acoes/:id/devolver`	C, D, Adm	Devolve para ajuste, com motivo
POST `/acoes/:id/encerrar`	D, Adm	Diretor aceita a ação concluída
GET `/tarefas`	qualquer	Tarefas visíveis a quem consulta (usada pela Agenda)
GET `/pedidos-prazo`	D, A, Adm	Lista pedidos
POST `/pedidos-prazo/:id/decidir`	D, A, Adm	Aprova ou recusa (Apoio só em reunião em andamento)

> Nas rotas `/acoes/:id/...`, toda gravação bem-sucedida renova `movimentada_em` (middleware em `rotas-acoes.js`).

Reuniões, combinados e configuração (`reunioes.js`, `reuniao-registros.js`, `ata-cabecalho.js`)

"Gestão" = D, A, Adm.

Rota	Quem	Função
GET `/combinados`	qualquer	Lista (ativos e arquivados)
POST `/combinados`	gestão	Cria
PATCH `/combinados/:id`	gestão	Edita ou arquiva (nunca exclui)
GET `/config`	qualquer	Configuração da empresa
PUT `/config`	gestão	Frequência dos combinados (`combinados_frequencia`, obrigatória), dia (`reuniao_dia`, 0–6) e hora (`reuniao_hora`, HH:MM) da reunião
PUT `/config/vocabulario`	D, Adm	Vocabulário da empresa
PUT `/config/ata`	D, A, Adm	Cabeçalho e brasões da ata
POST `/reunioes/iniciar`	gestão	Abre a reunião (ou retoma a em andamento)
GET `/reunioes` · `/reunioes/:id`	qualquer	Lista e detalhe
GET `/reunioes/:id/cartoes`	gestão	Cartões das seções
POST `/reunioes/:id/decisoes`	gestão	Registra decisão
PATCH/DELETE `/reunioes/:id/decisoes/:decisaoId`	gestão	Edita/exclui decisão (reunião em andamento)
POST `/reunioes/:id/acoes`	gestão	Nova ação durante a reunião
POST `/reunioes/:id/encerrar`	gestão	Encerra e gera a ata em rascunho
POST `/reunioes/:id/reabrir`	gestão	Reabre (só a partir de rascunho)
PUT `/reunioes/:id/ata`	gestão	Salva o texto da ata
POST `/reunioes/:id/enviar-ata`	gestão	Envia a ata
DELETE `/reunioes/:id`	gestão	Exclui reunião encerrada
GET `/reunioes/:id/candidatos`	gestão	Pessoas elegíveis à presença
PUT `/reunioes/:id/participantes`	gestão	Registra presença (ordem = ordem de seleção)
POST/PATCH/DELETE `/reunioes/:id/informacoes[/:infoId]`	gestão	Informações registradas
POST `/reunioes/:id/sugestoes`	C	Participante registrado sugere revisão da ata publicada (`tipo`: correcao, inclusao ou supressao)
POST `/reunioes/:id/sugestoes/:sugestaoId/responder`	gestão	Responde a sugestão pendente (`status`)

Avisos (`rotas-avisos.js`)

Rota	Quem	Função
GET `/avisos/atual`	qualquer	Aviso vigente (no máximo um)
GET `/avisos`	D, Adm	Lista
POST/PATCH/DELETE `/avisos[/:id]`	D, Adm	Gerencia (exclusão definitiva)

Produtividade e plataforma

Rota	Quem	Função
GET `/produtividade?de&ate[&secao]`	D, A, Adm, C	Indicadores por unidade; por pessoa só D, Adm e o chefe da unidade
GET `/plataforma/empresas`	PI	Lista empresas
POST `/plataforma/empresas`	PI	Cria empresa e primeiro Diretor
PATCH `/plataforma/empresas/:id`	PI	Edita nome/sigla, lista na entrada, ativa/desativa

Convenções de teste

Rota nova entra com teste em `test/rotas.test.js` (e `test/api.test.js` para regras), e com teste de isolamento em `test/isolamento-empresas.test.js` quando tocar tabela raiz.

Banco de dados

Dois backends com o mesmo esquema lógico: **SQLite** (`better-sqlite3`, arquivo `data/sgc.db`, demonstração e testes) e **PostgreSQL** (`pg`, produção no Supabase). O esquema base está em `server/schema.sql`; as mudanças posteriores são migrações incrementais em `server/migrations.js` e nos arquivos `server/migration-*.js`.

Este documento descreve as tabelas por finalidade. Para colunas exatas, consulte `server/schema.sql` e as migrações; elas são a fonte de verdade.

Tabelas

Estrutura e pessoas

Tabela	Finalidade
`empresas`	Órgãos atendidos pelo sistema (a Empresa 1 é o órgão original; existe uma empresa interna `plataforma`)
`secoes`	Árvore de unidades: `tipo` (`centro`, `coordenacao`, `subsecao`, `diretoria_adjunta`), `pai_id`, `chefe_id`, `na_reuniao`, ativa
`usuarios`	Contas: `perfil`, `secao_id`, `trocar_senha`, `ver_outros_centros`, ativo
`auth_contas`, `auth_identidades`	Vínculo entre a conta do sistema e a identidade do Supabase Auth (por UUID, nunca por e-mail)
`auth_sessoes`, `auth_sessoes_senha`	Sessões (guardadas por hash) e token CSRF
`auth_fluxos`, `auth_tentativas`	Fluxos de autenticação e limite de tentativas de login

Ações

Tabela	Finalidade
`diretrizes`	Direcionamento do Diretor; gera uma ação por seção destinatária
`acoes`	A ação: status, prazo, `prazo_original`, prioridade, esforço, autoria (`criado_por`), `responsavel_id`, `interna`, `recebida_em`, `movimentada_em`, `apresentada_em`, `iniciada_em`, `devolucoes`, `demandante_externo`, arquivamento
`acao_comentarios`	Comentários, inclusive os automáticos (edição, redirecionamento, mudança de responsável)
`tempo`	Tempo registrado em minutos
`acao_checklist`	Tarefas da ação (na tela, "Tarefas"): texto, `concluido`, `ordem`, `data`, `responsavel_id`

Tabela	Finalidade
`impedimentos`	Impedimentos da ação: descrição, `critico`, apoio, ciclo aberto → resolvido; histórico, nunca editado
`pedidos_prazo`	Pedidos de novo prazo e a decisão
`acao_prazos`	Histórico de mudanças de prazo (`origem` = `edicao` ou `pedido`)
`acao_vistas`	Ciência de recebimento por pessoa

Semana e reunião

Tabela	Finalidade
`atualizacoes`	Relato semanal versionado (`secao_id` + `semana` + `versao`) com a cópia congelada
`combinados`	Combinados da reunião (arquivados, nunca excluídos)
`reunioes`	Reunião: situação (`em_andamento`, `rascunho`, `enviada`), `ata_texto`, `combinados_snapshot`
`decisoes`	Decisões tomadas na reunião
`reuniao_participantes`	Presença, com nome e seção como estavam no registro e a ordem de seleção
`reuniao_informacoes`	Informações registradas durante ou depois da reunião
`ata_sugestoes`, `ata_revisoes`	Sugestões de revisão da ata publicada e suas respostas
`avisos`	Quadro de avisos com janela de vigência
`config`	Configuração por empresa (chave/valor): dia e hora da reunião, frequência dos combinados, vocabulário, cabeçalho e brasões da ata
`schema_migrations`	Versões de migração aplicadas

Quase todas as tabelas raiz têm `empresa_id` (padrão 1, por enquanto). Veja MULTIEMPRESA.md.

Migrações

Cada versão aplicada é registrada em `schema_migrations`. **Nunca altere uma versão já aplicada:** acrescente uma nova. Mudanças futuras exigem nova versão, inclusive para arquivos auxiliares de migração. As migrações são aditivas, por isso um banco mais novo que o código só gera aviso; uma migração pendente impede o início.

#	Nome	Assunto
1	`acoes_arquivadas`	Arquivamento de ações
2	`unicidade_pedidos_e_reunioes`	Um pedido aberto por ação e uma reunião ativa
3	`integridade_referencial`	Chaves estrangeiras alinhadas
4	`identidade_institucional`	Identidade do Supabase Auth; RLS e revogação de acesso público

#	Nome	Assunto
5	`login_email_senha`	Login por e-mail e senha
6	`autoria_acoes`	Autoria das ações
7	`sessao_deslizante`	Sessão com renovação
8	`perfil_administrador`	Perfil Administrador
9	`impedimentos_da_acao`	Impedimentos
10	`checklist_da_acao`	Tarefas da ação
11	`secao_diretoria_adjunta`	Tipo de seção Diretoria Adjunta
12	`secao_na_reuniao`	Seção dentro ou fora da reunião
13	`prazos_e_acoes_apresentadas`	Histórico de prazos e ações apresentadas
14	`acoes_ultima_movimentacao`	`movimentada_em`
15	`quadro_de_avisos`	Avisos
16	`registros_e_revisao_de_reuniao`	Presença, informações, sugestões de revisão
17	`ordem_participantes_reuniao`	Ordem da presença
18	`acoes_recebidas_e_vistas`	Ação recebida e ciência
19	`perfil_consulta`	Perfil Consulta
20	`tarefas_da_acao`	Data e responsável nas tarefas
21	`membros_e_validacao_de_acoes`	Membro da equipe, responsável e validação
22	`empresas`	Multi-empresa (Fase 1)
23	`contracao_multiempresa`	Chaves únicas por empresa (contração)
24	`perfil_plataforma`	Perfil Plataforma
25	`produtividade`	Esforço, devoluções e início da execução

Executando migrações

- **SQLite:** aplicadas ao iniciar.
- **PostgreSQL:** o servidor só verifica. Rode o migrador explicitamente:

```
SGC_MIGRATION_DATABASE_URL="postgres://..." npm run migrate -- --postgres
npm run migrate -- --sqlite caminho/do/arquivo.db
```

- Um PostgreSQL novo precisa antes do esquema base de `server/schema.sql`.
- Nunca remova dados duplicados automaticamente para instalar uma constraint: verifique, decida com o responsável e só então migre (ver MIGRACOES.md).
- Antes de migrar produção, conte os registros com `scripts/contagem-registros.js`.
- Mudanças que mexem em transações, migrações ou SQL compartilhado devem passar também por `npm run test:postgres`.

Scripts SQL de apoio (`scripts/`)

Arquivo	Uso
`criar-banco-supabase.sql`	Provisionamento do banco no Supabase
`migracao-9-supabase.sql`, `migracao-10-supabase.sql`	Aplicação manual de migrações específicas
`diagnostico-supabase.sql`	Diagnóstico
`excluir-reuniao-2026-10-02.sql`, `excluir-sugestao-teste-fulano.sql`, `limpar-acoes-piloto.sql`	Limpezas pontuais do piloto. Leia antes de executar: apagam dados
`contagem-registros.js`	Contagem de registros por tabela (use antes de migrar produção)

OPERAÇÃO

Operação: instalar, configurar, publicar e testar

Requisitos

- Node.js 22.12 ou superior (testado em 24.11.1).
- Para produção: um projeto Supabase (Auth + PostgreSQL). Para demonstração local: nada além do Node.

Demonstração local (SQLite, dados fictícios)

```
npm install
SGC_AUTH_MODE=demo DATABASE_URL="" npm start
```

Abra http://127.0.0.1:3000. O banco (`data/sgc.db`) é criado com dados fictícios. O modo demo escuta apenas em loopback e é recusado com `NODE_ENV=production` ou na Vercel. Para um banco isolado: `DATABASE_URL="" SGC_DB=/tmp/sgc-demo.db npm run seed`, iniciando depois com as mesmas variáveis.

No PowerShell (Windows), defina as variáveis antes: `$env:SGC_AUTH_MODE="demo"; $env:DATABASE_URL=""; npm start`.

Variáveis de ambiente

O modelo está em `.env.example`. **Nunca versione `.env`, `.env.local` nem chaves.**

Variável	Uso
`PORT`	Porta do servidor (padrão 3000)
`SGC_AUTH_MODE`	`supabase` (padrão institucional) ou `demo` (só local)
`SGC_PUBLIC_ORIGIN`	Origem pública do sistema; conferida no CSRF
`SUPABASE_URL`	URL do projeto Supabase
`SUPABASE_PUBLISHABLE_KEY`	Chave publishable/anon. Nunca use secret/service_role aqui
`SUPABASE_SERVICE_ROLE_KEY`	Exclusiva do servidor: criação de contas e troca de senha
`DATABASE_URL`	PostgreSQL da aplicação. Vazia = SQLite
`SGC_MIGRATION_DATABASE_URL`	Destino explícito de migrações e comandos administrativos
`SGC_PG_CA_FILE`, `SGC_PG_SSL`	CA do servidor e modo de verificação TLS (`certs/supabase-prod-ca-2021.crt`)
`SGC_DB`	Caminho do arquivo SQLite

Variável	Uso
`SGC_NOW`	"Agora" fictício, para testes e demonstrações (ex.: `2026-09-22T09:30:00`, interpretado no fuso da Bahia)
`SGC_TRUST_PROXY`	Número de proxies confiáveis (IP real no limite de login). Ativo por padrão na Vercel
`SGC_ADMIN_SENHA_INICIAL`, `SGC_ADMIN_SUBJECT`, `SGC_ADMIN_SECAO`	Comando `admin:criar`
`SGC_NOVA_SENHA`	Comando `senha:redefinir`
`SGC_EMPRESA`, `SGC_EMPRESA_SIGLA`, `SGC_EMPRESA_LISTADA`	Comandos de empresa
`SGC_LINT_EMPRESA`	`1` faz os testes falharem em SQL de tabela raiz sem `empresa_id`

Comandos

Comando	O que faz
`npm start`	Sobe o servidor
`npm run dev`	Igual, reiniciando ao alterar o servidor
`npm run seed`	Recria os dados fictícios (recusado com `DATABASE_URL`)
`npm run migrate -- --sqlite ARQUIVO` / `-- --postgres`	Aplica migrações
`npm run check:auth`	Confere a configuração do Supabase sem exibir chaves (`-- --online` consulta o projeto)
`npm run admin:criar -- --sqlite ARQUIVO "Nome" e-mail`	Cria o primeiro Administrador (`--postgres` para PostgreSQL)
`npm run admin:secao -- --sqlite ARQUIVO e-mail ID|nenhuma`	Vincula/desvincula um Administrador à Diretoria Adjunta
`npm run senha:redefinir`	Redefine a senha de uma conta (senha em `SGC_NOVA_SENHA`)
`npm run empresa:criar`	Cria uma empresa e o primeiro Diretor (senha provisória com troca obrigatória)
`npm run plataforma:criar`	Cria o usuário do perfil Plataforma
`npm test`	Suíte principal
`npm run test:postgres`	Integração com PostgreSQL descartável

Implantação em produção

1. **Supabase:** desabilite o cadastro público, exija confirmação de e-mail, informe a origem pública em *URL Configuration*. Passo a passo em AUTENTICACAO.md.
2. **Banco:** provisione o esquema base (`server/schema.sql` / `scripts/criar-banco-supabase.sql`) e rode o migrador. Antes, conte os registros (`scripts/contagem-registros.js`).
3. **Primeiro Administrador:** `npm run admin:criar` com `--postgres`.

4. **Publicação:** Vercel, conforme VERCEL.md (Root Directory na raiz, builders do `vercel.json`, Node 22.x ou 24.x, variáveis configuradas no painel).
5. **Homologação:** execute o roteiro de HOMOLOGACAO.md num projeto Supabase de teste antes de usar o de produção.
6. **Ensaio da reunião:** ENSAIO_TV.md.

Pontos de atenção:

- PostgreSQL exige a execução explícita do migrador a cada versão nova; o servidor não migra sozinho.
- Na Vercel, cabeçalhos de cache dos estáticos vêm do `vercel.json`, não do Express.
- **Não crie a 2ª empresa** antes de a migração 23 estar aplicada em produção.
- A chave `service_role` nunca vai para o repositório, o navegador, mensagens ou capturas de tela.

Testes

- `npm test` (Node test runner, `test/*.test.js`): regras, API, concorrência em SQLite em memória, migrações, isolamento entre empresas e adaptador PostgreSQL com pool simulado.
- `npm run test:postgres`: cluster PostgreSQL temporário, sem usar `DATABASE_URL`. Exige binários locais (`PG_BIN=/caminho/bin`). Veja TESTES_POSTGRESQL.md. Rode ao alterar transações, migrações ou SQL compartilhado.
- Interface: sem teste automatizado. Suba o servidor e confira no navegador em 1920×1080 (Modo Reunião) e 390×844 (celular).
- Sempre rode `npm test` depois de mexer em `server/`. Rota nova entra com teste (`test/rotas.test.js`, `test/api.test.js`; isolamento em `test/isolamento-empresas.test.js`).

Demonstrar uma data específica

```
SGC_NOW=2026-09-22T09:30:00 npm run seed
SGC_NOW=2026-09-22T09:30:00 npm start
```

Use a mesma variável no seed para os dados de exemplo fazerem sentido.

Instalação como aplicativo

O botão "Instalar aplicativo" aparece na barra lateral quando o navegador dispara `beforeinstallprompt`. Os ícones são gerados a partir de `logo_agilis.png` e `icone_agilis.png` (arte-fonte na raiz, fora de `public/`).

Solução de problemas

Sintoma	Causa provável
Servidor não inicia, fala em migração pendente	PostgreSQL sem migrar: rode `npm run migrate -- --postgres`

Sintoma	Causa provável
Servidor recusa iniciar sem configuração	`SGC_AUTH_MODE` padrão exige Supabase; configure o `.env` ou use o modo demo local
Login funciona no demo mas não em produção	O modo demo não vale em produção; use contas do Supabase
Erro 403 em toda mutação	`Origin` diferente de `SGC_PUBLIC_ORIGIN` ou CSRF ausente
Limite de tentativas atinge todos de uma vez	Falta `SGC_TRUST_PROXY` atrás de proxy
Datas deslocadas em um dia	Uso de getters locais de `Date`; use as funções de `logic.js` / `datas.js`

Glossário

Termo	Significado
Agilis	Nome de produto do SGC
SGC / SGA	Sigla do sistema (SGC no código e nos textos; SGA é o nome da pasta)
Ação	Unidade de trabalho com prazo, status, responsável, tempo, tarefas e impedimentos
Diretriz	Direcionamento do Diretor, que vira uma ação em cada seção destinatária
Tarefa	Item de apoio dentro de uma ação (`acao_checklist` no banco). Não muda o status
Impedimento	Algo que trava uma ação; `critico` deixa a seção vermelha. Histórico, nunca editado
Seção	Qualquer unidade da árvore: Centro, Coordenação, Subseção ou Diretoria Adjunta
Unidade de acompanhamento	Seção que vira cartão na reunião e linha do Painel: Centro, Coordenação, Diretoria Adjunta
Subseção	Seção abaixo de outra; nunca é cartão e soma no cartão acima
Diretoria Adjunta	Seção de primeiro nível, sem chefe, que substitui o Diretor nas ausências; membros têm perfil Administrador
Ação interna	Ação de subseção marcada `interna`; invisível ao Diretor e ao Apoio
Semáforo	Cor (vermelho/amarelo/verde) com forma e texto que resume a situação da unidade
Atualização semanal	Relato do chefe, versionado, congelado ao enviar
Fechamento	Segunda-feira, 18h
`refTerca` / referência da reunião	Data da reunião à qual as atualizações de hoje se referem
Pedido de prazo	Solicitação de novo prazo para ação criada por outra pessoa
Combinado	Regra de convivência lembrada na abertura da reunião; arquivado, nunca excluído
Decisão	Registro feito durante a reunião
Ata	Texto gerado ao encerrar a reunião; rascunho → enviada
Modo Reunião	Tela para TV com letras grandes, navegada por setas
Suplente	Segundo chefe da mesma seção, com os mesmos poderes do titular
Membro da equipe	Perfil que age só no que é dele
Consulta	Perfil de leitura das ações de todos os Centros
Plataforma	Perfil que administra empresas
Empresa	Órgão atendido; a Empresa 1 é o original
Vocabulário	Nomes dos tipos de unidade e do topo, próprios de cada empresa
Topo	Perfil mais alto da empresa (por padrão, "Diretor")
Ciência / vista	Registro de que a pessoa abriu a ação recebida
Ação parada	Em curso e sem movimentação há 7 dias ou mais
Esforço	Estimativa (pequeno, médio, grande) usada na produtividade
RLS	Row Level Security do PostgreSQL, habilitada pela migração 4

SGC — protótipo

Sistema de acompanhamento semanal dos Centros do Departamento de Planejamento, Orçamento e Gestão. Apoia a reunião presencial de terça-feira, 10h, entre o Diretor e os chefes de seção.

Este é um **protótipo com dados fictícios**, feito para validar o fluxo com o Diretor antes de construir a versão definitiva. Não use dados reais nele.

A documentação completa (visão geral, guia do usuário, regras de negócio, arquitetura, API, banco de dados, operação e glossário) está em docs/.

Como rodar

Requisitos: Node.js 22.12 ou superior (testado em Node 24.11.1).

```
npm install
SGC_AUTH_MODE=demo DATABASE_URL="" npm start
```

Abra http://127.0.0.1:3000. Nesse modo local, o banco (`data/sgc.db`, SQLite) é criado com dados fictícios. O modo demo escuta apenas loopback e é recusado com `NODE_ENV=production` ou na Vercel.

O modo padrão usa Supabase Auth (e-mail e senha) e exige configuração institucional antes de iniciar. Veja AUTENTICACAO.md para configurar o projeto, migrar e vincular contas. As credenciais de demonstração não funcionam nesse modo.

Comando	O que faz
`npm start`	Sobe o servidor (porta 3000; mude com `PORT=3100 npm start`)
`npm run dev`	Igual, reiniciando ao alterar arquivos do servidor
`npm run seed`	Apaga o banco e recria os dados fictícios
`npm run migrate -- --sqlite CAMINHO`	Aplica migrações ao arquivo SQLite indicado
`npm run migrate -- --postgres`	Aplica migrações usando `SGC_MIGRATION_DATABASE_URL` explícita
`npm run admin:criar -- --sqlite ARQUIVO "Nome" e-mail`	Cria o primeiro Administrador (senha provisória em `SGC_ADMIN_SENHA_INICIAL`; `--postgres` para PostgreSQL; `SGC_ADMIN_SECAO` põe na Diretoria Adjunta)
`npm run admin:secao -- --sqlite ARQUIVO e-mail ID	nenhuma`
`npm run check:auth`	Verifica configuração Supabase sem exibir a chave; `-- --online` consulta as configurações públicas do projeto
`npm test`	Roda os testes de regras, API e concorrência SQLite em memória, além do adaptador PostgreSQL com pool simulado

Comando	O que faz
`npm run test:postgres`	Cria PostgreSQL temporário e valida migrações, rollback e concorrência entre duas APIs; requer binários locais (instruções)

Com `DATABASE_URL` preenchida, a aplicação usa PostgreSQL e o comando de seed é recusado. Para uma demonstração SQLite isolada, use `DATABASE_URL="" SGC_DB=/tmp/sgc-demo.db npm run seed` e inicie com as mesmas variáveis e `SGC_AUTH_MODE=demo`. O seed aguarda o commit completo antes de terminar. No modo institucional não há carga fictícia automática.

PostgreSQL precisa estar migrado antes de iniciar esta versão; SQLite aplica as migrações ao iniciar. Consulte MIGRACOES.md para provisionamento, verificação de duplicidades e procedimento de atualização.

Variáveis de ambiente: `PORT`, `SGC_DB` (caminho do arquivo SQLite) e `SGC_NOW` (data e hora "de mentira", útil para demonstrar o fechamento de segunda 18h ou a terça da reunião, por exemplo `SGC_NOW=2026-09-22T09:30:00 npm start`; para os dados de exemplo fazerem sentido, use `npm run seed` com a mesma variável).

Calendário e fuso

A API e a interface usam `America/Bahia` para datas operacionais e horários. `SGC_NOW=2026-09-22T10:00:00` significa 10h na Bahia, mesmo em servidor UTC; valores com `Z` ou offset explícito representam o instante indicado. Timestamps antigos sem offset também são interpretados como horários de Bahia. Datas civis (`AAAA-MM-DD`) não mudam conforme o fuso do navegador.

A configuração do dia/horário é gravada integralmente ou desfeita em caso de erro. Consultas de semanas já registradas continuam disponíveis após mudar o dia da reunião. O corte semanal permanece às 12h no dia configurado, e o prazo regular segue no dia anterior às 18h; a política de corte para reuniões à tarde e as exceções de fechamento ainda precisam de validação. Correções após o prazo continuam permitidas nesta versão.

Como explorar

Na tela de entrada não há senha: escolha um perfil.

- **Diretor**: Painel da semana (semáforo por seção), Direcionar ação (para todos os Centros ou seções específicas, com prazo), Ações, Pedidos de novo prazo, Combinados, Reuniões e atas, Estrutura (criar seções e subseções, atribuir chefes) e o **Modo Reunião**.
- **Apoio do Diretor**: o mesmo do Diretor, exceto Estrutura. Pode conduzir o Modo Reunião.
- **Administrador**: acesso total: consulta todos os dados (inclusive as ações internas das subseções), faz tudo o que o Diretor, o Apoio e o Chefe fazem (direciona ações, decide prazos, conduz reunião, aceita e devolve ações, em qualquer seção) e gerencia contas, perfis e estrutura em "Contas e estrutura". É também o perfil de quem faz parte da Diretoria Adjunta, que substitui o Diretor nas ausências. Em modo institucional, o primeiro Administrador é criado com `npm run admin:criar` (veja AUTENTICACAO.md).

- **Chefe de seção**: Início, Minha atualização (feito, próximo, impedimentos, apoio), Minhas ações (status e tempo gasto em minutos), Histórico, Combinados e Atas. Vê a própria seção e as subordinadas.

Modo Reunião (para a TV)

Pelo menu "Iniciar reunião", que abre uma tela de conferência (seções sem atualização, pedidos de prazo, combinados) e só cria a reunião no botão **Iniciar reunião**; recarregar a página ou voltar no navegador não abre reunião nova, e uma reunião em andamento é retomada pelo botão **Retomar reunião**. Pensado para o notebook ligado por HDMI à TV da sala.

1. Abertura com os **Combinados da reunião** (conforme a frequência configurada).
2. Visão geral das seções, em ordem de necessidade (vermelhas primeiro).
3. Uma tela por seção: relato, ações, pedido de novo prazo (aprovar ou recusar na hora), decisões e novas ações.
4. Encerrar monta a **ata em rascunho**, que é revisada e "enviada" (nesta versão, apenas libera a leitura na tela Atas).

Teclas: seta direita avança, seta esquerda volta. Há também a "Versão para impressão" da pauta (plano B se a TV falhar).

Regras principais já implementadas

- Semana vai de terça a segunda; atualizações fecham na segunda, 18h. O semáforo é: vermelho (ação atrasada ou impedimento crítico), amarelo (atualização pendente ou ação vencendo em até 2 dias), verde (o resto).
- Ação concluída exige tempo registrado maior que zero minutos.
- Mudança de prazo só por pedido; o Diretor decide (o Apoio decide apenas dentro de uma reunião em andamento).
- Subseções: até 3 níveis. Ações internas de uma subseção não aparecem para Diretor e Apoio, só um resumo numérico.
- Combinados nunca são excluídos, apenas arquivados; cada reunião guarda uma cópia do que estava valendo.

Limites conhecidos desta versão

- Login Supabase Auth (e-mail e senha) implementado e testado com provedor simulado; a validação no projeto Supabase real e a inspeção visual da nova entrada ainda estão pendentes. A escolha de perfil por `x-user-id` só existe no modo demo local explícito.
- Sem e-mail nem lembretes. "Enviar ata" apenas libera a leitura no sistema.
- O chefe não tem tela própria para gerir subseções e ações internas (a estrutura existe no banco e é criada pelo Diretor).
- Sem trava automática do envio após segunda 18h (o chefe ainda consegue enviar correção).
- Sem relatórios, gráficos históricos ou anotações privadas do Diretor.
- Não foi testado com TV/HDMI reais nem em navegadores além do Chromium; a interface foi verificada em telas de 1920x1080, 1440x900 e 390 px de largura.

Estrutura do código

```
server/   API Express + SQLite (better-sqlite3) ou PostgreSQL (pg)
  app.js       monta o Express e registra as rotas
  rotas-estrutura.js  seções e usuários
  rotas-acoes.js      diretrizes, ações e pedidos de prazo
  rotas-semana.js     sessão, atualização semanal, painel, pauta
  contexto.js  auxiliares comuns das rotas (banco, relógio, configuração)
  reunioes.js  combinados, reuniões, decisões, ata
  logic.js     regras puras: semana, fechamento, semáforo
  db.js        esquema e consultas de árvore
  migrations.js migrações incrementais e verificação da versão
  migrate.js   comando de migração com destino explícito
  seed.js      dados fictícios
public/   interface (JavaScript puro, sem build)
  js/          uma tela por arquivo; main.js faz roteamento por hash
  js/regras.js status, transições, prioridades e ordem do semáforo, compartilhados com o
servidor
  estilo.css   sistema visual, modo TV e impressão
test/     testes da API
```

Para continuar o desenvolvimento com o Claude Code, abra esta pasta e leia o `CLAUDE.md`.

O andamento das correções e as limitações da validação estão em ESTABILIZACAO.md. O roteiro para testar o Modo Reunião na TV da sala está em ENSAIO_TV.md, e o da homologação com o Supabase real em HOMOLOGACAO.md.

CLAUDE.md — SGC (protótipo)

Sistema para gerir e acompanhar a produtividade semanal dos Centros do Departamento de Planejamento, Orçamento e Gestão. Uso principal: a reunião presencial de terça-feira, 10h, do Diretor com os chefes de seção (Centros e uma Coordenação), com TV HDMI na sala e notebook já disponível.

Idioma do produto, dos textos da interface, dos commits e dos comentários: português do Brasil. Nomes de código (tabelas, colunas, funções) também estão em português; mantenha.

Comandos

- `npm install` · `npm start` (porta 3000) · `npm run dev` · `npm run seed` (recria dados fictícios) · `npm test`
- `SGC_NOW=2026-09-22T09:30:00` muda o "agora" do servidor. Use em testes e demonstrações; nunca chame `new Date()` direto nas regras, use `agora()` de `server/logic.js`.
- Sempre rode `npm test` depois de mexer em `server/`. Para a interface não há teste automatizado: suba o servidor e confira no navegador (telas de 1920x1080 para o Modo Reunião e 390 px para o chefe).
- Datas: use as funções de `server/logic.js` e `public/js/datas.js`. O fuso de negócio é `America/Bahia`; datas civis usam aritmética UTC, e timestamps sem offset são interpretados nesse fuso. Não use getters locais de `Date` nas regras.
- Migrações: veja `MIGRACOES.md`. PostgreSQL verifica a versão ao iniciar e exige execução explícita do migrador; banco mais novo que o código só gera aviso (migrações são aditivas, ver `MIGRACOES.md`), migração pendente impede iniciar. SQLite aplica as migrações ao iniciar. Nunca remova dados duplicados automaticamente para instalar uma constraint.
- Integração PostgreSQL: `PG_BIN=/caminho/bin npm run test:postgres` cria um cluster descartável, sem usar `DATABASE_URL`. Veja `TESTES_POSTGRESQL.md`. Execute ao alterar transações, migrações ou SQL compartilhado; `npm test` continua sem exigir PostgreSQL.

Arquitetura

- `server/`: Express 4 + adaptadores SQLite (better-sqlite3, WAL, `foreign_keys` ligado) e PostgreSQL (pg). ES modules. Sem ORM; SQL direto. A interface de ambos os bancos é assíncrona: aguarde `get`, `all`, `run`, `seed` e os helpers de árvore. Use `await db.transaction(async () => { ... })`; transações aninhadas são rejeitadas. SQLite serializa consultas externas durante uma transação; PostgreSQL mantém o cliente por contexto assíncrono. A hierarquia é consultada no banco, sem cache local.
  - `app.js` só monta o Express, a segurança e as rotas, que ficam por assunto: `rotas-estrutura.js` (seções e usuários), `rotas-acoes.js` (diretrizes, ações e pedidos de prazo), `rotas-semana.js` (sessão, atualização semanal, painel e pauta), `reunioes.js` e `rotas-avisos.js` (quadro de avisos); `contexto.js` traz os auxiliares comuns (`q`, `q1`, `run`, `hoje`, configuração da reunião). Rota nova entra em `test/rotas.test.js`; `reunioes.js`

combinados, reuniões, decisões e ata; `helpers.js` painel, pauta e montagem de cartões; `logic.js` regras puras (status, transições, prioridades e ordem do semáforo vêm de `public/js/regras.js`, fonte única compartilhada com a interface; não duplique essas regras); `db.js` esquema e árvore de seções (CTE recursiva); `seed.js` dados fictícios.
    - Autenticação: `auth.js` implementa login e-mail/senha via Supabase Auth, sessão HttpOnly persistida por hash, CSRF e limite de tentativas. `configurarAuth()` falha sem configuração institucional; `SGC_AUTH_MODE=demo` permite `x-user-id` somente fora de produção. Nunca vincule identidades por e-mail nem confie em perfil enviado pelo cliente. Permissões de perfil/propriedade continuam nas rotas. Veja `AUTENTICACAO.md`.
    - Segurança usa `Date.now()` real, independentemente de `SGC_NOW`. Migração 4 habilita RLS e revoga acesso público aos objetos Agilis em PostgreSQL/public. Mudanças futuras exigem nova versão, inclusive para os arquivos auxiliares de migração.
- `public/`: SPA em JavaScript puro (módulos ES, sem build), rota por hash em `js/main.js`. Cada tela é uma função `(raiz, {id, q, refresh})` que preenche um elemento. Dialogs com `<dialog>` via `abrirForm` em `js/ui.js`. Delegação de eventos com `on(raiz, tipo, seletor, fn)`.
    - `js/agenda.js` (rota `#/agenda`, todos os perfis, inclusive Consulta): calendário do mês à esquerda e, à direita, o painel do dia escolhido, com as ações (pelo `prazo`) e as tarefas (pela `data`, atreladas às ações). Dados de `GET /acoes` (abertas + concluídas) e `GET /api/tarefas` (as tarefas das ações que a pessoa enxerga, mais as atribuídas a ela em ações de outra seção, marcadas `acao_visivel: false`: aparecem só como informação, sem abrir a ação). Para o Chefe é a agenda da seção; para Diretor, Apoio, Administrador e Consulta, a de todas as seções (seção em selo, filtro por seção). Cada dia é um botão com descrição por extenso (`aria-label`), navegável por setas; as marcas diferem pela forma (círculo = ação, quadrado = tarefa) e cada uma fica vermelha só se a própria ação ou tarefa está atrasada. Faixa "N itens atrasados" abre a lista das atrasadas; "Tarefas sem data" fica recolhida no fim do painel. Filtros, cada um com título: "Mostrar" (Tudo/Ações/Tarefas), "De quem" (Tudo ou "Só o que é meu", só para Chefe e Administrador com seção: as tarefas atribuídas a mim, em qualquer ação, e as ações da minha própria seção, sem as das subseções nem as de outras seções; uma dica embaixo diz o que está valendo) e "Seção" (só para quem vê todas as seções; fica desativado com "Só o que é meu", que já é uma seção); no celular ficam atrás do botão "Filtros", que conta quantos estão ativos. Quem pode marcar a tarefa (`pode_marcar`, vindo de `GET /api/tarefas`: chefe, Administrador e o membro dono da tarefa ou da ação) o faz direto na agenda (`PATCH /acoes/:id/checklist/:iid`); para o membro, "Só o que é meu" mostra as tarefas atribuídas a ele e as ações que ele executa; clicar na ação abre o mesmo `abrirAcao` do resto do app. O estado (mês, dia, filtros) vive só em memória.
    - `js/ata-marcacao.js`: marcação leve da ata (`**negrito**`, `*itálico*`, listas), compartilhada por `js/atas.js` (HTML, na tela) e `js/pdf-ata.js` (PDF), para as duas lerem o texto do mesmo jeito. `js/pdf-ata.js` monta e baixa o PDF inteiramente no navegador com jsPDF, vendorizado em `js/vendor/jspdf.umd.min.js` (sem CDN, licença MIT, carregado só quando o botão "Baixar PDF" é clicado). Papel A4, margem esquerda 2,5 cm e demais 2 cm, com numeração de página e a data da reunião no rodapé; só o texto do campo da ata entra no PDF, nada mais da tela.
    - Celular (até 760 px, `estilo.css`, seção "Celular"): app instalável com barra de topo compacta (`.topo-movel`), abas inferiores fixas (`.abas-movel`, definidas em `ABAS_MOVEL` em `main.js`: chefe Início · Ações · Atualização · Agenda; gestão Painel · Agenda · Ações ·

Prazos) e a folha "Mais" (`.folha-mais`) com o menu completo e a conta. A lateral some; no computador nada disso aparece. Diálogos viram folhas que sobem de baixo, com o rodapé fixo. Alvos de toque de 44 px ou mais, campos com fonte de 16 px, áreas seguras com `env(safe-area-inset-*)`. Ação principal de tela longa vai numa `.barra-acao` (fica fixa acima das abas). Tabela de ações leva `class="tabela-lista"` e `data-rotulo` nas células: no celular vira lista (título em cima, dados rotulados, Situação em linha própria). Filtros secundários ficam atrás de um botão "Filtros" no celular. Confira toda mudança de tela em 390x844, além do computador.
  - Ao buscar elementos numa tela, use sempre `$('#id', raiz)`: a tela é montada num elemento ainda solto do documento, então `document.querySelector` não o encontra.
  - `estilo.css` traz o sistema visual (paleta petróleo, títulos em serifa, semáforo com forma + texto), o modo TV (`body.tv`, classes `.tv-*`) e regras de impressão.
  - `imagens/logo-agilis.png` (barra lateral e telas de entrada) e os ícones (`favicon-32`, `apple-touch-icon`, `icon-192`, `icon-512`, `icon-maskable-512`) são gerados a partir de `logo_agilis.png` / `icone_agilis.png` na raiz (arte-fonte, fora de `public/`, não versionada). Instalável como aplicativo: `manifest.webmanifest` e `sw.js` (service worker deliberadamente sem cache — o sistema tem sessão e regras de negócio sensíveis a versão; ele só existe para satisfazer o critério de instalação). O botão "Instalar aplicativo" na barra lateral (`public/js/main.js`) só aparece quando o navegador dispara `beforeinstallprompt`.

Regras de negócio (fonte: plano do projeto; não altere sem combinar com o Diretor)

- Semana = terça a segunda. Fechamento: segunda 18h. `refTerca`: na terça depois das 12h, a referência passa para a terça seguinte.
- Semáforo por seção: vermelho se houver ação atrasada ou impedimento crítico aberto em ação em curso (tabela `impedimentos`; a marca `critico` de relatos semanais antigos ainda vale enquanto a tela do relato a oferecer); amarelo se a atualização não foi enviada ou há ação vencendo em até 2 dias; verde no resto. A cor de uma seção considera as subseções (vale a pior). Limiares ainda a validar com o Diretor.
- Ações: status `a_fazer`, `em_andamento`, `bloqueada`, `concluida`, com transições permitidas em `TRANSICOES` (`server/logic.js`). Concluir exige tempo registrado maior que 0 minutos (erro 422). O chefe registra tempo em minutos. Voltar o status para `a_fazer` exclui todo o tempo já registrado na ação (a ação recomeça do zero) e registra um comentário informando quanto foi excluído.
- Impedimentos pertencem a uma ação (`impedimentos`: descrição, `critico`, `apoio` opcional, autoria) e têm ciclo aberto → resolvido; são histórico, nunca editados nem excluídos (corrigir = resolver e registrar outro). Chefe (na própria árvore), Diretor, Apoio e Administrador registram e resolvem (`POST /api/acoes/:id/impedimentos` e `.../:iid/resolver`); Diretor e Apoio não veem ações internas. `bloquear` só vale com a ação em andamento; `retomar` volta a ação bloqueada para em andamento. Concluir a ação resolve os impedimentos abertos. Excluir a ação apaga os impedimentos dela.
- Tarefas dentro de uma ação (na tela, "Tarefas"; no banco e na API continuam `acao_checklist` e `/checklist`: texto, `concluido`, `ordem`, autoria, `data` de execução e `responsavel_id`, os dois

opcionais) são só apoio: não mudam o status nem entram na regra de concluir (que continua exigindo tempo registrado). A lista sai ordenada por data, as sem data no fim (`porDataDaTarefa` em `public/js/regras.js`, usado pelo servidor e pela tela). Tarefa com data passada e não feita leva `atrasada` (só destaque visual, não muda semáforo nem status). O responsável é escolhido entre as contas ativas vinculadas à seção da ação, às seções acima e às subseções (`GET /api/acoes/:id/responsaveis`, só para quem gere as tarefas); o servidor recusa outro. Conta responsável por tarefa não é excluída (use Desativar). Diferente de impedimentos, tarefas podem ser editadas e excluídas livremente (`POST/PATCH/DELETE /api/acoes/:id/checklist[/:iid]`). Mesma regra de quem mexe: quem executa a ação (o chefe da seção, o Administrador e, nas dele, o membro) — igual a quem registra tempo e muda status; o membro dono da tarefa também a marca como feita; Diretor e Apoio só veem. Bloqueado só se a ação já foi encerrada pelo Diretor. `checklist_total` / `checklist_feitos` aparecem em toda ação para o contador nas listas.

- Redirecionar ação (`POST /api/acoes/:id/redirecionar`, `{ secao_id, motivo }`): muda a seção responsável levando todo o histórico (status, prazo, tempo, checklist, impedimentos, pedidos e comentários) e registra um comentário com origem, destino e motivo. Diretor, Apoio e Administrador escolhem qualquer seção ativa; o chefe, só a própria seção ou as subseções dela. Não vale para ação concluída, encerrada ou arquivada, nem para uma seção que já recebeu a mesma diretriz.
- Ao direcionar (tela "Direcionar ação" e "Nova ação" do Modo Reunião), antes de criar, `confirmarPrazoNasSecoes` (`public/js/acao-comum.js`) mostra as ações em aberto com o mesmo prazo nas seções alvo e nas subseções delas (`GET /api/acoes?situacao=abertas&prazo=AAAA-MM-DD`); quem direciona confirma ("Direcionar mesmo assim") ou volta para revisar. É só aviso: o servidor não bloqueia.
- Acrescentar seções a um direcionamento (`POST /api/diretrizes/:id/secoes`, `{ secoes, prazo? }`; Diretor, Apoio e Administrador, botão "Acrescentar seções" em "Últimas ações direcionadas"): cada seção nova recebe a mesma ação (título, detalhamento, prioridade), ligada à mesma diretriz; as que já a receberam não mudam. Prazo: o da diretriz ou outro informado (obrigatório quando o da diretriz já passou). Mesmas regras de destino de `criarDiretriz` (primeiro nível, ativa, Diretor Adjunto não acrescenta a própria seção). Diretriz "Todos os Centros" que ganha seção passa a `especificos`. `GET /api/diretrizes` traz `secoes` (as seções que têm a ação hoje).
- Diretriz do Diretor vira uma ação por seção destinatária (todos os Centros ou seções escolhidas), sempre com prazo.
- Autoria (`ehAutor` e `permissoes` em `server/rotas-acoes.js`): quem responde pela ação como autor é o chefe ou suplente que a criou (`criado_por`), e o titular também pelas do suplente da sua seção; o Diretor e o Apoio, pelas criadas pela gestão (Diretor, Apoio ou Diretor Adjunto); o Administrador, por todas. O autor edita título e detalhamento (`pode_editar`) enquanto a ação não foi apresentada em reunião; muda o prazo (`pode_mudar_prazo`) enquanto ela está em curso; exclui (`pode_excluir`) só se ela não foi apresentada nem concluída — ação concluída ou apresentada não é excluída por ninguém, nem pelo Administrador. Status, tempo, checklist e impedimentos continuam com quem executa (o chefe da seção); a prioridade, com quem executa ou com o autor. Toda edição com mudança registra um comentário ("Ação editada: …").
- Ação sem atualização (`acoes.movimentada_em`, `DIAS_SEM_MOVIMENTO` = 7 em `public/js/regras.js`): toda gravação bem-sucedida em `/api/acoes/:id/...` (status, prioridade, edição, tempo, comentário, checklist, impedimento, pedido de prazo, redirecionamento, arquivamento, aceite) e a decisão de pedido de prazo renovam a data, por um middleware em `rotas-acoes.js` que grava

antes da resposta. Ação nova usa `criada_em`. A API devolve `movimentada_em` e `parada` (em curso e sem movimentação há 7+ dias). O Início do chefe (e do Diretor Adjunto, "Início da seção") tem o quadro "Precisa de atenção": atrasadas, sem atualização e com prazo até a reunião, com botões rápidos de status e tempo. O Painel e o cartão da reunião trazem `paradas` por seção (só ações visíveis ao Diretor). É só informativo: não muda o semáforo.

- Ação recebida (`acoes.recebida_em`, tabela `acao_vistas`): a ação "chega" à seção quando vem de fora — diretriz, seção acrescentada, ação criada pela gestão para a seção (`POST /api/acoes` por quem não é da seção) ou redirecionamento (que renova a data). Quem recebe: os chefes que enxergam a seção (titular, suplente e o chefe do Centro acima) e, na Diretoria Adjunta, os membros dela. Para cada um ela é `nova` (campo na lista e no detalhe) enquanto está em curso e ele não a abriu desde a chegada; abrir o detalhe chama `POST /api/acoes/:id/vista`, que guarda a primeira ciência e não conta como movimentação. Quem redireciona dentro da própria árvore já fica ciente. Para o membro, a ação chega quando o chefe a atribui a ele (`atribuida_em`). O bootstrap traz `acoes_novas` (selo de Minhas ações); o Início tem o quadro "Chegaram para você". A gestão vê a ciência em `vistas` (detalhe) e em `GET /api/diretrizes` (`secoes[].vista_em` / `vista_por`, tela "Últimas ações direcionadas"). Ciência é histórico: conta com registro em `acao_vistas` não é excluída (use Desativar); excluir a ação apaga a ciência dela. Só aviso dentro do sistema, sem e-mail.
- Ação apresentada em reunião (`acoes.apresentada_em`, data da primeira reunião): ao encerrar a reunião, as ações dos cartões (`cartoesReuniao`) ficam marcadas; reabrir e encerrar de novo não muda a data.
- Histórico de prazos (`acao_prazos`): toda mudança de prazo grava o anterior e o novo, com quem e quando, por edição do autor (`origem = 'edicao'`) ou por pedido aprovado (`'pedido'`, com `pedido_id`). `prazo_original` continua sendo o primeiro. O detalhe da ação traz `prazos`.
- Prazo de ação criada por outra pessoa só muda por pedido (`pedidos_prazo`). O Diretor decide; o Apoio só decide dentro de reunião em andamento (`reuniao_id`), por delegação. O autor altera o prazo direto pela edição.
- O Diretor Adjunto (Administrador com `secao_id` na Diretoria Adjunta) não direciona ações (diretrizes) para a própria seção: as dela ele cria em "Minhas ações". O Diretor direciona para a Diretoria Adjunta normalmente.
- Perfis: Diretor, Apoio, Chefe (vê a própria seção e as subordinadas), Membro da equipe (veja a regra própria abaixo), Consulta e Administrador (acesso total por decisão do proprietário: vê todas as ações, inclusive internas, e faz tudo o que o Diretor, o Apoio e o Chefe fazem, em qualquer seção, além de gerir contas, perfis e estrutura; nas rotas de atualização semanal informa `secao_id`. Continuam valendo as travas de integridade: sempre um Diretor ativo e Administrador só criado por `npm run admin:criar`). Administrador só nasce por `npm run admin:criar`; a API não cria nem promove. O Diretor cria Chefe e Apoio; o Administrador também cria Diretor. Sempre deve existir um Diretor ativo. Contas com senha definida por terceiros exigem troca no primeiro acesso (`usuarios.trocar_senha`). Rota nova de gestão inclui `'administrador'` em `permit` e, se listar ações, respeita `visiveisAcoes`. Suplente: um segundo Chefe para a mesma seção (mesmo `secao_id`), com os mesmos poderes do titular, para responder na ausência dele; não muda `secoes.chefe_id`. Cadastrado por Diretor ou Administrador em `POST /api/usuarios/acesso` com `suplente: true` (só perfil `chefe`; pula a confirmação de substituição). `GET /api/usuarios` marca `suplente: true` quando `secao_id` está preenchido e a conta não é a `secoes.chefe_id` daquela seção. Autoria de ações, tempo, impedimentos e do relato semanal (`atualizacoes.usuario_id`,

agora com `usuario_nome` nas consultas) distingue titular e suplente. Sem expiração automática: ativar/desativar é manual, como qualquer conta.

- Leitura das ações de todos os Centros (só leitura, sem ações internas de outros Centros, sem relatos): o perfil **Consulta** (`consulta`, sem seção) e o chefe a quem Diretor ou Administrador ligam `usuarios.ver_outros_centros` (`PATCH /api/usuarios/:id`, só para perfil `chefe`; trocar o perfil desliga). Os dois usam a tela "Ações dos Centros" (`#/acoes-centros`, `public/js/consulta.js`). No servidor, `visiveisAcoes` continua sendo o escopo de quem altera (o chefe, a própria árvore) e vale para toda rota que grava; a leitura ampliada (`visiveisLeitura`) vale só para `GET /api/acoes?escopo=todos` e `GET /api/acoes/:id`, que marcam `somente_leitura` (e zeram `pode_*`) nas ações de fora da árvore. O detalhe com `somente_leitura` não mostra nenhum controle nem campo de comentário. O perfil Consulta tem lista fechada em `app.js`: só `GET /api/bootstrap`, `GET /api/acoes`, `GET /api/acoes/:id` e `GET /api/tarefas` (além de entrar, sair e trocar a senha); qualquer outra rota responde 403 — rota nova não precisa fazer nada para ficar fechada a ele. Diretor e Administrador criam e editam contas Consulta.
- Membro da equipe (`usuarios.perfil = 'membro'`, com `secao_id`; não é suplente: o suplente é um chefe a mais): lê todas as ações e tarefas da árvore da própria seção (como o chefe dela, inclusive as internas) e age só no que é dele. `executa(user, ação)` em `rotas-acoes.js` decide: chefe da seção e Administrador, sempre; membro, só nas ações atribuídas a ele (`acoes.responsavel_id`) ou criadas por ele. Quem executa muda status, prioridade e tempo, cria/edita/exclui tarefas, registra impedimentos e comenta; título, detalhamento e prazo continuam com o autor (o membro, nas que criou; o chefe responde também pelas criadas por membros da sua seção). O membro não atribui, não arquiva, não redireciona, não pede prazo, não valida e não exclui o que não criou. Em ação de outra pessoa, ele só marca como feita (ou reabre) a tarefa atribuída a ele e comenta onde tem tarefa sua; o resto é somente leitura. O servidor devolve em cada ação `pode_executar`, `pode_comentar`, `pode_atribuir` e `somente_leitura` (e `pode_marcar` em cada tarefa): a tela esconde os controles por essas marcas, não pelo perfil. Lista fechada de rotas em `app.js` (`ROTAS_MEMBRO`): fora dela (relatos, reuniões, atas, combinados, estrutura, contas, aceite, validação, arquivar, redirecionar, pedir prazo) responde 403. O chefe (e o Administrador com seção) cadastra a equipe em "Minha equipe" (`GET/POST /api/equipe`, `PATCH /api/equipe/:id` para desativar/reativar; redefine a senha por `PATCH /api/usuarios/:id/login`, sem trocar e-mail); o Diretor e o Administrador também criam e mudam o perfil em Configuração. Desativar não apaga: as ações atribuídas continuam com o nome da pessoa, para o chefe reatribuir. O membro entra em "Minhas ações" (abas "Minhas" e "Da seção, só leitura") e na Agenda.
- Responsável pela ação (`acoes.responsavel_id`, `atribuido_por`, `atribuida_em`): um só, escolhido pelo chefe (ou Administrador) entre chefes, membros (e, na Diretoria Adjunta, Administradores) da seção e subseções (`GET /api/acoes/:id/executores`), ao criar a ação ou depois (`PATCH /api/acoes/:id`, só em ação em curso; registra comentário "Responsável: X → Y"). Quem assume a própria ação não "delega" a si mesmo. A ação atribuída chega como `nova` para o membro (mesmo mecanismo de `acao_vistas`, a partir de `atribuida_em`). A gestão não atribui.
- Validação de quem delegou (`validacaoDe` em `rotas-acoes.js`; `aguardando_validacao` = `chefe`, `diretor` ou vazio): só duas origens têm validação depois de a ação ser concluída. (1) Atribuída pelo chefe a um **membro** (`delegada`): o chefe da seção valida (`POST /api/acoes/:id/validar`) ou devolve para ajuste (`POST .../devolver`, com o motivo); titular e suplente podem. (2) **Demandada pelo Diretor** (diretriz ou "Demanda do Diretor" declarada): o Diretor aceita (`.../encerrar`) ou devolve. Quando as duas valem, em sequência: primeiro o chefe, depois o

Diretor (o Diretor só aceita depois da validação do chefe). Cada um só devolve a etapa que lhe cabe; o Administrador age em qualquer etapa. As demais ações (criadas pela seção ou pelo membro para si, internas, demanda externa sozinha) ficam prontas ao serem concluídas: o Diretor não aceita nem devolve. Mudar o status, reabrir ou trocar o responsável recomeça a validação do chefe. O Início do chefe tem o quadro "Aguardando sua validação" (contagem `acoes_a_validar` no bootstrap, somada ao selo de Minhas ações).

- Demanda de entidade externa (`acoes.demandante_externo`, texto de até 120 caracteres): marca a ação demandada por entidade fora da subordinação ao Diretor (órgão de controle, outra secretaria etc.). O nome é obrigatório quando marcada; é exclusiva da "Demanda do Diretor" (uma origem só); torna a ação visível ao Diretor (nunca interna) e não pode ser mudada numa ação que veio de diretriz. Quem a declara é o autor, ao criar ou editar a ação. Aparece com selo na lista, no detalhe, na Agenda e nas telas de leitura; o Diretor filtra por ela em Ações (`GET /api/acoes?origem=externa`). Não muda semáforo nem exige validação do Diretor.
- Multi-empresa (**em implantação; existem as Fases 1 e 2**; plano, decisões e regras em `MULTIEMPRESA.md`): o órgão atual é a Empresa 1 (`empresas`, migração 22) e as tabelas raiz têm `empresa_id` (padrão 1). Todo SQL de tabela raiz filtra por `empresaAtual()` (`server/empresa.js`: a empresa vem da conta logada, nunca do cliente); em `visiveisAcoes` o `a.empresa_id = ?` é sempre o primeiro marcador do `where`. Ao gravar, seção, pai, chefe, responsável, reunião e participantes precisam ser da mesma empresa. `SGC_LINT_EMPRESA=1 npm test` falha em SQL de tabela raiz sem `empresa_id` dentro de uma requisição; a consulta que precisa olhar todas as empresas (e-mail único) leva `/*global*/`. Rota ou consulta nova entra com teste em `test/isolamento-empresas.test.js`. A entrada pede só e-mail e senha: o servidor descobre o órgão pela conta vinculada (não pelo domínio do e-mail); conta ou órgão inativo respondem como senha errada, e `empresa` no corpo do login é só uma restrição opcional (`GET /api/auth/empresas` serve à demonstração). Migrações não chamam helpers ao vivo (a 13 tem a própria consulta). **Não crie a 2ª empresa antes de a migração 23 (contração: chaves únicas por empresa) ser aplicada em produção**; `empresa:criar` recusa enquanto o esquema não estiver contraído. O padrão `empresa_id = 1` ainda existe (endurecimento futuro). `npm run empresa:criar` (`server/criar-empresa.js`) cria a empresa e o primeiro Diretor (senha provisória com troca obrigatória) e recusa a 2ª empresa enquanto o esquema não passou pela contração. Identidade por empresa (Fase 4): o bootstrap traz `empresa {nome, sigla, slug}`, mostrada sob o logo e no título da aba; o cabeçalho da ata e os dois brasões (`ata_brasao_esq`, `ata_brasao`) são `config` da empresa; sem `ata_brasao_esq` salvo, só a Empresa 1 (`slug` `dpog`) mostra o brasão antigo da PMBA (`brasaoEsquerdo` em `public/js/ata-cabecalho.js`). Tela "Empresas" (`public/js/plataforma.js`, rotas `/api/plataforma/empresas`, `server/rotas-plataforma.js`): perfil `plataforma` (migração 24), só por `npm run plataforma:criar`, numa empresa interna `plataforma` fora da lista da entrada (entra por `/?empresa=plataforma`); cria empresa + primeiro Diretor, edita nome/sigla, tira da lista da entrada e desativa/reativa (desativar derruba as sessões); lista fechada em `app.js` (só `/bootstrap` e `/plataforma/*`). Vocabulário por empresa (organograma, etapa 1, ver `MULTIEMPRESA.md`): `config.vocabulario` por empresa, módulo `public/js/vocabulario.js` (puro, usado pelo servidor e pela tela) e `vocab.js` (atalhos da tela); texto de interface e de mensagem que cite Centro, Coordenação, Subseção, Diretoria Adjunta ou o perfil Diretor deve vir do vocabulário (`todasAs`, `dosPlural`, `nomeTipo`, `rotuloPerfilDe`…), não fixo no código. Tipos opcionais por empresa (`vocabulario.tipos.<tipo>.usa`, só Coordenação de primeiro nível e Diretoria Adjunta; `usaTipo`): o texto e o formulário que oferecem um desses tipos devem checar `usaTipo`. Etapas 2 e 3 do organograma prontas (posição livre da árvore e cartões por tipo, sem

migração). O nome do topo (perfil Diretor) nas frases: na tela use `oTopo()`, `doTopo()`, `aoTopo()`, `peloTopo()`, `OTopo()` (`public/js/vocab.js`), nunca "o Diretor" fixo; nas mensagens do servidor escreva a marca no texto (`{topo:o}`, `{topo:O}`, `{topo:do}`, `{topo:ao}`, `{topo:pelo}`), que o tratador de erros de `app.js` troca pelo nome e gênero da empresa de quem recebe (evite adjetivo que concorde com o topo). Falta a remoção do padrão `empresa_id = 1`. Antes de migrar produção, conte os registros com `scripts/contagem-registros.js`.

- Produtividade das unidades (`server/produtividade.js`, `GET /api/produtividade?de&ate[&secao]`, tela `public/js/produtividade.js`): avalia **unidades** (cartões e, abaixo, as subseções), não pessoas isoladas, sempre comparando com o período anterior de mesma duração. Indicadores: entregas (ações concluídas no período, com pontos pelo esforço estimado: pequeno 1, médio 2, grande 3, sem estimativa vale médio), pontualidade (concluída até o prazo vigente, e contra o prazo original), retrabalho (concluídas já devolvidas na validação, `acoes.devolucoes`), tempo de ciclo (da atribuição, recebimento ou criação até a conclusão; execução desde `acoes.iniciada_em`), dedicação (tempo lançado) e carga de hoje (em aberto, atrasadas, paradas, bloqueadas). Só soma as ações que quem consulta vê (`visiveisAcoes`): o Diretor e o Apoio não veem as internas e recebem só a contagem do que ficou de fora (`ocultas`). Quem vê: Diretor, Administrador e Apoio (todas as unidades) e o chefe (a própria unidade); **por pessoa** só o Diretor, o Administrador e o chefe da unidade (nunca o Apoio, o membro nem a Consulta). O esforço (`acoes.esforco`) é informado por quem cria ou executa a ação (e na diretriz); a devolução na validação soma em `devolucoes`; a primeira ida a "em andamento" grava `iniciada_em`. Não compare unidades de natureza diferente nem use um indicador sozinho.
- Diretoria Adjunta (`secoes.tipo = 'diretoria_adjunta'`, `DIRETORIA_ADJUNTA` em `public/js/regras.js`): seção de primeiro nível, no máximo uma, sem chefe; Centros, Coordenações e Subseções podem ficar abaixo dela (cada Centro e Coordenação é um cartão à parte). No resto é acompanhada como as demais: recebe ações (diretas, redirecionadas e diretrizes para seções escolhidas, mas não "todos os Centros", porque não é Centro), entra no Painel, no semáforo, no detalhe e na reunião, e envia relato semanal. Não vira nem deixa de ser Diretoria Adjunta por edição. Quem faz parte dela tem o perfil Administrador, com `usuarios.secao_id` apontando para ela, e só entra pelo servidor: `SGC_ADMIN_SECAO` em `npm run admin:criar` ou `npm run admin:secao` para quem já é Administrador. O vínculo não muda o acesso total do Administrador; dá a ele a tela "Minha atualização" da seção (sem `secao_id`, as rotas de atualização usam a seção do próprio Administrador). `GET /api/secoes` traz `membros` na Diretoria Adjunta. Depois do login, cai direto em "Início da seção" (`casaDe()` em `public/js/main.js`), a mesma tela do chefe.
- Quadro de avisos (`avisos`): uma mensagem do Diretor ou do Administrador, com janela de vigência (`data_inicio`, `data_fim`, ambas `AAAA-MM-DD`; `data_fim` obrigatória e nunca no passado). Aparece em destaque na tela Início do chefe e da Diretoria Adjunta (`GET /api/avisos/atual`, aberta a qualquer perfil), no máximo uma por vez: a vigente (`ativo = 1` e hoje dentro da janela) de maior id. `POST` / `PATCH` / `DELETE /api/avisos` são só de Diretor e Administrador (não o Apoio); exclusão é definitiva, diferente de combinados — não é documento de reunião.
- Seção na reunião (`secoes.na_reuniao`, padrão 1, qualquer seção; Diretor e Administrador mudam em `PATCH /api/secoes/:id`): fora da reunião, a seção sai dos cartões do Modo Reunião, da pauta e dos impedimentos críticos da ata, e leva junto as subseções abaixo dela (`arvoresDosCentros(db, { reuniao: true })`). O Painel continua mostrando todas, com a marca "fora da reunião".
- Seções em árvore, com os níveis configuráveis por empresa (`vocabulario.niveis`, 2 a 6, padrão 3; a Diretoria Adjunta conta como um nível; não dá para limitar abaixo da profundidade atual).

Posição por tipo: Centro e Coordenação ficam no primeiro nível ou diretamente sob a Diretoria Adjunta; a Diretoria Adjunta, só no primeiro nível; a Subseção, sob qualquer seção. **Unidade de acompanhamento** (cartão no Modo Reunião, linha do Painel e do semáforo, alvo de "Todos os Centros"): Diretoria Adjunta, Centro e Coordenação, onde estiverem na árvore (`TIPOS_DE_ACOMPANHAMENTO`, `TIPOS_DE_DIRETRIZ_GERAL`: sem a Adjunta); a Subseção nunca é cartão e entra na soma do cartão acima. `subarvore` (`db.js`), `arvoresDosCentros` (`helpers.js`), o relato, "Minha equipe" e a visibilidade do chefe e do membro descem **só por Subseções**: a Adjunta não herda as ações dos Centros abaixo dela. Ações internas de subseções (`interna = 1`) não aparecem ao Diretor nem ao Apoio, só contagens no resumo.

- Atualização semanal do chefe é versionada (`secao_id` + `semana` + `versao`); enviar de novo cria nova versão e o histórico é mantido. Exceção deliberada: o Administrador exclui qualquer versão do histórico (`DELETE /api/atualizacoes/:id`), para corrigir um relato de teste ou enviado por engano; sem cascata (nenhuma tabela referencia `atualizacoes`). Excluir a versão mais recente revela a anterior sozinho, já que a leitura sempre pega a de maior `versao`; excluir a única versão volta a seção para pendente.
- O relato é montado pelo servidor a partir das ações e dos impedimentos da seção e das subseções (`server/relato.js`). Concluídas: de 7 dias antes da reunião até a véspera (dia da Bahia, não UTC). Programadas: do dia da reunião até 7 dias depois. Atrasadas: prazo anterior a hoje e não concluídas (a mesma regra do semáforo). Vencem antes da reunião: em aberto com prazo de hoje à véspera (só na tela do chefe, fora da cópia congelada). Impedimentos: os abertos em ações em curso. `GET /api/atualizacao` devolve o relato ao vivo (com ações internas, marcadas) e `desatualizada` quando mudou desde o último envio; `?resumo=1` pula o relato ao vivo. O envio (`PUT /api/atualizacao` só com `semana` e `observacoes`) recalcula no servidor e congela `feito.snapshot` (formato 2), que **nunca contém ações internas, só contagens**; os campos antigos (`feito.extras`, `proximo`, `impedimentos`, `apoio`) são derivados da cópia para as telas ainda não migradas, e `critico` fica 0 (o semáforo lê os impedimentos). O corpo antigo com listas de texto continua aceito (legado, a remover). Diretor, Apoio e Administrador leem o relato por `public/js/relato-vista.js` (mesma forma nos dois formatos): feito e próximo como foram reportados, impedimentos e apoio como estão abertos agora (`cartoesReuniao` traz `impedimentos` por seção, só de ações visíveis ao Diretor). A ata lista os impedimentos críticos em aberto.
- Combinados da reunião: CRUD completo, mas **arquivar, nunca excluir**. Cada reunião guarda `combinados_snapshot`. Aviso quando há mais de 7 ativos (`LIMITE_COMBINADOS`). Frequência de exibição: `sempre`, `primeira_do_mes`, `quando_mudarem` (tabela `config`). Não entram no texto da ata (`gerarAta()`): já aparecem à parte, na tela da reunião e no `combinados_snapshot`.
- Reunião: `em_andamento` → (encerrar) `rascunho` → (enviar) `enviada`. Ata é gerada ao encerrar e Diretor, Apoio e Administrador podem editá-la em rascunho ou depois de enviada, e excluí-la (`DELETE /api/reunioes/:id`, só com a reunião encerrada; apaga as decisões e desvincula ações e pedidos, que permanecem). "Reabrir" só a partir de rascunho. `ata_texto` continua texto puro no banco (sem mudança de schema), com marcação leve inspirada em markdown (`**negrito**`, `*itálico*`, linha começando com `- ` ou `1. ` vira lista); a barra de formatação só manipula o `<textarea>`, e `renderizarAta()` (`public/js/atas.js`) escapa o texto antes de aplicar as tags, então não há risco de HTML injetado pelo campo. Ata antiga sem marcação continua exibida normalmente, como parágrafo.
- Modo Reunião: tela para TV, letras grandes, setas do teclado, semáforo sempre com forma e texto (não só cor). No cartão da seção, ações e decisões são editadas ao vivo, sem sair da reunião: a ação abre o mesmo diálogo do resto do app (`abrirAcao`, em `acao-comum.js`); decisão

tem edição e exclusão próprias (`PATCH`/`DELETE /api/reunioes/:id/decisoes/:decisaoId`, só quem gere reuniões, só com a reunião em andamento). A coluna de ações e decisões (`.tv-acoes`) rola verticalmente quando o conteúdo não cabe, sem empurrar o relato da seção ao lado.

Fora do protótipo (fase 2, em ordem sugerida)

1. Login real (contas institucionais, sessão, perfis) e trava de permissões no servidor além do cabeçalho simulado.
2. Lembretes por e-mail (sexta 15h) e envio real da ata; fechamento automático da segunda 18h.
3. Tela do chefe para subseções e ações internas.
4. Relatórios e histórico (produtividade por seção, tempo por ação), anotações privadas do Diretor.
5. Implantação: decidir hospedagem (servidor do órgão ou nuvem) e segurança; trocar SQLite por outro banco só se o volume ou a concorrência pedirem.

Convenções

- Erros de API: `{ erro: 'mensagem em português' }` com o status HTTP adequado (400, 403, 404, 409, 422). A interface mostra `e.message` ao usuário, então escreva mensagens que orientem o que fazer.
- Toda rota nova valida perfil (`permit`) e propriedade (chefe só mexe na própria seção). Escreva um teste em `test/api.test.js`.
- Textos da interface: verbos claros ("Direcionar ação", "Enviar aos chefes"), o mesmo nome do botão na confirmação. Sem jargão técnico.
- Datas em ISO (`AAAA-MM-DD`) no banco e na API; formate com `br()` na tela.
- Dados de exemplo são fictícios; não inclua dados reais do Departamento no repositório.

Login por e-mail e senha com Supabase Auth

O Agilis autentica com o Supabase Auth (e-mail e senha). O navegador envia as credenciais somente à API Express; o servidor as repassa ao Supabase, confere o usuário retornado e descarta senha e tokens. Em seguida emite uma sessão própria, opaca, em cookie `HttpOnly`. O navegador nunca recebe tokens do Supabase, e o frontend não usa chave `anon`, `publishable` ou `service_role`.

Configuração do Supabase

1. No painel do projeto, em Authentication, use o provedor de e-mail e senha. Desabilite o cadastro público (*Allow new users to sign up*): as contas são criadas pelo administrador.
2. Exija confirmação de e-mail. O Agilis recusa contas sem `email_confirmed_at` e contas anônimas.
3. Em Authentication → URL Configuration, informe a origem pública do Agilis como Site URL.
4. Crie os usuários no painel (Authentication → Users) e anote o **UUID** de cada um (`AUTH_USER_ID`). Redefinição de senha e convites são feitos pelo administrador; o Agilis não tem tela para isso.
5. Configure as políticas de senha e de MFA disponíveis no plano do projeto. O login do Agilis não solicita segundo fator: se o projeto exigir MFA, o login por senha não será concluído.

Ambiente e implantação

Node.js 22.12 ou superior. Configure no servidor:

Variável	Valor
`SGC_AUTH_MODE`	`supabase` (padrão)
`SGC_PUBLIC_ORIGIN`	Origem HTTPS fixa, por exemplo `https://agilis.exemplo.gov.br`, sem caminho
`SUPABASE_URL`	URL do projeto, por exemplo `https://projeto.supabase.co`
`SUPABASE_PUBLISHABLE_KEY` ou `SUPABASE_ANON_KEY`	Chave pública do projeto. Chaves administrativas (`sb_secret_`, `service_role`) são recusadas
`DATABASE_URL`	Conexão PostgreSQL do backend
`SGC_MIGRATION_DATABASE_URL`	Destino explícito usado pelo migrador e pelo comando administrativo
`SGC_PG_CA_FILE`	Caminho da CA oficial do banco, caso necessário para verificar o certificado

A aplicação recusa configuração incompleta antes de abrir o banco. HTTPS é obrigatório, exceto loopback em desenvolvimento. O proxy deve preservar o cabeçalho `Origin`, que é comparado com `SGC_PUBLIC_ORIGIN` em login e mutações.

Antes de iniciar, aplique `npm run migrate -- --postgres` no destino configurado. As migrações 4 e 5 criam as tabelas de autenticação e restringem permissões PostgreSQL. A conexão do backend precisa ser proprietária das tabelas ou usar um papel administrado com `BYPASSRLS`; nenhum segredo desse papel vai para o navegador. O modo institucional não cria usuários fictícios: prepare usuários locais, perfis e seções por procedimento administrativo e só então vincule as contas.

Vincular uma conta

Confira o ambiente com `npm run check:auth`. O comando verifica presença e formato das configurações; não acessa banco ou rede, não altera `.env` e não exibe a chave. Retorna código 1 se houver pendências. Com `npm run check:auth -- --online`, também consulta `/auth/v1/settings` do projeto, enviando apenas a chave pública. Isso não comprova validade das credenciais, existência de usuários nem vínculos.

O vínculo usa **URL do projeto + UUID do usuário Supabase**, nunca o e-mail. Nome, e-mail e claims não atribuem permissões.

Com o destino administrativo configurado e o usuário local já cadastrado:

```
node server/vincular-identidade.js --postgres USUARIO_ID SUPABASE_URL AUTH_USER_ID
```

Para um SQLite explicitamente indicado:

```
node server/vincular-identidade.js --sqlite /caminho/sgc.db USUARIO_ID SUPABASE_URL
AUTH_USER_ID
```

O comando verifica migrações e usuário ativo, é idempotente para o mesmo vínculo e recusa substituição ou apropriação de vínculo existente. Não altera perfil ou seção. Não existe endpoint público de cadastro ou vinculação. Para revogar acesso imediatamente, desative o usuário local (a API consulta o estado ativo e o perfil em toda requisição) e desative a conta no Supabase. Para trocar um vínculo, remova antes as sessões anteriores por procedimento administrativo.

Sessão e proteção das requisições

- Sessão opaca em cookie `HttpOnly`, `SameSite=Lax`, `Secure` em HTTPS (nome `__Host-sgc_senha`), sem domínio e com caminho `/`. O banco guarda só o hash do token (`auth_sessoes_senha`).
- A sessão vale por uma hora sem atividade e é renovada enquanto o usuário usa o sistema (no máximo uma gravação a cada 30 minutos por sessão), até o teto absoluto de 8 horas desde o login; depois disso o usuário entra de novo. Não há refresh token do Supabase: ele só é consultado no login. O Modo Reunião envia um ping a cada 10 minutos para não expirar em reuniões longas. Sessões abertas antes da migração 7 mantêm o prazo original.
- Login exige `Origin` igual à origem configurada e JSON. Mutações exigem também o token CSRF recebido no login/bootstrap. O cabeçalho `x-user-id` é ignorado fora do modo demo.
- Limite de tentativas em janelas de 15 minutos: 10 por e-mail e 100 por origem (`auth_tentativas`). Mensagens de erro não revelam se o e-mail existe.

- Sair remove a sessão do Agilis. Desativações urgentes devem desativar também o usuário local.
- APIs retornam `Cache-Control: no-store`. O cliente limpa seu cache e rejeita respostas pendentes da sessão anterior ao sair.
- O relógio de autenticação é o real do servidor e ignora `SGC_NOW`.
- Limites adicionais de taxa e proteção contra abuso devem ser configurados na infraestrutura de entrada antes da exposição pública.

Banco e Supabase

O frontend usa exclusivamente a API Express. As migrações 4 e 5 habilitam RLS e revogam acesso de `PUBLIC`, `anon` e `authenticated` às tabelas Agilis, inclusive grants por coluna, sequências e a função `subarvore`. O backend continua autorizado pelas regras da API. Essa proteção segue a separação de grants e RLS do Supabase.

`server/seguranca-supabase.sql` permite reaplicar essa política separadamente. Foi gerado de `server/seguranca-supabase.js` e deve permanecer sincronizado. O escopo é apenas os objetos conhecidos do Agilis em `public`; não altera outros aplicativos, grants futuros do schema nem papéis personalizados. Reveja acessos herdados e funções adicionais na implantação. Se o projeto não usa a Data API, o administrador também pode desabilitá-la.

TLS PostgreSQL valida certificados. `sslmode=require` na URL não desliga essa verificação. Se necessário, configure a CA oficial em `SGC_PG_CA_FILE`; não use `rejectUnauthorized=false`. `SGC_PG_SSL=disable` só é aceito para banco em loopback e é destinado a testes locais.

Cargas e demonstração

O exportador exige caminhos explícitos e gera arquivo novo, com permissão local `0600`:

```
node server/export-sqlite-to-pg.js /caminho/origem.db /caminho/carga.sql
```

Não executa SQL remoto. O SQL gerado e `server/seed-supabase.sql` exigem tabelas de domínio vazias, usam uma transação, ajustam identities antes do commit e não alteram permissões. Não apagam nem substituem dados existentes. Sessões e vínculos não são exportados. A aplicação deve estar parada durante a carga, e o esquema deve estar provisionado e migrado.

Para demonstração apenas local, use `SGC_AUTH_MODE=demo DATABASE_URL="" npm start`. Esse modo é recusado em produção/Vercel e escuta apenas `127.0.0.1`. Não coloque essa instância atrás de proxy público.

Validação e pendências

Roteiro de homologação

O passo a passo completo, incluindo criação do Administrador, contas por perfil e troca de senha, está em HOMOLOGACAO.md. A tabela abaixo é o resumo original do login.

Execute em ambiente de homologação, depois da configuração e da migração. Registre resultado e data, sem senhas, tokens ou chaves. Todos os itens continuam pendentes de execução no projeto Supabase real.

Verificação	Resultado esperado
`npm run check:auth` e opção `--online`	Configuração compatível e projeto responde
Conta vinculada, confirmada e ativa	Login conclui e mostra o perfil local correto
Conta Supabase sem vínculo	Acesso negado, sem criação automática de usuário
Senha errada e e-mail inexistente	Mesma mensagem genérica; bloqueio após excesso de tentativas
Diretor, Apoio e Chefe	Cada perfil acessa só as operações autorizadas; Chefe não vê ações internas fora da sua seção/subárvore
Tentativa de `x-user-id` sem sessão	API responde 401; lista demo responde 404
Logout e retorno à página anterior	Requisições protegidas exigem novo login
Usuário local desativado	Próxima requisição protegida é recusada
Sessão expirada	Aplicação avisa, solicita novo login e volta à tela em que o usuário estava; o rascunho da atualização semanal (guardado só na aba, em `sessionStorage`) é mantido e apagado ao sair
Acesso direto Supabase por `anon` / `authenticated`	Tabelas Agilis não ficam acessíveis fora da API Express

Testes locais cobrem provedor simulado, CSRF, origem, limite de tentativas, vínculo inexistente, sessão expirada/desativada, logout, regras de perfil, cache do cliente e TLS. PostgreSQL real verifica permissões e RLS com papéis `anon` / `authenticated` e sessões em conexões independentes.

Ainda é necessário configurar o projeto Supabase, criar e vincular as contas autorizadas e executar o login real. A tela de entrada não teve inspeção visual.

Perfis e o Administrador

Perfil	O que vê	O que faz
Diretor	Painel, ações (sem as internas das subseções), pedidos de prazo, reuniões e atas	Direciona ações, decide prazos, conduz reuniões, gerencia estrutura e cadastra contas de Chefe e Apoio
Apoio do Diretor	O mesmo do Diretor	Direciona ações e conduz reuniões; decide prazos só dentro de reunião em andamento
Chefe de seção	A própria seção e as seções subordinadas, e só elas	Atualiza o relato, gerencia as ações e o tempo da sua seção

Perfil	O que vê	O que faz
Administrador	Todos os dados, inclusive as ações internas das subseções	Só consulta o acompanhamento. Gerencia contas, perfis e estrutura (seções e subseções). Não direciona ações, não decide prazos, não conduz reunião, não comenta nem edita ações, combinados ou atas

O bloqueio de escrita do Administrador é aplicado no servidor (`app.js`): qualquer escrita fora de `/usuarios`, `/secoes` e `/auth` responde 403, além da permissão de cada rota.

Regras de contas:

- O Administrador cria contas de Diretor, Apoio e Chefe (com senha inicial). O Diretor continua criando Chefe e Apoio. Só o Administrador altera o perfil, o acesso e a senha de um Diretor.
- Nenhuma tela ou rota cria ou promove Administradores, e o perfil e o acesso do Administrador não mudam pela API: isso evita que o sistema fique sem administrador e que alguém se declare administrador.
- O sistema exige ao menos um Diretor ativo: o último não pode ser desativado nem trocado de perfil.

Primeiro Administrador

Depois de migrar o banco (versão 8), execute no servidor, com `SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY` no ambiente:

```
$env:SGC_ADMIN_SENHA_INICIAL = "senha provisória de 8 a 128 caracteres"
npm run admin:criar -- --sqlite data/sgc.db "Nome Completo" administrador@orgao.gov.br
# PostgreSQL: use --postgres e SGC_MIGRATION_DATABASE_URL explícita
```

O comando cria a conta no Supabase e o usuário local com perfil Administrador. A senha vai pelo ambiente, nunca na linha de comando, e é provisória: a pessoa cria a própria no primeiro acesso. Se a conta já existe no Supabase, use `SGC_ADMIN_SUBJECT=<UUID do usuário Supabase>` em vez da senha: o comando só vincula a conta, sem criar nem alterar senha. Pode-se criar mais de um Administrador.

Diretoria Adjunta: quem faz parte dela tem o perfil Administrador. Cadastre a seção na tela de Estrutura (tipo Diretoria Adjunta) e use `SGC_ADMIN_SECAO=<ID da seção>` ao criar o Administrador. Para um Administrador que já existe, vincule ou desvincule pelo servidor:

```
npm run admin:secao -- --sqlite data/sgc.db administrador@orgao.gov.br <ID da seção>
npm run admin:secao -- --sqlite data/sgc.db administrador@orgao.gov.br nenhuma
```

O vínculo não muda perfil nem acesso; nenhuma tela ou rota da API cria, promove ou vincula Administradores.

Senha inicial e troca obrigatória

Contas criadas pelo Diretor ou pelo Administrador, e contas cuja senha foi redefinida por eles (botão Login), nascem com `usuarios.trocar_senha = 1`. Enquanto isso, a API só responde `GET /bootstrap`, `POST /auth/trocar-senha` e `POST /auth/sair` (as demais rotas respondem 403 com `trocar_senha:`

`true`), e a interface mostra somente a tela "Crie a sua senha". A troca (`POST /api/auth/trocar-senha`) confirma a senha atual no Supabase, grava a nova pela API administrativa (é preciso `SUPABASE_SERVICE_ROLE_KEY` no servidor), libera a conta e encerra as outras sessões dela. O mesmo endpoint atende a troca voluntária pelo botão "Alterar senha" do menu; usuários não podem redefinir a própria senha pelo botão Login da gestão. A senha nova precisa ter de 8 a 128 caracteres e ser diferente da atual. Como a senha provisória fica conhecida por quem cadastrou, ela não deve ser reaproveitada.

Órgão descoberto pela conta (multi-empresa)

A tela de entrada pede só e-mail e senha (ver `MULTIEMPRESA.md`). O órgão (empresa) é descoberto pela própria conta: cada e-mail pertence a um só órgão, e, depois que o Supabase confirma e-mail e senha, o servidor acha a conta vinculada (`auth_contas`) e o órgão dela (`usuarios.empresa_id`). Não depende do domínio do e-mail, então vale para `gmail.com` e para vários órgãos no mesmo domínio. Órgão inativo ou conta inativa respondem o mesmo que senha errada ("E-mail, senha ou órgão incorretos."), para não revelar a que órgão um e-mail pertence. `POST /api/auth/entrar` aceita `empresa` (o identificador do órgão) só como restrição opcional, para endereços antigos com `?empresa=...`: se vier, a conta precisa ser desse órgão. O limite de tentativas conta por e-mail. Desativar o órgão (`empresas.ativa = 0`) encerra na hora as sessões das contas dele. `GET /api/auth/empresas` (público: `slug`, `nome`, `sigla` das ativas e listadas; `?slug=` devolve também uma empresa fora da lista) segue existindo para a demonstração. A conta de plataforma também entra só com e-mail e senha. `npm run empresa:criar -- --sqlite ARQUIVO|--postgres "Nome" slug "Nome do Diretor" email` cria a empresa e o primeiro Diretor (mesmas variáveis de ambiente de `admin:criar`; `SGC_EMPRESA_SIGLA` e `SGC_EMPRESA_LISTADA=0` opcionais). Os comandos `admin:criar` e `admin:secao` usam a Empresa 1; `SGC_EMPRESA=<ID>` escolhe outra.

Cadastro de chefes pelo Diretor

Em Estrutura → Usuários → Novo chefe com acesso, o Diretor informa nome, e-mail, senha inicial (8–128 caracteres) e seção ativa. Se ela já tiver chefe, o Diretor confirma explicitamente a substituição. O servidor cria a conta confirmada via Auth Admin API, cadastra o perfil fixo `chefe`, vincula o UUID e atribui a seção em transação. Não envia convite nem devolve ou armazena a senha no banco Agilis. A senha não tem troca obrigatória no primeiro acesso nesta versão.

Os membros da equipe de uma seção (perfil Membro da equipe) são cadastrados pelo próprio chefe, em Minha equipe: nome, e-mail e senha provisória, com a mesma troca obrigatória no primeiro acesso. O chefe também redefine a senha e desativa os membros da própria equipe; o Diretor e o Administrador fazem o mesmo, e mudam o perfil, em Configuração.

Configure `SUPABASE_SERVICE_ROLE_KEY` no servidor com a chave legada `service_role` do mesmo projeto Supabase. Na Vercel, cadastre-a em Production e faça redeploy. É uma chave administrativa exclusiva do backend; mantenha `SUPABASE_PUBLISHABLE_KEY` / `SUPABASE_ANON_KEY` para login. Sem a chave administrativa, os logins existentes continuam funcionando, mas a criação retorna indisponibilidade.

E-mail já existente não é apropriado nem tem a senha substituída. Na substituição, o chefe anterior perde a atribuição à seção, mas seu cadastro e histórico são preservados. A confirmação vale somente para o chefe exibido; alterações concorrentes exigem atualizar a página. Falha na transação local tenta excluir somente a nova conta criada. Se essa compensação falhar, ou uma chamada remota expirar após criar a conta, revise a conta no Supabase antes de repetir; sem vínculo ela não ganha acesso ao Agilis. A criação remota e o banco local não compartilham uma transação distribuída.

Referência: https://supabase.com/docs/reference/javascript/auth-admin-createuser

Gestão completa de seções e usuários

O Diretor gerencia os cadastros em **Estrutura**:

- **Seções:** criar, listar, editar nome/sigla/tipo/seção superior, atribuir chefia, ordenar, desativar/reativar e excluir. Alterações da hierarquia recusam ciclos e profundidade acima de três níveis; gravações compostas são transacionais.
- **Usuários:** criar Chefe ou Apoio com login, listar e-mail/perfil/seção, editar nome, perfil e atribuição, desativar/reativar e excluir. O Diretor pode editar o próprio nome e login, mas não excluir ou rebaixar seu perfil. Desativar ou mudar para Apoio libera a chefia anterior; reativar não restaura automaticamente essa atribuição.
- **Login:** alterar e-mail e/ou senha pelo botão Login para contas vinculadas. As alterações usam a API administrativa Supabase e revogam as sessões Agilis do usuário. A chave administrativa já exigida para criação é usada também aqui. Alterações remotas de credenciais e persistência local não compartilham transação: se ocorrer falha depois da resposta do Supabase, confira as credenciais no provedor antes de repetir.
- **Exclusão:** somente cadastros sem referências podem ser removidos. Usuários sem histórico podem ser excluídos mesmo com chefia atribuída: as seções são preservadas e ficam sem chefe. Usuários com histórico retornam conflito indicando as categorias e quantidades de registros; seções com subseções, usuários ou registros relacionados também retornam conflito. Nesses casos, use a desativação. Exclusão de usuário remove seus vínculos e sessões Agilis na mesma transação, mas preserva a conta no Supabase Auth, que pode servir a outros aplicativos. Essa conta fica sem acesso ao Agilis.

Contas vinculadas alteram e-mail pelo botão Login para atualizar também o provedor. O cadastro sem login permanece disponível para organizar os dados; não cria credenciais. As operações de escrita exigem Diretor, sessão válida, origem e CSRF. Não há migração de esquema nesta entrega.

Exclusão de ações pela gestão

Diretor e Apoio podem excluir ações visíveis criadas por Diretor ou Apoio, incluindo demandas originadas em diretrizes. O detalhe recebe `pode_excluir` da API e só mostra o botão quando autorizado. A API verifica a permissão novamente no DELETE. A exclusão remove comentários, tempos e pedidos de prazo da ação em uma transação; ações derivadas impedem remover a ação pai. As atas já gravadas e a diretriz de origem são preservadas. As regras anteriores de acesso do Chefe e de privacidade continuam válidas.

A migração 6 adiciona autoria estruturada (`acoes.criado_por`) e recupera autoria das diretrizes existentes. Ações antigas sem diretriz e sem autor comprovável permanecem sem atribuição: texto livre de comentários não concede permissão de exclusão. Novas ações registram o usuário autenticado; o cliente não escolhe o autor. Aplique a migração no destino explicitamente configurado junto da implantação desta versão.

Homologação com o Supabase real

Roteiro para provar, num projeto Supabase de homologação, o que os testes locais só simulam: login, contas criadas pelo Administrador, senha provisória com troca obrigatória e as permissões por perfil. Leva de 2 a 3 horas na primeira vez. Quem executa precisa ter acesso ao painel do Supabase e ao computador (ou servidor) onde o Agilis vai rodar.

Regras de segurança para toda a execução:

- Use um projeto Supabase **de homologação**, com contas de teste. Não use o projeto de produção nem dados reais.
- A chave `service_role` cria contas e altera senhas. Ela fica só no ambiente do servidor: não vai para o repositório, para o navegador, para mensagens ou para capturas de tela. Se vazar, gere outra no painel.
- Ao registrar evidências, anote data, resultado e o que foi visto. **Nunca** anote senhas, tokens ou chaves.

1. Preparar o projeto Supabase

0. Confirme que este é o projeto de **homologação**. Se ele já é o que usa dados reais, não execute este roteiro nele antes do ensaio numa cópia (etapa 3.1). A URL da API do projeto aparece em **Project Settings** → **API** (tem o formato `https://<identificador-do-projeto>.supabase.co`).
1. Em **Authentication** → **Providers**, mantenha só e-mail e senha.
2. Em **Authentication**, desligue o cadastro público (*Allow new users to sign up*). As contas são criadas pelo Agilis.
3. Em **Authentication** → **URL Configuration**, informe a origem do Agilis como Site URL: `http://localhost:3000` na fase local e a URL HTTPS da Vercel na fase seguinte.
4. Confira a política de senha do projeto. Ela vale para as senhas provisórias e para as trocas: se exigir mais que 8 caracteres ou complexidade, use senhas que a cumpram (veja o item de diagnóstico sobre "senha recusada").
5. Não ative MFA obrigatório nesta fase: o login do Agilis não pede segundo fator, e uma conta com MFA obrigatório não conclui o login.
6. Em **Project Settings** → **API**, separe: a URL do projeto, a chave pública (*publishable* ou *anon*) e a chave `service_role`.
7. Em **Project Settings** → **Database**, copie a string de conexão do PostgreSQL. Para o migrador e o comando do Administrador, prefira a conexão direta (porta 5432) ou o *Session pooler*; evite o *Transaction pooler* (porta 6543). A conexão direta usa IPv6 em vários planos: se o computador não conectar, use o *Session pooler*, que aceita IPv4.
8. A conexão exige TLS verificado. Se aparecer erro de certificado, baixe o certificado do projeto em **Project Settings** → **Database** → **SSL Configuration**, guarde-o na pasta `certs/` e aponte `SGC_PG_CA_FILE` para ele (o arquivo que já está no repositório pode não ser o do seu projeto).

2. Configurar o ambiente

Copie `.env.example` para `.env` (o arquivo é ignorado pelo git) e preencha:

Variável	Valor na fase local
`SGC_AUTH_MODE`	`supabase`
`SGC_PUBLIC_ORIGIN`	`http://localhost:3000` (só vale fora de produção; na Vercel, a URL HTTPS exata)
`SUPABASE_URL`	URL do projeto de homologação
`SUPABASE_PUBLISHABLE_KEY`	Chave pública (nunca a `service_role` aqui)
`SUPABASE_SERVICE_ROLE_KEY`	Chave `service_role`
`DATABASE_URL`	String de conexão do PostgreSQL de homologação
`SGC_MIGRATION_DATABASE_URL`	A mesma string, para o migrador e o comando do Administrador
`SGC_PG_CA_FILE`	`certs/supabase-prod-ca-2021.crt`, ou o certificado baixado do seu projeto (passo 1.8), se a conexão pedir a CA

Confira sem acessar o banco:

```
npm run check:auth
npm run check:auth -- --online
```

Esperado: nenhuma pendência; com `--online`, o projeto responde. O primeiro comando não mostra a chave.

3. Preparar o banco

3.1 Se o projeto já tem um banco do Agilis (é o caso mais comum)

Antes de qualquer alteração:

1. **Faça um backup e confirme como restaurá-lo.** Se o banco tem só dados de teste, o backup é opcional e dá para migrar direto no banco existente; se tem dados de piloto ou dados reais, ensaie tudo primeiro numa **cópia** (restaure o backup num projeto novo de homologação, ou use `pg_dump` e `psql`, conforme o seu plano do Supabase). As migrações 4 a 8 mudam permissões e a tabela de usuários.

2. **Rode o diagnóstico, que só lê.** No painel: **SQL Editor** → **New query**, cole o conteúdo de scripts/diagnostico-supabase.sql e execute. Ele não altera nada e não mostra nomes nem e-mails, só contagens e o estado da estrutura; o resultado pode ser compartilhado.

3. **Interprete o resultado:**

Linha do diagnóstico	Se for assim	O que fazer
Versões aplicadas	"tabela ausente"	Normal em banco criado só com o esquema base ou com a carga inicial: o migrador aplica as versões 1 a 10
Versões aplicadas	Termina antes da 8	O migrador aplica as que faltam
Reuniões em andamento	Mais de 1	A migração 2 **interrompe** sem apagar nada. Decida qual reunião fica e encerre as demais (veja "Se houver duplicidades antigas" em MIGRACOES.md)
Ações com mais de um pedido de prazo pendente	Mais de 0	Mesma situação: resolva os pedidos duplicados antes
Seções com chefe que não existe	Mais de 0	A migração 3 interrompe. Corrija a chefia dessas seções
Usuários sem e-mail, ou e-mails repetidos	Mais de 0	Cada login precisa de um e-mail único. Complete ou corrija antes de criar os acessos
Diretores ativos	0	O sistema exige ao menos um. Depois de criar o Administrador, cadastre o Diretor pela tela de contas
Tabelas com RLS ligada	Menos que o total	A migração liga a RLS em todas
Permissões de anon/authenticated	Diferente de "nenhuma"	As migrações **revogam** esse acesso. Confirme que nenhum outro sistema, planilha ou API pública do Supabase lê essas tabelas diretamente
Tabelas cujo dono não é o papel da conexão	Não é "nenhuma"	O backend precisa conectar como dono ou com `BYPASSRLS` (etapa 3.3)
Ações, atualizações, reuniões, tempo	Volume alto	Há histórico de verdade: ensaie na cópia e guarde o backup

4. **Aplique as migrações** (na cópia primeiro, depois no banco de homologação/produção, com a aplicação parada):

   ```
   npm run migrate -- --postgres
   ```

   Esperado: a lista de versões aplicadas termina na 8. Rodar de novo não aplica nada. Se a migração parar com um diagnóstico de duplicidades, nada foi alterado: corrija os dados apontados e repita.

5. **Rode o diagnóstico de novo.** Esperado: versões 1 a 10, restrição de perfil com `administrador`, RLS ligada em todas as tabelas, índices únicos presentes e permissões de `anon` / `authenticated` iguais a "nenhuma".

3.2 Se o banco estiver vazio (projeto novo)

Aplique o esquema base `server/schema.sql` pelo editor SQL do Supabase e depois rode `npm run migrate -- --postgres`.

3.3 Papel de conexão

O backend precisa conectar como dono das tabelas (ou papel com `BYPASSRLS`), porque as migrações 4 a 8 ligam a segurança por linha e revogam o acesso direto de `anon` e `authenticated`. Se a API responder erro de permissão, este é o ponto a revisar.

A migração 8 (perfil Administrador e `usuarios.trocar_senha`) já foi validada em PostgreSQL 18.4, num cluster local e descartável, partindo do esquema antigo e do novo. O diagnóstico também foi testado lá, em banco vazio, banco antigo com dados problemáticos e banco migrado; a única consulta não exercitada fora do Supabase é a de `auth.users`. Aqui a migração roda pela primeira vez no PostgreSQL do Supabase.

4. Criar o primeiro Administrador

```
$env:SGC_ADMIN_SENHA_INICIAL = "senha provisória de 8 a 128 caracteres"
npm run admin:criar -- --postgres "Nome do Administrador" administrador@exemplo.gov.br
Remove-Item Env:SGC_ADMIN_SENHA_INICIAL
```

Esperado: a mensagem "Administrador criado" e o aviso de que a senha precisa ser trocada no primeiro acesso. Ao conferir no painel do Supabase (Authentication → Users), a conta aparece confirmada.

Se a conta já existe no Supabase, use `$env:SGC_ADMIN_SUBJECT = "<UUID>"` no lugar da senha: o comando só vincula a conta.

Depois, suba o servidor:

```
npm start
```

5. Testes

Marque cada linha e anote a data. Use janelas anônimas ou navegadores distintos para perfis diferentes.

A. Administrador e primeiro acesso

#	Passo	Resultado esperado	OK
A1	Abrir `http://localhost:3000`	Tela de entrada com e-mail e senha	
A2	Entrar como Administrador com a senha provisória	Tela "Crie a sua senha para continuar", sem menu	
A3	Trocar o endereço para `#/painel`	A tela de troca continua; nada mais abre	
A4	Enviar senha atual errada	Mensagem "A senha atual não confere"	

#	Passo	Resultado esperado	OK
A5	Enviar nova senha curta, ou confirmação diferente	Mensagem clara, sem chamar o servidor	
A6	Trocar a senha corretamente	Entra no Painel da semana como Administrador	
A7	Sair e entrar com a senha provisória	Recusado ("E-mail, senha ou acesso inválidos")	
A8	Entrar com a senha nova	Entra direto, sem nova troca	
A9	Conferir o menu	Painel, Ações, Pedidos de prazo, Combinados, Reuniões e atas, Contas e estrutura; sem Direcionar ação nem Iniciar reunião	

B. Contas por perfil

#	Passo	Resultado esperado	OK
B1	Em Contas e estrutura, criar um **Diretor** com senha provisória	Conta criada com login; aparece como Diretor	
B2	Criar um **Apoio do Diretor**	Conta criada	
B3	Criar um **Chefe de seção** vinculado a uma seção com subseções	Conta criada; se a seção já tinha chefe, pede confirmação de substituição	
B4	Cada conta nova entra com a senha provisória	Tela obrigatória de nova senha; depois da troca, o menu do perfil	
B5	Entrar como Diretor e tentar criar outro Diretor	O perfil Diretor não é oferecido (a API também recusa)	
B6	Como Diretor, criar um Chefe e um Apoio	Funciona	
B7	Como Diretor, tentar alterar a conta do Administrador	Sem opções de alteração; a API recusa	
B8	Como Administrador, desativar o único Diretor ativo	Recusado, com a mensagem de que é preciso ao menos um Diretor ativo	
B9	Tentar criar ou promover outro Administrador pela tela	Não existe essa opção	

C. Permissões por perfil

#	Passo	Resultado esperado	OK
C1	Chefe: abrir Minhas ações	Vê ações da própria seção e das subseções, e de nenhuma outra	
C2	Chefe de outra seção: abrir o mesmo	Não vê as ações da primeira	
C3	Chefe: tentar abrir Painel da semana pelo endereço	Volta ao Início	
C4	Diretor: abrir Ações	Não vê as ações internas das subseções	
C5	Administrador: abrir a mesma lista	Vê também as internas	
C6	Administrador: abrir uma ação	Detalhe sem campo de comentário, sem prioridade, sem arquivar; só "Fechar"	
C7	Administrador: abrir Combinados, Pedidos de prazo, Reuniões e atas	Somente leitura, sem botões de alterar ou decidir	

#	Passo	Resultado esperado	OK
C8	Apoio: abrir Estrutura pelo endereço	Volta ao Painel	

D. Senhas

#	Passo	Resultado esperado	OK
D1	Qualquer usuário: menu → **Alterar senha**	Diálogo com senha atual, nova e confirmação	
D2	Trocar com sessão aberta em outro navegador	A outra sessão cai (401) e o aviso de sessão expirada aparece	
D3	Administrador: Contas e estrutura → usuário → **Alterar e-mail ou senha**, definir senha nova	Sessões da pessoa encerradas; no próximo acesso a troca é obrigatória de novo	
D4	Administrador: usar o mesmo botão na própria conta com senha	Recusado, com orientação de usar "Alterar senha"	
D5	Senha errada e e-mail inexistente	Mesma mensagem genérica; depois de muitas tentativas, bloqueio temporário	

E. Segurança

#	Passo	Resultado esperado	OK
E1	Criar uma conta só no painel do Supabase e tentar entrar	Acesso negado; nenhum usuário é criado sozinho	
E2	Desativar um usuário em Contas e estrutura	A próxima requisição dele é recusada	
E3	Enviar `x-user-id` com `curl.exe` sem sessão: `curl.exe -i -H "x-user-id: 1" http://localhost:3000/api/bootstrap`	401; e `/api/usuarios-demo` responde 404	
E4	Acessar o banco pela API pública do Supabase: `curl.exe -i "<SUPABASE_URL>/rest/v1/usuarios?select=*" -H "apikey: <chave pública>"`	Nenhuma linha é devolvida (erro de permissão ou lista vazia)	
E5	Deixar uma sessão aberta e inativa por mais de 1 hora	Próxima ação pede novo login e volta à mesma tela	

F. Implantação na Vercel (depois de tudo acima)

Siga VERCEL.md: configure as variáveis com a URL HTTPS exata em `SGC_PUBLIC_ORIGIN`, atualize o Site URL do Supabase, faça o deploy e repita A1 a A9 e E3. Confira os passos "Conferir o deployment" desse documento.

6. Se algo falhar

Sintoma	Causa provável
Servidor não inicia: "Configure SUPABASE_URL..."	Variáveis ausentes ou `SGC_PUBLIC_ORIGIN` sem HTTPS fora do local
"Migrações pendentes" ao iniciar	Falta rodar `npm run migrate -- --postgres`

Sintoma	Causa provável
"Cadastro indisponível: configure SUPABASE_SERVICE_ROLE_KEY"	Chave `service_role` ausente ou errada no servidor
"E-mail já cadastrado ou senha recusada pela política do projeto"	A mesma mensagem cobre e-mail repetido no Supabase e senha que não cumpre a política do projeto (tamanho, complexidade, senha vazada). Confira as duas causas
"Requisição de login inválida" ou 403 ao gravar	A origem do navegador difere de `SGC_PUBLIC_ORIGIN` (`localhost` ≠ `127.0.0.1`)
Login recusado com senha correta	Conta sem e-mail confirmado, sem vínculo local, usuário inativo ou MFA obrigatório
"Troca de senha indisponível..."	O servidor está sem a `service_role`
"A senha atual não confere" com a senha certa	O projeto Supabase pode ter alterado a senha por fora do Agilis; redefina pelo botão Login
API responde erro de permissão no banco	O papel de conexão não é dono das tabelas nem tem `BYPASSRLS` (etapa 3, item 4)

Não copie strings de conexão, cookies ou chaves para o relato do problema. Descreva o passo, a mensagem exibida e o horário; o log do servidor permite localizar o resto.

7. Ao terminar

- Desative ou apague as contas de teste no Agilis e no painel do Supabase.
- Guarde a tabela preenchida, sem segredos, junto ao projeto.
- Marque em `ESTABILIZACAO.md` o que passou e o que falhou; itens que falharam viram correção antes de qualquer dado real.
- Ainda ficam para depois deste roteiro: o ensaio na TV (ENSAIO_TV.md) e as respostas do Diretor às decisões de regra.

Migrações do banco

As migrações incrementais ficam em `server/migrations.js`. Cada versão aplicada é registrada em `schema_migrations`. Não altere versões já aplicadas: acrescente uma nova versão.

Comportamento de inicialização

- SQLite: cria o esquema base e aplica migrações antes de servir requisições. A carga de demonstração também aguarda as migrações.
- PostgreSQL: verifica as versões ao iniciar, sem executar DDL automaticamente. Se houver migrações pendentes, a inicialização falha com uma mensagem orientando a execução do migrador.
- PostgreSQL novo precisa ter o esquema base de `server/schema.sql` provisionado antes da migração. Não use o script de carga de demonstração como migração de um banco existente.

Banco mais novo que o código

A verificação ao iniciar (PostgreSQL) **não falha** quando o banco tem migrações mais novas do que a versão da aplicação conhece: registra um aviso e segue. Isso permite migrar o banco antes de publicar o código novo e voltar para a versão anterior (rollback) sem tirar o sistema do ar. Em troca, **toda migração deve ser aditiva** (coluna ou tabela nova, com valor padrão ou opcional), para a versão anterior continuar funcionando com ela. Mudança destrutiva (renomear ou apagar coluna, restringir dado existente) exige janela combinada e publicação coordenada. Migração pendente (o código conhece versões que o banco ainda não tem) continua impedindo a inicialização, e o migrador continua recusando um banco com versões que ele não conhece.

Execução explícita

SQLite, indicando o arquivo:

```
npm run migrate -- --sqlite /caminho/sgc.db
```

Esse comando ignora `DATABASE_URL`, mesmo que esteja definida no `.env`.

PostgreSQL, após configurar `SGC_MIGRATION_DATABASE_URL` no ambiente seguro da implantação:

```
npm run migrate -- --postgres
```

O migrador não utiliza `DATABASE_URL` como destino implícito. Não registre a URL com credenciais em código, logs ou histórico de comandos compartilhado.

Versões

ID	Nome	Alteração
1	`acoes_arquivadas`	Adiciona `acoes.arquivada` se estiver ausente, preservando as linhas existentes
2	`unicidade_pedidos_e_reunioes`	Cria índices únicos para um pedido pendente por ação e uma reunião em andamento
3	`integridade_referencial`	Acrescenta FK adiada de chefe no SQLite e remove quatro cascatas PostgreSQL, preservando os registros
4	`identidade_institucional`	Cria tabelas de identidade da integração OAuth anterior (Entra), hoje sem uso; em PostgreSQL/public habilita RLS e revoga permissões públicas aos objetos Agilis
5	`login_email_senha`	Cria `auth_contas` (vínculo com usuário Supabase), `auth_sessoes_senha` e `auth_tentativas`; reaplica RLS e revogações em PostgreSQL/public
6	`autoria_acoes`	Adiciona `acoes.criado_por` e preenche a autoria das ações vindas de diretrizes; reaplica RLS e revogações em PostgreSQL/public
8	`perfil_administrador`	Aceita o perfil `administrador` em `usuarios.perfil` e adiciona `usuarios.trocar_senha` (0 nos usuários existentes). No SQLite a tabela `usuarios` é reconstruída, como na versão 3, preservando dados, índices, views e triggers; no PostgreSQL a restrição de perfil é recriada. Reaplica RLS e revogações em PostgreSQL/public
7	`sessao_deslizante`	Adiciona `auth_sessoes_senha.criada_em` (0 nas sessões já abertas, que expiram no prazo original); reaplica RLS e revogações em PostgreSQL/public
9	`impedimentos_da_acao`	Cria `impedimentos` (ligados a `acoes`, com `critico`, `apoio`, autoria e ciclo aberto → resolvido) e seus índices, inclusive o parcial dos abertos; em PostgreSQL/public reaplica RLS e revogações. Não altera nem apaga dados existentes; os impedimentos em texto dos relatos já enviados continuam nesses relatos
10	`checklist_da_acao`	Cria `acao_checklist` (ligada a `acoes`, com texto, `concluido`, `ordem` e autoria) e seu índice; em PostgreSQL/public reaplica RLS e revogações. Não altera nem apaga dados existentes
11	`secao_diretoria_adjunta`	Aceita o tipo `diretoria_adjunta` em `secoes.tipo`. No SQLite a tabela `secoes` é reconstruída, como nas versões 3 e 8, preservando dados, índices, views e triggers; no PostgreSQL a restrição de tipo é recriada. Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes
12	`secao_na_reuniao`	Adiciona `secoes.na_reuniao` (1 em todas as seções existentes: continuam aparecendo na reunião); reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes

ID	Nome	Alteração
13	`prazos_e_acoes_apresentadas`	Adiciona `acoes.apresentada_em` e cria `acao_prazos` (histórico de prazos). Copia para o histórico os pedidos de prazo já aprovados e marca como apresentadas as ações visíveis ao Diretor, das seções que aparecem na reunião, criadas antes do fim de cada reunião já realizada (guarda a data da primeira). Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga outros dados
14	`acoes_ultima_movimentacao`	Adiciona `acoes.movimentada_em` e a preenche com a mais recente das movimentações já registradas (criação, conclusão, tempo, comentário, checklist, impedimento, histórico e pedidos de prazo); mudanças de status antigas não tinham data. Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga outros dados
15	`quadro_de_avisos`	Cria `avisos` (texto, `data_inicio`, `data_fim`, `ativo` e autoria) para o aviso do Diretor/Administrador na tela Início; reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes
16	`registros_e_revisao_de_reuniao`	Adiciona `acoes.demanda_diretor` (origem declarada pela seção, sem alterar autoria). Cria informações da reunião, presença individual de titulares/suplentes, sugestões de revisão com resposta e versões publicadas da ata. Reaplica RLS e revogações em PostgreSQL/public. Preserva os registros existentes; reuniões antigas não recebem presença presumida
17	`ordem_participantes_reuniao`	Adiciona `reuniao_participantes.ordem` para preservar a sequência de seleção. Presenças existentes recebem a ordem alfabética anteriormente exibida, sem presumir a ordem dos cliques. Não cria tabelas nem altera permissões
18	`acoes_recebidas_e_vistas`	Adiciona `acoes.recebida_em` (vazio nas ações existentes: nenhuma aparece como nova) e cria `acao_vistas` (ciência de cada pessoa que recebeu a ação, chave `acao_id` + `usuario_id`). Reaplica RLS e revogações em PostgreSQL/public, agora incluindo `acao_vistas`. Não altera nem apaga dados existentes
19	`perfil_consulta`	Aceita o perfil `consulta` em `usuarios.perfil` e adiciona `usuarios.ver_outros_centros` (0 em todos os usuários existentes). No SQLite a tabela `usuarios` é reconstruída, como nas versões 3 e 8, preservando dados, índices, views e triggers; no PostgreSQL a restrição de perfil é recriada. Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes
20	`tarefas_da_acao`	Adiciona a `acao_checklist` as colunas `data` (data de execução, `AAAA-MM-DD`) e `responsavel_id` (referência a `usuarios`), ambas opcionais; os itens existentes ficam sem data e sem responsável. Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes
21	`membros_e_validacao_de_acoes`	Aceita o perfil `membro` em `usuarios.perfil` (no SQLite a tabela `usuarios` é reconstruída, como nas versões 3, 8 e 19, preservando dados, índices, views e triggers; no PostgreSQL a restrição de perfil é recriada) e acrescenta a `acoes` as colunas opcionais `responsavel_id`, `atribuido_por`, `atribuida_em` (quem executa a ação e quem a atribuiu), `validada_chefe_em`, `validada_chefe_por` (validação do chefe) e `demandante_externo` (entidade fora da subordinação ao Diretor que demandou a ação), com índice em `responsavel_id`. Reaplica RLS e revogações em PostgreSQL/public. Não altera nem apaga dados existentes; as ações já concluídas passam a seguir a regra nova de validação sem mudar de estado

ID	Nome	Alteração
22	`empresas`	Multi-empresa, fase de expansão (ver `MULTIEMPRESA.md`). Cria `empresas` e registra a Empresa 1, o órgão atual, com `listada_no_login` e `slug`. Acrescenta `empresa_id` (obrigatório, padrão 1, referência a `empresas`) a `secoes`, `usuarios`, `diretrizes`, `acoes`, `combinados`, `avisos`, `reunioes`, `atualizacoes` e `config`, com índice; os registros existentes recebem 1 pelo próprio padrão, sem reescrita. Acrescenta `uq_config_empresa_chave` e `uq_reuniao_em_andamento_empresa` ao lado das chaves únicas globais, que continuam até a fase de contração. Reaplica RLS e revogações em PostgreSQL/public (inclui `empresas`). Não altera nem apaga dados existentes
23	`contracao_multiempresa`	Multi-empresa, fase de contração (ver `MULTIEMPRESA.md`). Troca as chaves únicas globais pelas por empresa: `config` passa a ter chave primária `(empresa_id, chave)` (reconstruída no SQLite; `alter` no PostgreSQL, que também remove o índice redundante `uq_config_empresa_chave`) e cai `uq_reuniao_em_andamento` (continua valendo uma reunião em andamento por empresa, `uq_reuniao_em_andamento_empresa`). Idempotente; não altera nem apaga dados. Só depois dela uma 2ª empresa pode existir. O padrão `empresa_id = 1` continua (sai num endurecimento à parte). **Código anterior à fase 2 não funciona com este esquema** (o upsert por `chave` deixa de existir): publique o código da fase 2 ou posterior ANTES de aplicar esta versão, e use como ponto de retorno só deploys dessa fase em diante
24	`perfil_plataforma`	Acrescenta o perfil `plataforma` à restrição de `usuarios.perfil` (reconstrução da tabela no SQLite, como nas versões 8, 19 e 21; `alter` da restrição no PostgreSQL) e reaplica RLS e revogações. É o perfil que cria e administra as empresas pela tela "Empresas", sem acesso a dados de negócio. Não altera nem apaga dados. A versão anterior do código só recusa o perfil novo (ninguém o tem até alguém criar uma conta de plataforma)
25	`produtividade`	Dados para os indicadores de produtividade das unidades. Acrescenta a `acoes` três colunas opcionais: `esforco` (estimativa 'pequena', 'media' ou 'grande', com restrição de valores), `devolucoes` (inteiro, padrão 0: vezes que a ação foi devolvida na validação) e `iniciada_em` (primeira vez em andamento). As ações existentes ficam com esforço e início vazios e nenhuma devolução. Não altera nem apaga dados; a versão anterior do código ignora as colunas

A unicidade das versões semanais já faz parte do esquema base (`secao_id`, `semana`, `versao`). Os destinos e as regras de exclusão/atualização de todas as FKs do esquema migrado são comparados automaticamente entre SQLite e PostgreSQL.

Integridade referencial na versão 3

`secoes.chefe_id` passa a exigir um usuário existente também no SQLite, com verificação adiada até o commit, como no PostgreSQL. Isso permite cadastrar seção e usuário na mesma transação. Se uma seção antiga apontar para um chefe inexistente, a migração informa os IDs das seções e interrompe a execução sem corrigir ou excluir registros automaticamente.

PostgreSQL passa a usar `ON DELETE NO ACTION`, como SQLite, nos vínculos de comentários, tempo e pedidos com ações, e de decisões com reuniões. Uma exclusão direta do registro pai é recusada enquanto existirem dependentes. A API continua removendo explicitamente os dependentes na mesma transação quando a exclusão de uma ação própria é autorizada. Scripts externos que dependiam das cascatas precisam fazer essa remoção explícita; não reaplique o esquema base para atualizar bancos existentes.

SQLite reconstrói `secoes`, preservando colunas, registros, índices, views e triggers. O migrador usa a fila da conexão e `BEGIN IMMEDIATE`; desativa FKs antes da transação, executa `foreign_key_check` antes do commit e reativa a fiscalização ao terminar, inclusive após rollback. Consultas externas na mesma conexão aguardam todo esse procedimento. A implantação deve pausar outras instâncias durante a migração e ter espaço para a cópia temporária da tabela. Procedimento baseado na documentação de reconstrução de tabelas SQLite.

No PostgreSQL, as quatro constraints são substituídas dentro da transação de migração. Definições inesperadas interrompem a operação para revisão, e as substituições anteriores são desfeitas. A distinção entre cascata e exclusão explícita segue as regras de chaves estrangeiras PostgreSQL.

Esta versão alinha os vínculos existentes. Não acrescenta novas regras de domínio para status/prioridade, unicidade de chefia ou ciclos hierárquicos, nem novos vínculos para `diretrizes.reuniao_id` e `pedidos_prazo.reuniao_id`, que continuam sem FK nos dois esquemas.

Atualização de ambiente existente

Para as versões 4 e 5, configure o projeto Supabase Auth e prepare os vínculos antes de reabrir a aplicação. A migração PostgreSQL altera permissões: acessos diretos de `anon` e `authenticated` deixam de funcionar. O backend deve usar papel proprietário ou administrado com `BYPASSRLS`; veja AUTENTICACAO.md. Dados existentes não são vinculados por e-mail automaticamente. Nenhuma sessão nem identidade fictícia é criada pela migração.

1. Faça backup e confirme o procedimento de restauração.
2. Teste a migração em uma cópia isolada dos dados.
3. Pause as gravações da aplicação antiga durante a atualização do esquema.
4. Execute o comando com o destino explícito e confira o resultado.
5. Inicie a nova versão e valide abertura/encerramento da reunião, pedidos de prazo e atualizações.

Todas as migrações pendentes de uma execução ficam na mesma transação. Falhas desfazem tanto o DDL executado nessa transação quanto os registros de versão. No PostgreSQL, uma trava transacional coordena execuções simultâneas do migrador. Em SQLite, a fila coordena a conexão local; a implantação deve ter um único processo migrador.

Se houver duplicidades antigas, o comando informa os IDs envolvidos e interrompe a migração. Não exclui pedidos, não encerra reuniões e não escolhe automaticamente um registro vencedor. Consultas para diagnóstico:

```
select acao_id, count(*)
from pedidos_prazo
where status = 'pendente'
group by acao_id
having count(*) > 1;

select id, data, semana
from reunioes
where status = 'em_andamento'
order by id;
```

A resolução desses registros exige revisão do histórico e uma decisão registrada. Após corrigir a inconsistência, execute novamente o migrador. Reexecuções após sucesso não reaplicam versões.

Não há comando automático de downgrade. Para reverter uma implantação, avalie a compatibilidade da versão anterior com os índices novos; se for necessário restaurar o banco, use o backup validado. Não remova linhas de `schema_migrations` para forçar uma reaplicação.

Validação desta entrega

Foram testados SQLite em memória e arquivos temporários: idempotência, esquema antigo, preservação dos registros, bloqueio de duplicidades, rollback de DDL e destino explícito no CLI. A integração em PostgreSQL 17.9 local também validou migradores simultâneos com pools independentes, rollback de DDL e do registro de versões, bloqueio de dados duplicados e índices únicos em escritas diretas. Consulte TESTES_POSTGRESQL.md para repetir a validação. O ensaio com cópia dos dados e permissões do ambiente de implantação continua necessário.

Referências: travas transacionais PostgreSQL e índices parciais.

Integração PostgreSQL local

```
PG_BIN=/caminho/para/postgresql/bin npm run test:postgres
```

Se `initdb` e `pg_ctl` já estiverem no `PATH`, basta `npm run test:postgres`. Os binários precisam pertencer à mesma instalação, com suas bibliotecas e arquivos de suporte. Execute como usuário comum, não como root. A suíte usa sockets Unix e exige macOS ou Linux, além das dependências instaladas do projeto.

A execução validada utilizou PostgreSQL 17.9, Node 24.11.1 e macOS. Os binários foram obtidos separadamente com `@embedded-postgres/darwin-arm64@17.9.0-beta.16`, do projeto embedded-postgres; não foram adicionados às dependências da aplicação. Esse pacote requer seu script `postinstall` para preparar os links das bibliotecas. Também é possível usar uma instalação local de PostgreSQL.

Isolamento e limpeza

- Cada execução cria um cluster em `/tmp/sgc-pg-*`, com banco novo por cenário e esquema de `server/schema.sql`.
- A conexão é construída explicitamente para o socket do cluster. `DATABASE_URL`, `SGC_DB` e `SGC_MIGRATION_DATABASE_URL` não selecionam o destino. O módulo de banco ainda carrega `.env`, mas essas configurações não são usadas nesta suíte.
- PostgreSQL não escuta TCP. O diretório temporário e o socket restringem o acesso ao usuário local. As duas APIs de teste escutam apenas `127.0.0.1`, em portas dinâmicas.
- Os dados são fictícios, o relógio é fixado e cada API possui seu próprio pool. São duas instâncias Express no mesmo processo Node, com conexões PostgreSQL independentes.
- Ao terminar, inclusive em falhas de asserção, a suíte fecha APIs e pools, encerra PostgreSQL e remove o cluster. Se houver interrupção forçada do processo ou falha ao parar o servidor, o diretório pode permanecer; confirme que o servidor foi encerrado antes de removê-lo.
- A ausência dos binários causa falha explícita. A suíte não substitui PostgreSQL por mocks nem marca esses cenários como ignorados.

Cenários verificados

1. Migradores simultâneos aplicam cada versão uma vez; a inicialização recusa migrações pendentes.
2. Uma falha SQL após DDL desfaz coluna, índice e registro de versões; nova tentativa funciona.
3. Leituras externas não veem alterações sem commit, e rollback preserva a gravação de outra conexão.

4. Oito solicitações simultâneas criam um pedido; decisões concorrentes têm um vencedor e prazo coerente.
5. Oito atualizações semanais preservam conteúdos e recebem versões distintas.
6. Abertura, encerramento e reabertura concorrentes respeitam a unicidade de reunião ativa.
7. Um trigger provoca falha real na atualização do prazo; a decisão inteira é revertida e a conexão continua utilizável.
8. IDs automáticos de seções e usuários continuam válidos após a carga com IDs explícitos.
9. Índices únicos rejeitam pedidos e reuniões duplicados também em SQL direto.
10. Reuniões legadas duplicadas interrompem migrações sem excluir dados.
11. Ações e decisões concorrentes ao encerramento entram na ata ou são recusadas; revisão concorrente ao envio não altera o conteúdo publicado.
12. Chefes inexistentes e exclusões diretas de registros com histórico são rejeitados; referências adiadas permitem cadastrar seção e chefe juntos.
13. Os catálogos SQLite/PostgreSQL têm as mesmas FKs, destinos e ações de exclusão/atualização.
14. A migração de cascatas antigas preserva os registros e desfaz alterações parciais se uma constraint inesperada for encontrada.
15. A API continua excluindo ações próprias e seus dependentes explicitamente após a migração.
16. RLS e revogação de grants bloqueiam os papéis públicos, inclusive grants por coluna, sequências, função e policies legadas permissivas.
17. Vínculos e sessões institucionais persistem entre conexões e respeitam unicidade.
18. Exportação transacional preserva conteúdo, ordena referências de pais, recusa destino ocupado e desfaz carga após erro.
19. O SQL de demonstração legado também recusa destino ocupado e não modifica permissões.

O runner reporta **21 testes**, contando dois agrupadores e seus **19 cenários**. `npm test` executa separadamente 77 testes e não exige binários PostgreSQL.

Limites

A suíte usa o adaptador `createPgDb` com pools locais e sem TLS. Verifica RLS/grants localmente, mas não certifica certificados, pooler, permissões efetivas do projeto Supabase, login no tenant Entra, múltiplos processos Node ou carga prolongada. A configuração de TLS é verificada separadamente por testes unitários. A comparação de catálogos cobre as FKs existentes; novas constraints de domínio não fazem parte desta entrega. Nenhum banco existente precisa ser migrado para executar estes testes.

Implantação do Agilis na Vercel

O `vercel.json` publica `public/**` como arquivos estáticos e empacota `server/index.js` como função Node. `/` entrega o HTML, `/api` e `/api/*` usam Express e os demais caminhos procuram arquivos em `public`. A navegação do app usa fragmentos (`#`). Arquivos inexistentes devem retornar 404. A página inicial não precisa abrir o banco para ser servida.

A publicação explícita de estáticos é necessária porque, com `builds`, a Vercel inclui somente as saídas dos builders. Referência: configuração oficial.

Como os estáticos vão direto pelo `@vercel/static`, sem passar pelo Express, cabeçalhos de cache para eles (`Cache-Control` em `server/app.js`) não valem na Vercel — só localmente (`npm start`). Na Vercel, use o `headers` do próprio `vercel.json`.

Configurar o projeto

Use a raiz deste repositório como Root Directory. Mantenha os builders do `vercel.json`, sem substituir o Output Directory por `public` (isso removeria a API). Selecione Node.js 22.x ou 24.x. Configure as variáveis no ambiente correspondente (Production ou Preview) e faça novo deployment após alterá-las.

Variável	Configuração
`SGC_AUTH_MODE`	`supabase`
`SGC_PUBLIC_ORIGIN`	URL HTTPS exata usada no navegador, sem caminho, por exemplo `https://seu-projeto.vercel.app`
`SUPABASE_URL`	URL do projeto Supabase
`SUPABASE_PUBLISHABLE_KEY` ou `SUPABASE_ANON_KEY`	Chave pública do mesmo projeto
`DATABASE_URL`	Conexão PostgreSQL fornecida pelo painel Supabase, acessível pela função
`SGC_PG_CA_FILE`	`certs/supabase-prod-ca-2021.crt` se esse for o certificado CA indicado para o banco

Na Vercel o servidor confia no primeiro proxy (`trust proxy`), de modo que o limite de tentativas de login use o IP real de cada usuário; fora dela, use `SGC_TRUST_PROXY` (número de proxies) somente atrás de um proxy confiável. A interface envia CSP restritiva e demais cabeçalhos de segurança, e as fontes ficam em `public/fontes` (sem Google Fonts).

A CA já é incluída no pacote da função. Não desabilite a verificação TLS. SQLite é recusado na Vercel, inclusive se `DATABASE_URL` estiver ausente. Não configure `SGC_AUTH_MODE=demo`.

Use uma origem estável para homologação. Acessar um alias diferente de `SGC_PUBLIC_ORIGIN` impede login e gravações pela proteção de origem. Configure Preview separadamente, preferencialmente com banco e usuários de homologação.

Preparar o banco e as contas

Siga AUTENTICACAO.md para provisionar o esquema, aplicar as migrações e vincular os usuários. Execute o migrador fora da função, com `SGC_MIGRATION_DATABASE_URL` apontando explicitamente ao destino pretendido. A inicialização PostgreSQL apenas verifica as migrações; não migra nem semeia dados automaticamente.

O usuário do Supabase Auth precisa estar confirmado e vinculado a um usuário local ativo. Criar somente uma conta no Supabase não concede acesso ao Agilis.

Conferir o deployment

1. Abra `/`, `/estilo.css` e `/js/main.js`: devem retornar 200, com HTML, CSS e JavaScript respectivamente.
2. Abra `/api/auth/config`: deve retornar JSON com `modo: supabase`. Um 500 nessa etapa exige consultar o log da função (variáveis, conexão, CA ou migrações).
3. Faça login com uma conta confirmada e vinculada. Confirme o perfil, uma operação autorizada e o logout.
4. Após sair, confirme que uma API protegida retorna 401 e que um arquivo inexistente retorna 404.

Um deployment com build aprovado ainda pode falhar na inicialização da API. Os testes locais não comprovam configuração, conectividade ou login no projeto remoto. Não copie senhas, strings de conexão ou cookies para logs compartilhados.

Multi-empresa

O Agilis nasceu para um órgão só. Este documento registra o plano para atender várias empresas (ou órgãos) no mesmo sistema **sem perder os dados do órgão atual**, que passa a ser a Empresa 1.

Decisões

1. **Quem cria empresas:** só o administrador da plataforma. Não há autocadastro.
2. **Cada pessoa pertence a uma só empresa.** Um e-mail, um login do Supabase Auth, um usuário. Vários vínculos por pessoa ficam para depois (exigiria ligar vários usuários ao mesmo login em `auth_contas`).
3. **O método é o mesmo para todas:** Diretor, chefes, membros, reunião semanal, semáforo. Variam o nome, o logo, o cabeçalho e o brasão da ata, e o dia e a hora da reunião. Vocabulário configurável fica para depois, se as empresas novas pedirem.
4. **O órgão é descoberto pela conta, na entrada** (decisão do proprietário, revista em 05/11/2026: antes a pessoa escolhia numa caixa de seleção). Endereço único; a pessoa digita só e-mail e senha.
5. **Uma base compartilhada**, com `empresa_id` nas tabelas, e não uma base por empresa.
6. **Administrador** continua total *dentro* da empresa. A plataforma ganha um perfil próprio, que cria e gerencia empresas mas não lê os dados de negócio delas.

Como a entrada acha o órgão

- A tela de entrada pede só e-mail e senha. Depois que o Supabase confirma a identidade, o servidor acha a conta vinculada (`auth_contas`) e o órgão dela. **Não usa o domínio do e-mail**: cada e-mail já pertence a um só órgão, então funciona também com `gmail.com` e com vários órgãos no mesmo domínio.
- Conta inativa, órgão inativo ou e-mail desconhecido respondem como senha errada ("E-mail, senha ou órgão incorretos."), sem revelar a que órgão um e-mail pertence. O limite de tentativas conta por e-mail.
- `empresa` (o identificador) continua aceito no login só como restrição opcional, para endereços antigos com `?empresa=slug`. `GET /api/auth/empresas` segue público (ativas e listadas; `?slug=` devolve também uma empresa fora da lista) e é usado pela demonstração.
- Futuro "uma pessoa em várias empresas": exigirá ligar vários usuários ao mesmo login e, aí sim, voltar a perguntar o órgão quando houver mais de um.

Fases (cada uma publicada separada)

Fase	O que é	Estado
0	Backup, ensaio da migração numa cópia e contagem de registros antes e depois (`scripts/contagem-registros.js`)	script pronto; falta rodar em produção

Fase	O que é	Estado
1	**Banco, só acrescenta** (migração 22): tabela `empresas`, Empresa 1, `empresa_id` nas tabelas raiz, chaves únicas por empresa ao lado das globais	publicada
2	**Servidor:** todo SQL filtra pela empresa de quem está logado; login com escolha da empresa; **testes de isolamento** (duas empresas, toda rota provada contra acesso cruzado)	pronta e testada localmente (sem migração nova); ainda não publicada
3	Plataforma: `npm run empresa:criar` (empresa + primeiro Diretor) e depois a tela "Empresas"	comando e tela "Empresas" prontos e testados; a tela exige o perfil `plataforma` (migração 24, ainda não aplicada em produção) e uma conta criada por `npm run plataforma:criar`
4	Identidade: seletor na entrada, nome na barra e no título da aba, ata por empresa (cabeçalho e os dois brasões)	pronta (sem migração); logo da empresa na barra fica para depois
5	**Contração:** troca as chaves únicas globais pelas por empresa (migração 23). **Só depois disso pode existir a 2ª empresa.** Remover o padrão 1 de `empresa_id` fica como endurecimento à parte (exige que todo INSERT, o seed e os testes informem a empresa)	migração 23 pronta e testada (SQLite e PostgreSQL); falta aplicar em produção

Regras para não perder dados nem derrubar o sistema

- Migração e publicação seguem **expandir → publicar → contrair**: cada passo é aditivo e a versão anterior do código continua funcionando com o banco já migrado (o servidor tolera banco mais novo, ver `MIGRACOES.md`).
- **Tabelas raiz** (levam `empresa_id`): `secoes`, `usuarios`, `diretrizes`, `acoes`, `combinados`, `avisos`, `reunioes`, `atualizacoes`, `config`. As **filhas** (comentários, tempo, tarefas, prazos, pedidos, impedimentos, ciência, decisões, informações, presenças, sugestões e versões da ata) herdam a empresa da ação ou da reunião.
- O padrão `empresa_id = 1` ainda existe: o código atual sempre informa a empresa, mas ele protege inserts avulsos (seed, testes). Removê-lo é um endurecimento à parte, depois da Fase 5. Até lá, a defesa contra um insert esquecido é a rede de segurança (`SGC_LINT_EMPRESA`) e os testes de isolamento.
- As duas chaves únicas globais (`config.chave` e "uma reunião em andamento", `uq_reuniao_em_andamento`) caem na migração 23, que só deve ser aplicada depois de publicado o código da fase 2 (ele já usa `on conflict(empresa_id, chave)` e a unicidade por empresa).
- Antes de cada migração em produção: backup, ensaio numa cópia (um ramo do Supabase) e `node scripts/contagem-registros.js --postgres > antes.txt`; depois, `depois.txt` e `diff`. Nunca na terça da reunião.

Como a Fase 2 isola as empresas

- **Contexto da empresa** (`server/empresa.js`): depois de autenticar, `escopoDaEmpresa` fixa o `empresa_id` da conta para a requisição (contexto assíncrono). O número vem sempre da conta no banco, nunca do cliente. `empresaAtual()` falha alto se não houver empresa em curso.

- **Todo SQL de tabela raiz filtra por `empresa_id`** (`from`, `update`, `delete from`, `insert into`). Os escopos de ação (`visiveisAcoes` / `visiveisLeitura`) começam sempre por `a.empresa_id = ?`. Tabelas filhas (tempo, tarefas, impedimentos, pedidos de prazo, decisões…) herdam a empresa da ação ou da reunião, que é sempre carregada já filtrada; consultas que partem de uma filha (ex.: lista de pedidos de prazo) filtram por `a.empresa_id` na junção.
- **Referências cruzadas recusadas ao gravar**: seção, pai, chefe, responsável, reunião e participantes precisam ser da mesma empresa.
- **Rede de segurança** (`SGC_LINT_EMPRESA=1`, ligada nos testes de isolamento): dentro de uma requisição autenticada, SQL que leia ou grave tabela raiz sem `empresa_id` falha na hora. A consulta que precisa olhar todas as empresas (e-mail único no sistema inteiro) leva o marcador `/*global*/`, e fica à vista na revisão. Rode `SGC_LINT_EMPRESA=1 npm test` ao mexer em SQL.
- **Testes** (`test/isolamento-empresas.test.js`, `test/login-empresas.test.js` e o teste PostgreSQL "multi-empresa… duas empresas"): duas empresas no mesmo banco, com o esquema da contração simulado; provam que nenhuma leitura traz dado da outra, que conhecer o número de um registro não dá acesso, que gravações cruzadas são recusadas sem alterar nada, e que configuração, avisos, combinados, reunião em andamento e direcionamentos são independentes.
- Rotas de ação, estrutura, semana, reuniões e avisos, o cabeçalho da ata, os cadastros e os comandos do servidor foram todos escopados. Novas rotas e consultas devem seguir a mesma regra, com teste de isolamento.
- A migração 13 (histórico de prazos) ganhou a própria consulta de árvore: migração não pode depender de código que passou a exigir `empresa_id`, coluna que só existe a partir da versão 22.
- O principal risco é vazamento entre empresas: por isso os testes de isolamento vêm junto da Fase 2, e uma proteção extra no banco (RLS por empresa) fica como evolução.

Produtividade

Os indicadores (ver `CLAUDE.md`) são por empresa como todo o resto: cada consulta já sai filtrada pela empresa de quem pergunta, e `/produtividade` está no teste de isolamento.

Organograma por empresa

Cada empresa tem a sua própria árvore de seções (já isolada). O que variava era o molde: os nomes (Centro, Coordenação…), a profundidade e quais unidades viram cartão na reunião. Exemplos reais: **Deplan** = Diretoria (o perfil Diretor) → Diretoria Adjunta → Centros → Coordenações, podendo haver Coordenação direto sob a Adjunta; **SGTO** = Superintendência (o perfil do topo) → Diretorias → Coordenações. Plano em etapas, cada uma preservando o comportamento atual como padrão:

1. **Vocabulário por empresa** (pronta, sem migração): `config.vocabulario` (JSON) guarda, por empresa, singular, plural e gênero de cada tipo de unidade e o nome do perfil do topo. Módulo puro compartilhado `public/js/vocabulario.js` (servidor e tela), `public/js/vocab.js` (atalhos da tela), `PUT /api/config/vocabulario` (Diretor e Administrador), vindo no bootstrap (`vocabulario`).

O gênero acerta os artigos ("Todos os Centros" / "Todas as Diretorias"). Edição em Configuração → "Nomes das unidades". **Tipos opcionais**: a empresa pode não usar a Coordenação de primeiro nível nem a Diretoria Adjunta (`usa: false`, p. ex. a SGTO, que não tem unidade sem chefe própria): o tipo some dos formulários e dos textos, o servidor recusa criar ou trocar unidade para ele (400) e recusa desligar o tipo enquanto houver unidade ativa dele (409). Centro e Subseção sempre existem. Aplicado nos menus, títulos, "Direcionar ação", Configuração, ata e mensagens da API. **Ainda com o texto fixo**: as demais ocorrências de "Diretor" (por exemplo "Demandada pelo Diretor", "Peça ao Diretor") e algumas legendas; a lista vai sendo trocada à medida que as telas são revisadas.

2. **Posição livre da árvore** (pronta, sem migração): Centro e Coordenação ficam no primeiro nível ou diretamente sob a Diretoria Adjunta; esta fica só no primeiro nível, no máximo uma, e pode ter unidades abaixo; a Subseção fica sob qualquer seção. Os níveis da árvore são configuração da empresa (`vocabulario.niveis`, 2 a 6, padrão 3).
3. **Unidade de acompanhamento** (pronta, **sem migração**: a regra sai do tipo, não de uma marcação): são cartões a Diretoria Adjunta, o Centro e a Coordenação, onde estiverem na árvore (decisão do proprietário para o Deplan: a Adjunta e a Coordenação sob ela também são cartões); a Subseção entra na soma do cartão acima, e Centro ou Coordenação abaixo da Adjunta não somam nela. "Todos os Centros" vale para Centros e Coordenações, sem a Adjunta. Painel, reunião, ata, relato, "Minha equipe" e a visibilidade descem só por Subseções. A SGTO não usa Adjunta nem Coordenação de primeiro nível (tipos opcionais desligados).

Informações, presença e revisão da ata

Durante a reunião, Diretor, Apoio ou Administrador podem usar **Registrar informação** para incluir comunicados e contexto sem criar ação ou decisão. Esses registros podem ser editados ou excluídos durante a reunião e entram na ata ao encerrá-la.

Use **Presença** para registrar individualmente quem participou, incluindo titular e suplente quando ambos estiverem presentes. A ata guarda os nomes e as seções como estavam no registro de presença. Uma reunião antiga não recebe participantes automaticamente: ter enviado um relato ou ser chefe de uma seção não comprova presença.

Para reuniões passadas, Diretor, Apoio e Administrador podem abrir **Reuniões e atas**, selecionar a reunião e usar **Registrar presença** e **Registrar informação**, mesmo com a ata publicada. Informações também podem ser editadas e excluídas. A presença registrada libera sugestões de revisão dos chefes participantes. Ao salvar, as seções **Participantes** e **Informações** são atualizadas automaticamente na ata, com Informações abaixo de Novas ações. As demais edições do editor são salvas junto dos registros; bases desatualizadas são recusadas e a transação é desfeita. Alterações em atas publicadas preservam as versões anteriores. Os nomes e seções dos participantes já registrados são preservados, inclusive quando a conta fica inativa.

A ordem dos participantes segue a seleção: cada caixa marcada recebe uma numeração. Desmarcar e marcar novamente move a pessoa para o final. A migração 17 mantém a ordem alfabética exibida anteriormente para presenças legadas, pois a ordem original dos cliques não foi armazenada.

Em **Configuração**, a gestão cadastra o texto do cabeçalho e o brasão da unidade, à direita. O brasão da PMBA (`public/imagens/brasao_PMBA.png`, copiado da arte fornecida no projeto) fica à esquerda. O cabeçalho é institucional e vale para a visualização e os PDFs das atas. A imagem enviada é convertida para PNG no navegador, reduzida para até 360 pixels e validada pelo servidor; texto e PNG ficam na tabela `config`. A remoção do brasão direito também é feita nessa tela.

Depois da publicação, chefes e suplentes registrados como participantes encontram **Sugerir revisão** na tela da ata. Podem solicitar correção, inclusão ou supressão. A gestão revisa o editor da ata e usa **Revisar e responder**. Ao acolher a sugestão, o texto do editor e a resposta são salvos na mesma transação. Uma sugestão não acolhida também recebe justificativa. A sugestão fica visível com autor, data, resultado e resposta. Cada alteração efetiva da ata publicada preserva a versão anterior; o histórico fica disponível na mesma tela. Edições concorrentes com uma base desatualizada são recusadas.

Reunião antecipada

O aplicativo separa **data real** (`reunioes.data`) e **referência semanal / data prevista** (`reunioes.semana`). Uma reunião de terça realizada na segunda usa os relatos da terça, registra a segunda como data real e mantém a terça seguinte como próxima reunião. A antecipação é explicitada na abertura e na ata. Ela não altera o dia regular da configuração, nem os prazos das ações. As ações concluídas são

selecionadas pela data de conclusão no fuso da Bahia, nos sete dias que antecedem a referência semanal, independentemente do prazo, arquivamento ou encerramento.

Para uma antecipação planejada, recomenda-se um próximo ajuste específico de agenda: cadastrar uma exceção por referência semanal com data/hora real e prazo de envio dos relatos. O painel, o trilho da semana e os lembretes passariam a exibir essa exceção. A interface deve avisar os chefes antes da reunião e destacar os relatos ainda pendentes, permitindo que a gestão prossiga conscientemente. O encerramento pode congelar os cartões e relatos efetivamente apresentados, para que envios posteriores não alterem o registro daquela sessão.

Também se recomenda alertar quando já existe reunião encerrada para a mesma referência semanal e oferecer uma reunião complementar de forma explícita. Essa regra evita iniciar acidentalmente outra reunião na data originalmente prevista sem bloquear sessões extraordinárias legítimas. A agenda de exceções, o aviso de reunião complementar e o congelamento integral dos cartões são propostas para evolução; não são implementados neste ajuste.

Demanda do Diretor em ação já cadastrada

Uma ação já cadastrada pode receber ou perder a marcação **Demanda do Diretor** pelo botão **Editar ação**, respeitando a autoria existente (titular também edita as do suplente; Administrador edita qualquer seção). A marcação pode mudar após apresentação em reunião sem alterar o título, o detalhamento ou a ata já publicada. A alteração fica nos comentários do histórico. Ações encerradas e a origem automática de uma diretriz não mudam. Ao marcar uma ação interna, o formulário informa que ela ficará visível ao Diretor; desmarcar a origem não a torna interna novamente.

Implantação

A versão nova exige as migrações **16 e 17**. SQLite aplica as migrações na inicialização. Para PostgreSQL, configure `SGC_MIGRATION_DATABASE_URL` e execute `npm run migrate -- --postgres` junto da publicação do código novo. Não execute o código antigo após migrar: ele recusa uma versão desconhecida. As tabelas são acessadas pela API Express; RLS e revogações impedem acesso direto pelos papéis públicos do Supabase.

Ensaio do Modo Reunião na TV

Roteiro para testar o Agilis na sala, com o notebook e a TV reais, antes da primeira reunião. Leva cerca de 30 minutos, com duas pessoas: uma no notebook e outra sentada no lugar mais distante da TV.

Use dados fictícios. Não leve dados reais para este ensaio.

Antes de ir à sala

1. No notebook, na pasta do projeto, prepare a demonstração com a data da reunião (PowerShell):

   ```
   $env:SGC_AUTH_MODE = "demo"; $env:DATABASE_URL = ""; $env:SGC_NOW = "2026-09-22T09:30:00"
   npm run seed
   npm start
   ```

   Abra http://127.0.0.1:3000 e confirme que a tela de entrada aparece. Mantenha essa janela do terminal aberta.

2. Anote o modelo da TV, a resolução em que ela aparece no Windows e o navegador que será usado na reunião (o sistema só foi verificado no Edge e no Chromium).

3. Leve o cabo HDMI e, se houver, um adaptador para o notebook.

Na sala

Ligação e tela

- ☐ TV e notebook conectados; modo "Duplicar" ou "Somente segunda tela" escolhido, com resolução 1920x1080 (ou a nativa da TV).
- ☐ Suspensão de tela, protetor de tela e notificações desligados no notebook (Windows: Configurações > Sistema > Energia; Foco e notificações).
- ☐ Navegador em tela cheia (F11), zoom em 100% (Ctrl+0).
- ☐ Entrar como **Diretor (demonstração)** e clicar em **Iniciar reunião** no menu. A tela de conferência deve aparecer sem abrir a reunião; só o botão **Iniciar reunião** cria.

Legibilidade (peça a quem está sentado no fundo da sala)

- ☐ Ler em voz alta o título de cada tela, o nome dos Centros e os prazos das ações.
- ☐ Ler os textos menores: os rótulos "Feito", "Próximo", "Impedimentos", "Apoio necessário", "Ações da seção", o subtítulo da Visão geral e a dica "Tecla → para começar".
- ☐ Distinguir os marcadores do topo: quadrado (crítico), triângulo (atenção) e círculo (em dia). Perguntar se a diferença aparece sem depender da cor.

- ☐ Verificar se o vermelho, o amarelo e o verde se distinguem com a luz da sala acesa e apagada, e se algum fica lavado ou brilhante demais.
- ☐ Verificar o contraste do texto cinza-esverdeado sobre o fundo escuro (nomes dos chefes, "Nada informado").

Se algum texto não for lido do fundo da sala, anote qual e a distância. É com isso que se ajustam os tamanhos.

Condução da reunião

Percorra este roteiro como numa reunião real.

1. ☐ Abertura com os combinados: aparecem e cabem na tela?
2. ☐ Tecla → avança até a Visão geral e depois para a primeira seção; ← volta; **Esc** volta à Visão geral. As setas não funcionam com uma janela de formulário aberta (é o esperado).
3. ☐ Em uma seção com pedido de novo prazo, **Aprovar** e **Recusar** funcionam e o resultado aparece na hora.
4. ☐ **Registrar decisão** e **Nova ação**: a janela é legível, dá para digitar sem o mouse e o texto digitado aparece grande o bastante.
5. ☐ **Exibir tempo (aparece na TV)** mostra e oculta o tempo gasto. Deve começar oculto.
6. ☐ Percorrer todas as seções (6 na demonstração). Anotar o tempo por seção e se algum conteúdo fica cortado ou exige rolagem.
7. ☐ **Sair sem encerrar**, voltar por **Iniciar reunião** e **Retomar reunião**: a reunião continua de onde estava, com as decisões e ações registradas.
8. ☐ **Encerrar reunião**: a ata é montada em rascunho. Revisar, editar e **Enviar aos chefes**.

Falhas e plano B

- ☐ Recarregar a página (F5) no meio da reunião: a reunião é retomada, sem repetir a abertura.
- ☐ Desconectar e reconectar o cabo HDMI: a imagem volta sem perder o estado.
- ☐ Abrir **Painel da semana > Versão para impressão** e imprimir (ou salvar em PDF): serve de plano B se a TV falhar.
- ☐ Deixar o sistema aberto por 10 minutos sem tocar e confirmar que a tela não apaga nem pede novo login.

O que registrar

Item	Resultado (OK / ajustar)	Observação
TV e resolução usadas		
Textos ilegíveis do fundo da sala		
Marcadores distinguíveis sem cor		
Cores com a luz acesa e apagada		
Setas do teclado e Esc		
Janelas de decisão e nova ação		
Tempo por seção		

Item	Resultado (OK / ajustar)	Observação
Cortes ou rolagem inesperada		
Retomada após F5 e após cabo HDMI		
Plano B impresso		

Guarde uma foto da TV com cada tela principal (abertura, Visão geral, uma seção) para comparar depois dos ajustes.

Depois do ensaio

Envie as anotações. Ajustes de tamanho, cor e disposição são rápidos e ficam no arquivo `public/estilo.css`, na seção "Modo Reunião (TV)".

Estabilização do SGC/Agilis

Branch: `codex/estabilizacao-sgc`.

Primeira entrega — persistência PostgreSQL e tratamento de falhas

- [x] Registrar a referência: 18 testes existentes aprovados.
- [x] Vincular consultas ao cliente que abriu a transação PostgreSQL.
- [x] Isolar o contexto de transações simultâneas e consultas externas.
- [x] Garantir rollback e liberação do cliente em caso de erro; descartar o cliente se o rollback falhar.
- [x] Rejeitar transações aninhadas explicitamente, sem adquirir outra conexão.
- [x] Corrigir a comparação do pai nas consultas de criação e reordenação de seções.
- [x] Encaminhar falhas assíncronas na identificação do usuário ao tratamento de erros da API.
- [x] Propagar falhas de carregamento inicial das seções e eliminar a inicialização duplicada sem espera.
- [x] Executar 26 testes, incluindo oito novos casos de regressão.

O adaptador PostgreSQL usa contexto assíncrono por transação. A execução de cada consulta escolhe o cliente da transação ativa. Na segunda entrega, a inicialização passou a verificar a conexão com `select 1`, pois a árvore de seções deixou de usar cache local.

Evidência e limites

Comando: `npm test`.

- 18 testes originais de integração da API aprovados.
- 6 testes do adaptador PostgreSQL aprovados: confirmação, rollback, isolamento concorrente, descarte de conexão, erro de inicialização e transação aninhada.
- 2 testes adicionais de API aprovados: falha de identificação e estrutura com pais nulos/não nulos.

Os testes PostgreSQL usam um pool instrumentado, sem conexão externa. Eles verificam o encaminhamento das consultas e os comandos de controle, mas não substituem a integração com um servidor PostgreSQL real. O teste de estrutura usa SQLite e rejeita a sintaxe incompatível identificada; não certifica todas as consultas PostgreSQL.

Nenhum banco existente foi alterado. Não houve implantação ou mudança de permissões no Supabase. O modo de demonstração continua sem autenticação real.

Segunda entrega — SQLite, privacidade, arquivamento e datas

- [x] Serializar transações SQLite e operações externas, inclusive leituras, na conexão local.

- ☑ Padronizar `db.transaction(callback)` como Promise nos dois bancos; todas as consultas e a carga de demonstração devem ser aguardadas.
- ☑ Garantir rollback do seed e impedir sua execução por CLI com `DATABASE_URL` preenchida antes de qualquer exclusão local.
- ☑ Consultar a hierarquia diretamente no banco, removendo o cache por instância das permissões.
- ☑ Ocultar pedidos internos na pauta, cartões, detalhes de reunião e geração de novas atas.
- ☑ Recusar decisões de pedidos não visíveis, preservando a exceção de compartilhamento explícito.
- ☑ Retirar ações arquivadas e seus pedidos dos indicadores/listas operacionais, preservando histórico e tempo registrado.
- ☑ Permitir desativar seções com ações apenas arquivadas; exigir reativação antes de restaurar ou criar ações.
- ☑ Rejeitar datas inexistentes, incluindo fevereiro e anos bissextos.
- ☑ Executar 36 testes com sucesso e verificar o CLI de seed em arquivo temporário.

A fila SQLite protege as operações na mesma conexão da aplicação. Isso não substitui as futuras constraints e validações transacionais de negócio contra disputas entre requisições ou processos. A consulta da árvore é atualizada a cada uso; o custo adicional deverá ser medido na etapa de desempenho.

Atas já persistidas não foram reescritas. Se houver atas antigas com conteúdo interno indevido, será necessário revisar os dados em uma etapa própria, preservando a trilha de auditoria.

A validação PostgreSQL continua usando pool simulado. Não foram encontrados PostgreSQL, `psql` ou Docker instalados no ambiente. Nenhum banco remoto foi acessado. As alterações continuam locais, sem implantação.

Terceira entrega — concorrência de pedidos/atualizações e semáforo

- ☑ Revalidar pedidos dentro da transação antes de inserir, evitando duplicidades por chamadas simultâneas na API.
- ☑ Usar a mesma trava de ação (`FOR UPDATE`) nos fluxos de criar/decidir pedidos em PostgreSQL; SQLite usa sua fila transacional.
- ☑ Atualizar decisões apenas enquanto o pedido estiver pendente e confirmar o prazo na mesma transação.
- ☑ Validar `aprovar` como booleano, sem interpretar a string `"false"` como aprovação.
- ☑ Impedir aprovação de pedido baseado em prazo antigo; permitir recusá-lo para liberar um novo pedido.
- ☑ Validar e travar a reunião informada ao decidir por delegação; uma reunião encerrada gera conflito.
- ☑ Serializar a escolha da versão semanal por seção; a resposta retorna o conteúdo salvo pela própria chamada.
- ☑ Propagar impedimentos críticos da última atualização das subseções ao Centro, mantendo o indicador de envio do próprio Centro.

- [x] Executar 41 testes com sucesso.

Novos testes exercitam oito pedidos simultâneos (um criado e sete conflitos), duas decisões concorrentes (uma aceita), seis atualizações com versões distintas, rollback após falha de gravação do prazo, validação de decisão e propagação/resolução do impedimento da subseção.

Limites: a concorrência foi exercitada na API com SQLite em memória. As travas PostgreSQL ainda exigem validação com banco real e múltiplas conexões. Unicidade por constraints, migrações e concorrência do ciclo de reuniões continuam pendentes; estas alterações não certificam toda operação concorrente do sistema.

Quarta entrega — migrações versionadas e ciclo de reuniões

- [x] Adicionar migrador incremental com registro de versões e rollback de todas as alterações pendentes em caso de falha.
- [x] Exigir destino explícito no CLI e uma variável própria para migração PostgreSQL.
- [x] Verificar a versão PostgreSQL na inicialização, sem DDL automático; aplicar migrações antes do uso de SQLite.
- [x] Substituir a alteração silenciosa da coluna de arquivamento por migração rastreável.
- [x] Impor unicidade de pedido pendente por ação e de reunião em andamento por índices parciais.
- [x] Recusar migração de dados duplicados com diagnóstico e sem excluí-los.
- [x] Serializar abertura/reabertura; proteger decisões, ações, encerramento, revisão e envio da ata por transações e travas.
- [x] Validar que ações/decisões concorrentes ao encerramento entram na ata ou são recusadas.
- [x] Executar 52 testes com sucesso e verificar o CLI em SQLite temporário.

O procedimento está em `MIGRACOES.md`. A implantação PostgreSQL agora exige migração prévia; a versão não inicia com migrações pendentes. Não foi aplicado DDL a bancos existentes do usuário nem a serviços remotos. PostgreSQL real e testes entre instâncias continuam pendentes, assim como o alinhamento das demais constraints e das exclusões em cascata.

O push do commit anterior foi adiado pelo usuário. Esta entrega permanece local e sem novo commit.

Quinta entrega — calendário, fuso e configuração atômica

- [x] Compartilhar o calendário de `America/Bahia` entre API e interface, sem depender do fuso do servidor/navegador.
- [x] Interpretar `SGC_NOW` e timestamps antigos sem offset no fuso de Bahia; preservar timestamps com offset explícito.
- [x] Validar toda a configuração antes de gravar e salvar as chaves na mesma transação.
- [x] Atualizar o bootstrap completo após configuração e recarregá-lo nas navegações (respeitando o cache de cinco segundos existente).
- [x] Preservar a consulta de semanas registradas antes da mudança do dia da reunião.
- [x] Corrigir os textos fixos de terça-feira e o fechamento no sábado para reuniões de domingo.
- [x] Informar na interface que correções continuam disponíveis após o prazo regular.

- [x] Executar 57 testes com sucesso, incluindo processos com `TZ=UTC`, `America/Los_Angeles` e `Asia/Tokyo`.

O corte da referência semanal permanece às 12h no dia configurado, inclusive para reuniões agendadas à tarde. Essa regra ainda precisa de definição de produto; não foi alterada silenciosamente. O fechamento continua informativo, sem bloqueio de correções no backend. A leitura de semanas antigas foi preservada, mas a edição continua seguindo o dia atualmente configurado.

Os testes verificam funções de apresentação e calendário, além da API. A inspeção visual em navegador não foi executada porque a ferramenta de controle de navegador não está disponível nesta sessão. Dados antigos não foram reescritos; timestamps sem offset são interpretados como horários locais de Bahia.

Sexta entrega — integração PostgreSQL real

- [x] Criar `npm run test:postgres` com cluster descartável, conexão por socket local e bancos independentes por cenário.
- [x] Validar migrações concorrentes, rollback de DDL, diagnóstico de duplicidades e índices únicos em PostgreSQL 17.9.
- [x] Exercitar duas instâncias da API com pools independentes: pedidos, decisões, versões semanais e ciclo de reuniões.
- [x] Provocar falha real de escrita via trigger e confirmar rollback integral da decisão/prazo.
- [x] Confirmar que ações/decisões concorrentes ao encerramento entram na ata ou são recusadas e que o envio preserva a ata publicada.
- [x] Corrigir identities de seções e usuários após os IDs explícitos da carga fictícia; validar constraints adiadas antes de reiniciar as identities na mesma transação.
- [x] Passar nos 11 cenários PostgreSQL (12 testes contando o agrupador) e nos 57 testes de regressão.

Reprodução e limites em `TESTES_POSTGRESQL.md`. O ambiente foi local e descartável; bancos existentes e Supabase não foram alterados. Os binários PostgreSQL ficam fora das dependências da aplicação. As APIs são instâncias no mesmo processo Node, com pools independentes; múltiplos processos, TLS, RLS e permissões remotas não foram certificados. Alterações seguem locais, sem novo commit ou push.

Sétima entrega — integridade referencial entre bancos

- [x] Adicionar migração 3, sem alterar as versões anteriores, para as diferenças de FKs identificadas nos esquemas.
- [x] Exigir chefe existente no SQLite, com FK adiada até o commit, como PostgreSQL.
- [x] Substituir quatro cascatas PostgreSQL por exclusão explícita, mantendo o comportamento da API e a proteção existente no SQLite.
- [x] Preservar registros, índices, views e triggers na reconstrução SQLite; validar referências antes do commit e reativar FKs após sucesso/falha.
- [x] Testar migração legada, referências órfãs, rollback de DDL, reabertura de arquivo e consultas externas durante a reconstrução.

- [x] Comparar destinos e ações de todas as FKs SQLite/PostgreSQL e testar exclusões diretas e pela API.
- [x] Passar nos 63 testes de regressão e nos 15 cenários PostgreSQL (16 testes com o agrupador).

Procedimento e impacto sobre scripts externos em `MIGRACOES.md`. Nenhum banco existente do usuário foi migrado. Novas regras de domínio e novos vínculos ausentes em ambos os esquemas permanecem fora desta entrega; as diferenças identificadas entre as FKs existentes foram alinhadas. Alterações seguem locais, sem novo commit ou push.

Oitava entrega — acesso institucional e permissões

- [x] Integrar Microsoft Entra ID, escolhido pelo usuário, com código de autorização, PKCE, state, nonce e verificação criptográfica dos ID tokens.
- [x] Vincular Tenant ID + Object ID a usuário local por comando administrativo, sem criar permissões por e-mail ou claims do navegador.
- [x] Persistir sessões por hash, usar cookie HttpOnly/Secure e exigir Origin + CSRF em mutações; revogar sessão no logout.
- [x] Remover confiança em `x-user-id` no modo institucional; exigir demo explícita apenas fora de produção e escutar demo em loopback.
- [x] Adicionar migração 4 com tabelas privadas de autenticação, RLS e revogação de grants públicos PostgreSQL, inclusive por coluna/sequência/função.
- [x] Corrigir exportador e SQL de carga: destino vazio, transação integral, sem truncamento ou concessão pública e sem exportar sessões/vínculos.
- [x] Exigir TLS PostgreSQL verificado e permitir CA oficial configurável.
- [x] Atualizar entrada da interface e impedir que respostas da sessão anterior repovoem cache/CSRF.
- [x] Validar 73 testes gerais e 19 cenários PostgreSQL (21 testes com agrupadores).

Configuração e operação em `AUTENTICACAO.md`. Node mínimo atualizado para 22.12 para a biblioteca de validação de tokens. Registro real no Entra, Tenant/Client IDs, segredo no ambiente seguro, URL pública e vínculos administrativos ainda precisam ser configurados para homologação. O login real com MFA e a inspeção visual não foram realizados; a ferramenta de navegador não estava disponível. A autorização de perfil/propriedade existente nas rotas foi preservada; não houve reescrita completa dessa camada. Não foram alterados serviços remotos, dados do usuário ou permissões efetivas do Supabase. Alterações permanecem locais, sem commit ou push.

Nona entrega — preparação da homologação institucional

- [x] Adicionar `npm run check:auth`, diagnóstico local sem conexão ao banco, alteração de ambiente ou exposição de segredos.
- [x] Adicionar opção explícita `--online` para comparar a descoberta pública Microsoft com os endpoints esperados.
- [x] Testar configurações ausentes/inválidas, restrição ao endpoint oficial, respostas divergentes e sigilo dos erros.

- ☑ Documentar resultados esperados para login, MFA, perfis, acesso negado, revogação e expiração.

O diagnóstico local confirmou ausência de `SGC_PUBLIC_ORIGIN`, `ENTRA_TENANT_ID`, `ENTRA_CLIENT_ID` e `ENTRA_CLIENT_SECRET`. Nenhuma consulta ao tenant real ou alteração administrativa foi realizada. A continuação da homologação exige o registro institucional e essas configurações; o segredo deve ficar no ambiente seguro, não na conversa. Suíte geral atualizada para 77 testes. PostgreSQL e suas migrações não foram alterados nesta entrega.

Décima entrega — versionamento, cabeçalhos de segurança e continuidade do trabalho na interface

- ☑ Inicializar o repositório git (branch `main`) com o estado das nove entregas anteriores; `.env.example` deixou de ser ignorado.
- ☑ Atualizar `better-sqlite3` para 12.x, que instala sem compilação no Node 24/Windows; a suíte voltou a rodar aqui.
- ☑ Enviar CSP restritiva, `nosniff`, `X-Frame-Options`, `Permissions-Policy` e HSTS (só com HTTPS); ativar `trust proxy` na Vercel ou por `SGC_TRUST_PROXY`, para o limite de login enxergar o IP real.
- ☑ Hospedar Manrope e DM Sans em `public/fontes`; nenhuma URL externa em HTML ou CSS.
- ☑ Migração 7: sessão deslizante (1 h de inatividade, teto de 8 h); Modo Reunião renova a sessão a cada 10 minutos.
- ☑ Interface: aviso de sessão expirada com retorno à tela anterior; rascunho da atualização semanal em `sessionStorage`; Minhas ações recarrega só os dados (mantém a aba e os minutos digitados); exclusão de ação usa o diálogo padrão.
- ☑ 102 testes gerais aprovados. Verificação no Edge (CDP), com servidor em modo demo: fontes locais carregadas, zero violações de CSP no console, aba e minutos preservados, rascunho recuperado após recarregar, telas do chefe (390 px) e do Diretor (1920x1080) conferidas visualmente.

Limites: os testes de PostgreSQL real (`npm run test:postgres`) não foram executados aqui (sem binários); as expectativas de versão em `test/postgres/integracao.test.js` foram atualizadas, mas a migração 7 em PostgreSQL ainda precisa ser confirmada nesse ambiente. A expiração real de sessão na interface (aviso e retorno) foi verificada só por leitura do código e pelos testes de servidor. O ensaio na TV/HDMI real continua pendente.

Décima primeira entrega — início explícito da reunião e regras compartilhadas

- ☑ Abrir `#/reuniao` deixou de criar reunião: a rota mostra uma tela de conferência e só o botão "Iniciar reunião" cria; reunião em andamento é retomada por `#/reuniao/:id`, e reuniões já encerradas redirecionam para a ata. A abertura com combinados não se repete ao recarregar.
- ☑ Corrigido: reunião iniciada no próprio dia depois das 12h (por exemplo terça às 14h30) apontava para a semana seguinte e mostrava cartões vazios; agora trata da semana de hoje. O

corte das 12h continua valendo só para o painel. Teste confirmado contra a versão anterior (falhava com `2026-09-29`).
- [x] `public/js/regras.js` passou a ser a fonte única de status, transições, prioridades, ordem do semáforo e hora de fechamento; servidor e interface deixaram de ter cópias. `test/regras.test.js` detecta divergência de rótulos e transições.
- [x] Textos do prazo de atualização passaram a usar a hora de fechamento compartilhada e o dia real (antes diziam "segunda" fixo).
- [x] 108 testes gerais aprovados; fluxo verificado no Edge (início, retomada, recarga sem repetir abertura, reunião inexistente com erro amigável).

Pendências vistas na inspeção: na TV, o subtítulo da Visão geral e o contador do topo (seções, decisões, ações) ficam pequenos demais para a sala; revisar no ensaio com a TV real.

Décima segunda entrega — divisão do `app.js` por assunto

- [x] `server/app.js` (700 linhas) virou uma montagem de 49 linhas; as rotas foram para `rotas-estrutura.js`, `rotas-acoes.js` e `rotas-semana.js`, com auxiliares comuns em `contexto.js`. A divisão foi feita por fatiamento das linhas originais, sem reescrever a lógica.
- [x] Sem mudança de comportamento: as 52 rotas antigas e novas são idênticas (comparação automática do método e do caminho) e a ordem relativa entre rotas de mesmo prefixo foi preservada.
- [x] Verificação estática (ESLint `no-undef` e `no-unused-vars`) sem erros nos arquivos do servidor; `test/rotas.test.js` fixa o inventário de rotas.
- [x] 109 testes gerais aprovados.

Limite: o ESLint foi executado uma vez, fora do projeto, e não faz parte de `npm test`.

Décima terceira entrega — refino visual (cartão de ação do chefe e Modo Reunião)

- [x] Cartão de ação do chefe reorganizado em camadas: título e linha de prazo; selos de estado (atrasada, prioridade, demandada, pedido enviado, aguardando aceite); status e tempo como ações principais; "Detalhes e histórico" e "Pedir novo prazo" como ações secundárias; Arquivar, Desarquivar e Excluir recolhidos em "Mais". A prioridade saiu do cartão como campo editável e continua alterável em "Detalhes e histórico". Ação atrasada ganha faixa vermelha, como nos cartões do painel. Sem emojis.
- [x] Modo Reunião: o corpo da tela deixou de ficar preso em 14 px e passou a acompanhar a escala da TV. Menor texto efetivo: em 1920x1080, de 14 px para 19,9 px; em 1366x768, de 11,3 px (rótulos das caixas) para 14,1 px. Legendas, dicas e tempo subiram para 0,9 a 0,95 rem.
- [x] Marcadores de seção da TV: antes viravam cápsulas inclinadas (o estilo genérico dos botões sobrescrevia o losango) e só se distinguiam pela cor; agora têm forma própria (quadrado = crítico, triângulo = atenção, círculo = em dia), coerente com a regra de semáforo com forma e texto.
- [x] "Em andamento" deixou de quebrar em duas linhas na lista de ações da TV, e ações atrasadas não deslocam mais o texto. Removido o preload de fonte, que gerava aviso no console.

- ☑ Verificado no Edge (CDP), com dados de demonstração: comparação antes/depois em 390 px e 1280 px (chefe) e 1920x1080 e 1366x768 (TV); aba e minutos digitados continuam preservados; sem avisos de console. 109 testes gerais aprovados.

Limites: ainda não foi feito o ensaio na TV/HDMI real; a legibilidade a distância só se confirma lá. A reorganização do cartão não foi validada com um chefe de verdade.

Décima quarta entrega — perfil Administrador e senha inicial provisória

Decisões do responsável pelo projeto: o Administrador só consulta e cadastra (contas, perfis e estrutura); o Diretor continua cadastrando Chefe e Apoio; a senha inicial é provisória, com troca obrigatória no primeiro acesso; o primeiro Administrador nasce por comando no servidor.

- ☑ Migração 8 (`perfil_administrador`): novo perfil e `usuarios.trocar_senha`. No SQLite reconstrói `usuarios` (testada com banco anterior: dados, chaves, índices e views preservados, sem referências órfãs); no PostgreSQL recria a restrição de perfil.
- ☑ Administrador consulta painel, pauta, ações (inclusive internas), pedidos de prazo, reuniões, atas e cadastros; toda escrita fora de `/usuarios`, `/secoes` e `/auth` é recusada no servidor, além da permissão de cada rota. Telas de acompanhamento ficam somente para leitura (detalhe da ação, combinados, pedidos de prazo, atas); o menu não oferece Direcionar ação nem Iniciar reunião.
- ☑ Contas: o Administrador cria Diretor, Apoio e Chefe; o Diretor, Chefe e Apoio. Só o Administrador mexe em Diretor e Administrador; nenhuma rota cria ou promove Administrador; o último Diretor ativo é protegido. Redefinir a senha de alguém, ou criar a conta, exige troca no próximo acesso.
- ☑ Troca de senha: `POST /api/auth/trocar-senha` (confirma a senha atual, grava pela API administrativa, encerra as outras sessões), tela obrigatória "Crie a sua senha" sem menu, e botão "Alterar senha" no menu.
- ☑ `npm run admin:criar` cria o primeiro Administrador (conta nova com senha provisória pelo ambiente, ou vínculo de conta existente). Usuário Administrador incluído nos dados de demonstração.
- ☑ Chefe de Seção confirmado por teste: vê a própria seção e as subordinadas, e só elas.
- ☑ 118 testes gerais aprovados. Verificação no Edge: Administrador em modo demo (menu, leitura, redirecionamentos, estrutura) e fluxo institucional com provedor simulado (cadastro de Diretor pelo Administrador, primeiro acesso, troca com mensagens de erro, senha provisória invalidada).

Limites: a migração 8 foi validada depois, em PostgreSQL 18.4 real (veja a décima quinta entrega); a suíte `npm run test:postgres` não roda no Windows e teve só as expectativas de versão atualizadas. O fluxo de senha foi verificado com Supabase simulado, não com o projeto real; a API administrativa do Supabase precisa aceitar a atualização de senha por `PUT /auth/v1/admin/users/{id}`. O Administrador enxerga as ações internas das subseções por decisão explícita ("acessa todos os dados"): isso amplia o que o Diretor vê hoje.

Décima quinta entrega — migração 8 validada em PostgreSQL real e roteiro de homologação

- ☑ Migração 8 executada num PostgreSQL 18.4 real e descartável (binários do pacote `embedded-postgres`, TCP local, fora do projeto), partindo do esquema **antigo** com usuários, seção e chefia: restrição de perfil recriada, coluna `trocar_senha` criada (não nula, padrão 0), dados e chefia preservados, perfil inválido continua recusado, reexecução não altera nada, `verificarMigracoes` aceita a versão, RLS de `usuarios` segue ligada, e a lógica do comando `admin:criar` cria o Administrador e o vínculo. Também confirmado que, partindo do esquema **novo** (`schema.sql`), as migrações não conflitam (uma única restrição de perfil).
- ☑ `scripts/diagnostico-supabase.sql`: diagnóstico somente leitura para um banco Agilis já existente (versões aplicadas, perfis, dados que bloqueiam a migração, volume, RLS e permissões), sem nomes nem e-mails no resultado; testado em PostgreSQL 18.4 local em banco vazio, antigo com dados problemáticos e migrado. `HOMOLOGACAO.md` ganhou a etapa 3.1 para banco existente, com a tabela de interpretação.
- ☑ `HOMOLOGACAO.md`: roteiro para o projeto Supabase real (preparo, ambiente, banco, primeiro Administrador, 36 verificações em cinco blocos (administrador, contas, permissões, senhas e segurança) mais a etapa da Vercel, diagnóstico e encerramento).

Limites: o teste foi num PostgreSQL local, não no gerenciado pelo Supabase (papéis, RLS de `anon` / `authenticated` e o pooler só se provam no roteiro de homologação). A suíte oficial `npm run test:postgres` continua exigindo Linux ou macOS.

Próximas entregas

1. Ampliar a integração para implantação com múltiplos processos, conexão de produção e cópia isolada dos dados reais quando houver ambiente destinado a isso.
2. Avaliar novas constraints de domínio e vínculos ainda ausentes nos dois esquemas, com diagnóstico dos dados legados.
3. Homologar Entra ID com contas institucionais, revisar permissões efetivas do Supabase e ampliar a auditoria de autorização das rotas.
4. Formalizar o corte da semana, fechamento e exceções para correções.
5. Corrigir atualização de estado, proteção de formulários e concorrência de respostas na interface.
6. Validar jornadas em navegador, desempenho, implantação, backup e restauração.

Decisões de produto pendentes

- Provedor definido: Supabase Auth (e-mail e senha), substituindo a integração Entra ID inicial. Configuração do projeto e vínculos pendentes.
- Política de correções após o fechamento semanal.
- Momento de mudança da semana de referência para reuniões com horário configurável.
- Efeito da falta de atualização de uma subseção no semáforo do Centro.

Essas decisões não bloqueiam as correções técnicas independentes. A liberação para dados reais depende de identidade validada, acesso restrito, operações atômicas e restauração comprovada.

Publicação do Agilis — 05/10/2026

Publicação concluída no projeto Vercel `agilis`, em agilis-alpha.vercel.app. Data de referência no fuso `America/Bahia`.

Deployment ativo: `dpl_EXZaHHJkBUbJfAaya43hA1DqsS3P`, inspeção na Vercel. A publicação utilizou uma cópia dos 73 arquivos de execução (`public`, `server`, `certs` e configuração), conferida contra o projeto local. Arquivos de ambiente, dados locais e configurações pessoais ficaram fora do envio.

Manifesto SHA-256: `d1a77be51ed97175f11c7b64658658274899ac861e75bd4dd988591767bc9da2`.

Banco

A migração **16**, `registros_e_revisao_de_reuniao`, foi aplicada pelo migrador da aplicação ao projeto Supabase `kbepvsnkmehgcnebkijy`, com TLS verificado. O deployment foi construído antes da migração e promovido em seguida.

- Antes e depois: **36 ações**, **2 reuniões**, nenhuma reunião em andamento.
- As quatro tabelas novas existem, têm RLS habilitada e não concedem leitura direta aos papéis `anon` ou `authenticated`.
- A verificação das versões da aplicação passou após a migração.

Verificações da publicação

Conferência	Resultado
Página inicial	HTTP 200
`/api/auth/config`	HTTP 200, modo `supabase`
`/api/bootstrap` sem sessão	HTTP 401
Inclusão de informação de reunião sem sessão	HTTP 401, nenhuma alteração
Arquivo inexistente	HTTP 404
JavaScript de login, reunião, atas, revisão, ação e seção; CSS	HTTP 200 e conteúdo idêntico ao código testado
Deployment resolvido pelo endereço público	Nova versão em estado `READY`

Antes da publicação passaram **182 testes automatizados** e o ensaio de navegador em 390×844 e 1920×1080, com dados fictícios. Foram exercitados login e olhinho, demanda declarada pela seção, edição do checklist, presença, informações para a ata, conclusão com prazo futuro e revisão da ata publicada. A suíte de integração PostgreSQL em cluster temporário não iniciou neste Windows: ela exige o ambiente Unix documentado em `TESTES_POSTGRESQL.md`. A migração e as permissões foram verificadas no PostgreSQL de produção. O teste remoto não realizou login com senha de usuário nem criou registros de negócio.

Avisos existentes do Supabase

A consulta aos advisors retornou avisos sobre objetos e configurações que esta publicação não criou. Foram registrados para acompanhamento; não foram alteradas configurações de autenticação ou funções auxiliares fora do escopo.

- **RLS habilitada sem policies:** informativo compatível com a arquitetura atual, que revoga acesso direto e usa a API Express. A proteção das tabelas novas foi confirmada no catálogo. Referência do aviso.
- **`public.subarvore` sem `search_path` fixo:** o catálogo confirma `SECURITY INVOKER` e ausência de permissão de execução para `anon` e `authenticated`. A fixação do caminho fica para uma migração específica. Referência do aviso.
- **`public.rls_auto_enable` com grants de execução:** a função preexistente é `SECURITY DEFINER`, retorna `event_trigger` e fixa `search_path=pg_catalog`. Revisar esses grants em uma manutenção própria. Aviso para `anon`, aviso para `authenticated`.
- **Proteção contra senhas vazadas desabilitada:** configuração existente do Supabase Auth. Orientações oficiais.

Os detalhes dos ajustes e a proposta de evolução da agenda de reuniões antecipadas estão em REUNIOES.md. A publicação inicial foi feita pela CLI a partir do código local. O código, a migração 16 e os módulos novos são versionados juntos para manter compatibilidade com o banco nas futuras publicações pela integração Git.